

# CẤU TRÚC DỮ LIỆU

---



## TÓM TẮT

Phần B đi qua bảy nhóm cấu trúc dữ liệu, nhưng chia theo *thao tác được làm rõ đi* chứ không theo hình dạng. Mảng và hai con trỏ làm rõ việc quét có cửa sổ; ngăn xếp, hàng đợi và heap làm rõ việc lấy phần tử “đúng lúc”; bảng băm làm rõ tra cứu theo khoá; cây tìm kiếm làm rõ tra cứu theo thứ tự; cây chỉ số làm rõ truy vấn trên đoạn kèm cập nhật; DSU làm rõ việc hợp nhất; trie và các cấu trúc chuỗi làm rõ việc so khớp. Đọc xong phần này, khi nhìn một đề bài bạn sẽ hỏi “thao tác nào lặp lại nhiều nhất” thay vì “bài này dùng cây gì”. Nếu chỉ nhớ một điều: cấu trúc dữ liệu là một cách trả trước chi phí — bạn bỏ công sắp đặt trước để mỗi truy vấn sau rõ đi, và mọi cấu trúc đều có một thao tác mà nó làm *tệ hơn* mảng thường.

Có một cách nhìn thống nhất cho cả phần này, và nó đáng nói ngay từ đầu. Mọi cấu trúc dữ liệu đều là một sự đánh đổi giữa *trật tự bạn duy trì* và *giá bạn trả để duy trì nó*. Mảng thô không duy trì trật tự nào nên chèn rõ mà tìm đất. Cây tìm kiếm duy trì trật tự toàn phần nên tìm rõ mà mỗi lần chèn phải trả tiền giữ cân bằng. Heap duy trì một trật tự yếu hơn — chỉ cần biết ai nhỏ nhất — nên rõ hơn cây, đổi lại không trả lời được câu “phần tử thứ  $k$  là ai”. Khi bạn đọc mỗi chương, hãy tự hỏi cấu trúc đó duy trì *đúng bao nhiêu* trật tự, vì đó là thứ quyết định nó nhanh ở đâu và mù ở đâu.

Thứ tự bảy chương không phải ngẫu nhiên. Chương 6 và 7 là những cấu trúc mà bạn dùng gần như mọi ngày, và chúng cũng là nơi các kỹ thuật “ngây thơ nhưng đúng” như hai con trỏ hay ngăn xếp đơn điệu biến  $O(n^2)$  thành  $O(n)$ . Chương 8 và 9 là hai cách tra cứu đối lập — băm cho tra cứu chính xác, cây cho tra cứu theo thứ tự — và ranh giới giữa chúng là câu hỏi “bài có cần thứ tự không”. Chương 10 và 11 là hai cấu trúc mà người luyện thi đấu dùng nhiều nhất vì tỉ lệ sức mạnh trên độ dài code rất cao. Chương 12 khép lại bằng chuỗi, miễn mà cấu trúc dữ liệu chuyên biệt tạo ra khác biệt lớn nhất.

## Bản đồ phần B — Cấu trúc dữ liệu

## TÓM TẮT

Bảy chương của phần B được xếp theo thao tác mà chúng làm rõ đi, và bản đồ này cho bạn bảng tra ngược: gặp thao tác nào thì mở chương nào. Chuỗi phụ thuộc bắt buộc là  $6 \rightarrow 7 \rightarrow 9 \rightarrow 10$ ; chương 8 (băm) và 11 (DSU) độc lập, đọc lúc nào cũng được; chương 12 cần chương 8 cho hashing chuỗi. Nếu chỉ nhớ một điều: chọn cấu trúc bằng cách liệt kê thao tác và tần suất, không bằng cách nhớ tên.

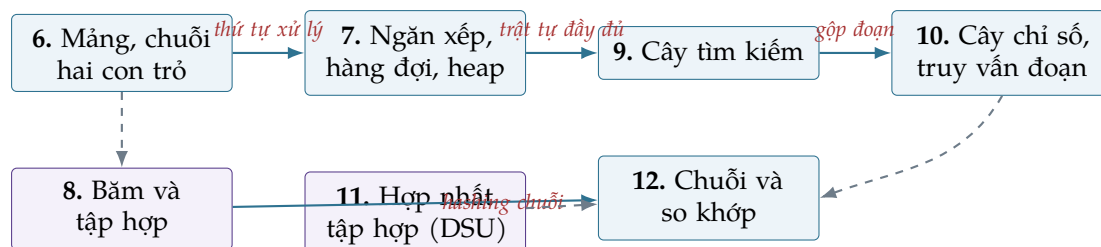
A. Bảng tra ngược: thao tác  $\rightarrow$  chương

| Thao tác lặp lại nhiều             | Chương | Cấu trúc điển hình         |
|------------------------------------|--------|----------------------------|
| Quét cửa sổ, tổng đoạn tĩnh        | 6      | hai con trỏ, tổng tiền tố  |
| Lấy phần tử nhỏ nhất liên tục      | 7      | heap / hàng đợi ưu tiên    |
| “Phần tử lớn hơn kế tiếp”          | 7      | ngăn xếp đơn điệu          |
| Tra cứu chính xác theo khoá        | 8      | bảng băm                   |
| Tra cứu theo thứ tự, cận trên/dưới | 9      | cây tìm kiếm cân bằng      |
| Truy vấn đoạn kèm cập nhật điểm    | 10     | Fenwick / segment tree     |
| Truy vấn đoạn trên mảng tĩnh       | 10     | sparse table               |
| Hợp nhất hai nhóm, hỏi cùng nhóm   | 11     | DSU                        |
| So khớp mẫu, tiền tố chung         | 12     | trie, KMP, Z, Aho–Corasick |

## B. Vì sao chia như vậy

Chia theo hình dạng — “mảng, danh sách liên kết, cây, đồ thị” — là cách chia của người đã biết đáp án. Người đang giải bài không nhìn thấy hình dạng, họ nhìn thấy thao tác: “tôi phải hỏi tổng đoạn này một triệu lần và thỉnh thoảng sửa một phần tử”. Bảng tra ở trên chính là cầu nối từ thao tác về cấu trúc, và nó là thứ đáng thuộc nhất của cả phần B.

## C. Phụ thuộc và thứ tự đọc



**Hình 1.** Phụ thuộc giữa bảy chương của phần B. Hai chương tô tím độc lập với mạch chính; chương 12 là nơi hội tụ của băm, DSU và cây chỉ số.

## D. Cốt lõi hay tham khảo

**Cốt lõi:** hai con trỏ và tổng tiền tố (6); ngăn xếp đơn điệu và heap (7); băm với khoá ngẫu nhiên hoá (8); Fenwick và segment tree cơ bản kèm lazy (10); DSU với hai tối ưu (11); KMP, Z và hashing chuỗi (12). **Tham khảo:** cài đặt chi tiết cây đỏ–đen và treap (9); segment tree beats và cây bền vững (10); suffix automaton (12).

Đọc bằng ví dụ: chương B/06 có vết chạy cửa sổ và phân ví dụ số âm; chương B/10 vẽ các khối lowbit và kiểm thử tự hợp thành lazy.

# Mảng, chuỗi và hai con trỏ (Arrays, Strings, Two Pointers)

**PHỤ THUỘC** A/03 cho ngân sách chỉ phí và phân cấp bộ nhớ; A/04 cho cách viết bất biến — chương này là nơi bất biến trả công nhiều nhất.

## TÓM TẮT

Mảng là cấu trúc dữ liệu bị đánh giá thấp nhất: nó không có “thuật toán” nào hào nhoáng, nhưng nó thắng gần như mọi cấu trúc khác về hằng số và về tính thân thiện với cache. Chương này đi qua bốn kỹ thuật biến  $O(n^2)$  thành  $O(n)$  mà không cần thêm cấu trúc nào: tổng tiền tố, mảng hiệu, hai con trỏ, và cửa sổ trượt. Điểm chung của cả bốn là một ý duy nhất — *đừng tính lại từ đầu, hãy cập nhật phần thay đổi*. Nếu chỉ nhớ một điều: khi thấy hai vòng lặp lồng nhau, hãy hỏi “khi vòng ngoài tiến một bước, vòng trong có thật sự phải quay lại từ đầu không”; câu trả lời thường là không, và đó là toàn bộ chương này.

## KIẾN THỨC CẦN CÓ TRƯỚC

- **mảng (array)** — dãy phần tử cùng kiểu nằm liên tiếp trong bộ nhớ; truy cập phần tử thứ  $i$  tốn  $O(1)$  bằng phép tính địa chỉ.
- **tổng tiền tố (prefix sum)** — mảng  $P$  với  $P_i = a_1 + \dots + a_i$ ; cho phép lấy tổng đoạn  $[l, r]$  bằng  $P_r - P_{l-1}$  trong  $O(1)$ .
- **mảng hiệu (difference array)** — mảng  $D$  với  $D_i = a_i - a_{i-1}$ ; cho phép cộng một hằng số vào cả đoạn trong  $O(1)$ , rồi khôi phục mảng gốc bằng tổng tiền tố.
- **hai con trỏ (two pointers)** — kỹ thuật duy trì hai chỉ số cùng quét qua dữ liệu, mỗi chỉ số chỉ đi tiến, cho tổng chỉ phí  $O(n)$  thay vì  $O(n^2)$ .
- **cửa sổ trượt (sliding window)** — trường hợp riêng của hai con trỏ: duy trì một đoạn liên tiếp thoả điều kiện, mở rộng bên phải và co bên trái.
- **tính đơn điệu** — tính chất “nếu đoạn  $[l, r]$  thoả thì mọi đoạn con của nó cũng thoả” (hoặc chiều ngược lại). Đây là điều kiện sống còn để hai con trỏ đúng.

## CÂU HỎI MỞ ĐẦU

- Q1. Vì sao mảng — cấu trúc thô sơ nhất — lại thường thắng những cấu trúc tinh vi hơn trên máy thật, dù độ phức tạp tiệm cận tệ hơn?
- Q2. Hai con trỏ chạy  $O(n)$  dù trông như hai vòng lặp lồng nhau. Lập luận nào cho ta biết nó thật sự tuyến tính?
- Q3. Kỹ thuật hai con trỏ hỏng khi nào? Cho một bài mà áp dụng nó sẽ cho kết quả sai.
- Q4. Tổng tiền tố và mảng hiệu là hai phép toán ngược nhau. Việc nhận ra điều đó cho bạn thêm kỹ thuật gì?
- Q5. Vì sao “sắp xếp trước” lại là nước đi mở đầu hiệu quả đến vậy, dù bản thân nó tốn  $O(n \log n)$ ?

Có một hiện tượng đáng chú ý trong lập trình thi đấu: rất nhiều bài được gắn nhãn khó lại có lời giải chỉ gồm một vòng lặp và hai biến chỉ số. Không cấu trúc dữ liệu, không đệ quy, không công

thức. Người chưa quen nhìn lời giải đó thấy nó như trò ảo thuật — rõ ràng bài đòi xét mọi đoạn con, mà đây chỉ quét một lượt.

Không có ảo thuật nào. Cả chương này chỉ khai thác một quan sát duy nhất, chính là câu hỏi số 10 trong danh sách heuristic ở chương A/01: *cái gì thực sự thay đổi giữa hai bước liên tiếp?* Nếu đáp án của bước  $i + 1$  chỉ khác đáp án của bước  $i$  một chút, thì tính lại từ đầu là lãng phí, và toàn bộ chi phí sụp từ  $O(n^2)$  xuống  $O(n)$ . Bốn kỹ thuật trong chương — tổng tiền tố, mảng hiệu, hai con trỏ, cửa sổ trượt — là bốn cách hiện thực hoá quan sát đó cho bốn tình huống khác nhau.

## 1.1 Vì sao bắt đầu bằng mảng

Trước khi vào kỹ thuật, cần một lời biện hộ cho mảng, vì người mới thường vội bỏ qua nó để đi học cấu trúc “xịn” hơn.

Mảng có ba ưu thế mà không cấu trúc nào lấy lại được. Thứ nhất, truy cập phần tử là một phép tính địa chỉ, không có con trỏ để lần theo. Thứ hai, các phần tử nằm liên tiếp trong bộ nhớ, nên khi bạn đọc phần tử  $i$  thì phần cứng đã kéo sẵn cả một khối chứa  $i + 1, i + 2, \dots$  vào cache — duyệt tuần tự gần như miễn phí sau lần đọc đầu. Thứ ba, không có chi phí cấp phát cho từng phần tử.

Hệ quả thực hành mạnh hơn nhiều người nghĩ, và nó nổi thẳng với hình phân cấp bộ nhớ ở chương A/03: một thuật toán  $O(n \log n)$  nhảy ngẫu nhiên khắp bộ nhớ có thể thua một thuật toán  $O(n^2)$  quét tuần tự, ở những kích thước  $n$  rất thực tế. Đây là lý do các cấu trúc dữ liệu “cây” trong thực chiến thường được cài bằng mảng phẳng thay vì bằng con trỏ (B/10), và là lý do khi tối ưu hằng số thì việc đầu tiên nên thử là đổi cấu trúc có con trỏ sang mảng.

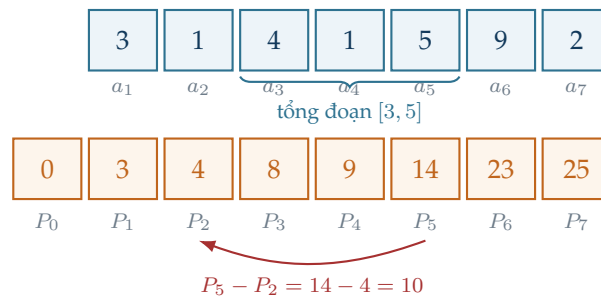
### CẠM BÃY

Cạm bẫy ngược lại cũng có thật: mảng đắt ở phép chèn và xoá giữa mảng, vì phải dịch chuyển toàn bộ phần đuôi. Nếu bài của bạn chèn và xoá liên tục ở giữa, mảng là lựa chọn sai và bạn cần cấu trúc ở chương B/09 hoặc B/10. Quy tắc: mảng thắng khi *cấu trúc dữ liệu tĩnh, truy vấn nhiều*; nó thua khi *cấu trúc thay đổi liên tục ở giữa*.

## 1.2 Tổng tiền tố: trả trước một lượt quét, sau đó mọi truy vấn miễn phí

Bài toán mở đầu: cho mảng và  $q$  truy vấn, mỗi truy vấn hỏi tổng của một đoạn  $[l, r]$ . Cách ngây thơ tốn  $O(n)$  mỗi truy vấn, tổng  $O(nq)$ . Với  $n = q = 10^5$  thì đó là  $10^{10}$  phép toán — quá xa ngân sách ở chương A/03.

Ý tưởng: tính trước mảng  $P$  với  $P_0 = 0$  và  $P_i = P_{i-1} + a_i$ . Khi đó tổng đoạn  $[l, r]$  bằng  $P_r - P_{l-1}$ . Chi phí:  $O(n)$  một lần, rồi  $O(1)$  mỗi truy vấn.



**Hình 1.1.** Tổng tiền tố. Hàng trên là mảng gốc, hàng dưới là tổng tiền tố với một ô đệm  $P_0 = 0$  — ô đệm này loại bỏ mọi trường hợp riêng khi  $l = 1$ , và bỏ nó đi là nguồn lỗi lệch một đơn vị phổ biến nhất của kỹ thuật này.

Ý tưởng mở rộng theo ba hướng và cả ba đều hay gặp. *Hai chiều*: với ma trận,  $P_{i,j}$  là tổng của hình chữ nhật từ góc trên trái tới  $(i, j)$ ; tổng của một hình chữ nhật bất kỳ tính bằng bốn giá trị theo nguyên lý bao hàm–loại trừ ( $A/05$ ). Trong thị giác máy tính, cấu trúc này có tên riêng là *ảnh tích phân* và nó là thứ khiến bộ phát hiện khuôn mặt Viola–Jones chạy được thời gian thực năm 2001. *Phép toán khác tổng*: XOR dùng được vì có phép ngược; tích thì phải cẩn thận với số 0; còn cực đại thì không dùng được vì không có phép ngược — đó chính là lúc bạn cần sparse table hoặc cây phân đoạn (**B/10**). *Tổng tiền tố theo điều kiện*: đếm số phần tử thoả tính chất nào đó trong đoạn, bằng cách xây tổng tiền tố trên mảng chỉ báo 0/1.

Một mẹo hệ quả đáng nhớ: bài “đếm số đoạn con có tổng bằng  $k$ ” giải bằng tổng tiền tố cộng bảng băm — vì tổng đoạn  $[l, r] = k$  tương đương  $P_r - P_{l-1} = k$ , tức là với mỗi  $r$  ta chỉ cần đếm xem có bao nhiêu  $P$  trước đó bằng  $P_r - k$ . Cách nghĩ này — *biến điều kiện trên đoạn thành điều kiện trên hai điểm đầu mút* — là một mẫu dùng lại rất nhiều lần.

### 1.3 Mảng hiệu: phép ngược của tổng tiền tố

Nếu tổng tiền tố giải bài “cập nhật ít, truy vấn đoạn nhiều”, thì mảng hiệu giải bài đối xứng: “cập nhật đoạn nhiều, đọc kết quả một lần ở cuối”.

Cách làm: để cộng  $v$  vào mọi phần tử trong đoạn  $[l, r]$ , ta chỉ cần  $D_l += v$  và  $D_{r+1} -= v$ . Mỗi cập nhật tốn  $O(1)$  thay vì  $O(r - l + 1)$ . Cuối cùng, lấy tổng tiền tố của  $D$  để khôi phục mảng thật.

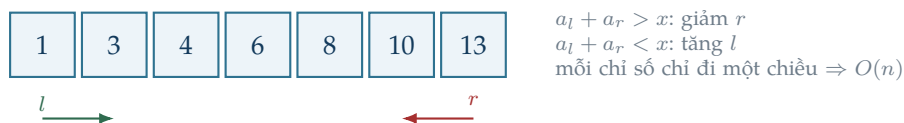
Nhận ra hai kỹ thuật này là nghịch đảo của nhau cho bạn một mẫu tư duy tổng quát: nếu một phép toán tốn kém khi làm trực tiếp, hãy tìm một biểu diễn khác của cùng dữ liệu mà trong đó phép toán ấy rẻ, rồi chuyển đổi một lần ở cuối. Cùng ý tưởng xuất hiện lại ở phép biến đổi Fourier (**D/21**), nơi phép nhân chập tốn  $O(n^2)$  trong miền thời gian trở thành phép nhân từng điểm  $O(n)$  trong miền tần số.

### 1.4 Hai con trỏ: vì sao hai vòng lặp lồng nhau lại tuyến tính

Bài toán mẫu: mảng đã sắp tăng, tìm cặp có tổng bằng  $x$ . Vét cạn xét mọi cặp,  $O(n^2)$ . Kỹ thuật hai con trỏ: đặt  $l$  ở đầu,  $r$  ở cuối. Nếu  $a_l + a_r > x$  thì giảm  $r$ ; nếu nhỏ hơn thì tăng  $l$ ; nếu bằng thì tìm thấy.

Vì sao đúng? Bất biến là mẫu chốt, và đây là một bất biến đẹp: mọi cặp đã bị loại đều chắc chắn không phải đáp án. Khi  $a_l + a_r > x$ , mọi cặp gồm  $a_r$  với một phần tử ở vị trí  $\geq l$  đều có tổng  $\geq a_l + a_r > x$ , nên loại  $r$  là hợp lệ. Vì sao tuyến tính? Vì  $l$  chỉ tăng và  $r$  chỉ giảm, mỗi chỉ số đi tối đa  $n$  bước, tổng

cộng tối đa  $2n$  bước — lập luận này được gọi là *tính chi phí bằng tổng quãng đường của con trỏ*, và nó dùng lại cho mọi biến thể.



**Hình 1.2.** Hai con trỏ trên mảng đã sắp. Điều kiện sống còn là *tính đơn điệu*: khi  $r$  giảm thì tổng chỉ có thể giảm, khi  $l$  tăng thì tổng chỉ có thể tăng. Mất tính đơn điệu — ví dụ khi mảng chưa sắp, hoặc khi có số âm trong bài cửa sổ trượt về tổng — thì kỹ thuật này cho kết quả sai chứ không phải chậm.

Biến thể quan trọng nhất là *cửa sổ trượt*: tìm đoạn liên tiếp dài nhất (hoặc ngắn nhất) thỏa một điều kiện. Khuôn chung: mở rộng  $r$  từng bước; sau mỗi lần mở rộng, nếu điều kiện bị vi phạm thì co  $l$  cho tới khi hợp lệ trở lại; cập nhật đáp án. Điều kiện để khuôn này đúng là tính đơn điệu theo nghĩa: nếu đoạn  $[l, r]$  vi phạm thì mọi đoạn chứa nó cũng vi phạm.

Đây chính là chỗ hỏng thường gặp. Bài “đoạn con dài nhất có tổng  $\leq S$ ” giải được bằng cửa sổ trượt nếu mọi số đều không âm; có số âm thì tổng không còn đơn điệu theo độ dài, và cửa sổ trượt cho đáp án sai. Bài đó phải chuyển sang tổng tiền tố cộng cấu trúc tìm kiếm. Khi gặp một bài cửa sổ trượt, câu hỏi đầu tiên luôn phải là: *tính chất này có đơn điệu không?*

#### CẦN NHỚ

Ba câu hỏi trước khi dùng hai con trỏ: (i) dữ liệu có thứ tự cần thiết chưa — nếu chưa, có sắp được không; (ii) tính chất có đơn điệu không — có lại thì luôn “dễ thỏa hơn”, mở ra thì luôn “khó thỏa hơn”; (iii) mỗi con trỏ có chỉ đi một chiều không. Thiếu bất kỳ điều nào thì kết quả sai, và cái sai đó *im lặng* — chương trình vẫn chạy, chỉ ra số khác.

### 1.5 Sắp xếp như một nước đi mở đầu

Một mẫu xuất hiện liên tục: rất nhiều bài trở nên dễ sau khi sắp xếp. Điều này đáng được nói thành nguyên tắc, vì nó là heuristic có tỉ lệ thành công cao và chi phí thấp —  $O(n \log n)$  gần như luôn nằm trong ngân sách.

Sắp xếp có tác dụng vì nó tạo ra thứ mà ba kỹ thuật trên cần: *tính đơn điệu*. Sau khi sắp, hai con trỏ dùng được, tìm kiếm nhị phân dùng được, tham lam thường lộ ra, và các phần tử giống nhau nằm cạnh nhau nên đếm bội số thành chuyện tầm thường.

Một biến thể đáng nhớ là *sắp xếp theo tiêu chí do bạn thiết kế*. Ví dụ với các đoạn thẳng, sắp theo điểm kết thúc cho bài xếp lịch tham lam (C/15); sắp theo điểm bắt đầu cho bài gộp đoạn giao nhau; sắp theo hiệu của hai đại lượng cho các bài đánh đổi. Việc chọn khóa sắp xếp thường chính là toàn bộ lời giải, và cách kiểm tra nó đúng là lập luận đổi chỗ ở chương A/04.

### 1.6 Chuỗi: mảng có tính chất riêng

Chuỗi là mảng ký tự, nên mọi kỹ thuật trên đều áp dụng được, nhưng có ba điểm riêng đáng nhớ ngay từ đây.

Thứ nhất, *so sánh hai chuỗi không phải  $O(1)$  mà là  $O(\text{độ dài})$*  — điều mà chương A/03 đã cảnh báo. Dùng chuỗi làm khóa của cấu trúc tìm kiếm thì nhân thêm một thừa số vào độ phức tạp.

Thứ hai, bảng chữ cái thường nhỏ, và điều đó mở ra kỹ thuật đếm bằng mảng cố định 26 hoặc 128 phần tử thay vì bảng băm — nhanh hơn nhiều lần. Rất nhiều bài chuỗi trong phỏng vấn (đếm anagram, cửa sổ chứa đủ ký tự) là bài cửa sổ trượt với một mảng đếm 26 phần tử.

Thứ ba, các bài so khớp mẫu có cấu trúc đặc thù và cần công cụ riêng — hàm tiền tố, hàm Z, băm chuỗi — toàn bộ là nội dung chương B/12.

## 1.7 Nhìn cửa sổ di chuyển, rồi chủ động làm nó hỏng

Với  $a = [2, 1, 3, 2]$  và  $S = 4$ , ta tìm đoạn dài nhất có tổng không vượt  $S$ . Hình 1.3 cho thấy vì sao một bước mở rộng có thể kéo theo nhiều bước co. Chỉ sau khi co xong mới cập nhật đáp án. Tổng số bước co vẫn không quá  $n$ , vì đầu trái không quay lại.

|        |   |   |   |   |          |                        |
|--------|---|---|---|---|----------|------------------------|
| [1, 1] | 2 | 1 | 3 | 2 | tổng = 2 |                        |
| [1, 2] | 2 | 1 | 3 | 2 | tổng = 3 |                        |
| [1, 3] | 2 | 1 | 3 | 2 | tổng = 6 | vi phạm: bỏ 2 rồi bỏ 1 |
| [3, 3] | 2 | 1 | 3 | 2 | tổng = 3 |                        |
| [3, 4] | 2 | 1 | 3 | 2 | tổng = 5 | vi phạm: bỏ 3          |
| [4, 4] | 2 | 1 | 3 | 2 | tổng = 2 |                        |

**Hình 1.3.** Vết chạy cửa sổ trượt với chỉ số từ 1. Ô xanh là cửa sổ hiện tại, ô trắng nằm ngoài. Hai hàng tổng 6 và 5 là trạng thái tạm thời chưa hợp lệ; đáp án tốt nhất là độ dài 2 ở đoạn  $[1, 2]$ .

Để kiểm tra giả thiết, đổi dữ liệu thành  $[5, -2]$ , vẫn lấy  $S = 4$ . Khuôn vừa dùng loại 5 ngay ở bước đầu, nên không thể tìm lại đoạn dài 2 có tổng 3. Đây là phản ví dụ đủ nhỏ để nhớ: số âm có thể cứu một cửa sổ từng vi phạm. Đừng sắp xếp để chữa bài này vì sắp xếp phá tính liên tiếp của đoạn gốc.

**Bảng 1.1.** Chọn kỹ thuật trên mảng theo thao tác cần làm. Chi phí giả sử phép toán trên một giá trị tồn hằng số.

| Kỹ thuật      | Giả thiết / cơ chế                        | Chi phí                          | Chỗ dễ sai                       |
|---------------|-------------------------------------------|----------------------------------|----------------------------------|
| Tổng tiền tố  | Mảng tĩnh, lấy hiệu hai tiền tố           | Xây $O(n)$ , hỏi $O(1)$          | Thiếu $P_0$ ; tràn tổng          |
| Mảng hiệu     | Cộng đoạn, đọc kết quả sau cùng           | Sửa $O(1)$ , khôi phục $O(n)$    | Thiếu ô $r + 1$ ; hỏi giữa chừng |
| Cửa sổ trượt  | Bỏ một đầu nút không mất nghiệm tương lai | Tổng $O(n)$ nếu hai đầu chỉ tiến | Tổng có số âm mất đơn điệu       |
| Tiền tố + băm | Đếm tổng đoạn bằng $k$ , cho phép số âm   | $O(n)$ kỳ vọng, bộ nhớ $O(n)$    | Quên tần suất tiền tố 0 bằng 1   |

Bảng 1.1 phân biệt bốn bài toán thường bị nhận nhầm thành cùng một mẫu. Khi đếm tổng bằng  $k$ , hãy tra số tiền tố bằng  $P_r - k$  trước khi thêm  $P_r$ ; đảo thứ tự sẽ đếm cả đoạn rỗng nếu  $k = 0$ .

## 1.8 Gốc rễ: từ băng từ tới ánh tích phân

Mảng là cấu trúc dữ liệu cổ nhất trong ngành và lý do rất vật lý: bộ nhớ máy tính *vốn* là một dãy ô đánh số, nên mảng không phải một phát minh mà là hình dạng tự nhiên của phần cứng. Mọi cấu trúc khác trong phần B đều được xây *trên* mảng, và cái giá của chúng là chi phí gián tiếp so với truy cập trực tiếp.

Tổng tiền tố thì cổ hơn máy tính. Ý tưởng “lập bảng tổng lũy tích để tra nhanh” đã có trong các bảng thống kê và bảng thiên văn từ trước; trong toán, đó chính là mối quan hệ giữa tổng và sai phân, tương ứng rời rạc của tích phân và đạo hàm — và mối quan hệ đó giải thích vì sao mảng hiệu là nghịch đảo của tổng tiền tố, đúng như đạo hàm là nghịch đảo của tích phân.

Điều thú vị là kỹ thuật này được phát minh lại nhiều lần ở các ngành khác nhau. Trong tính toán song song, phép *scan* (tổng tiền tố) được nhận ra là một nguyên thủy cơ bản mà rất nhiều thuật toán song song quy về được, và nó song song hoá được trong  $O(\log n)$  bước — điều không hiển nhiên chút nào vì định nghĩa của nó có vẻ hoàn toàn tuần tự. Trong thị giác máy tính, cùng cấu trúc mang tên *ảnh tích phân*: Viola và Jones dùng nó năm 2001 để tính tổng cường độ trên hình chữ nhật bất kỳ trong bốn phép truy cập, và chính điều đó biến bộ phát hiện khuôn mặt của họ từ ý tưởng thành sản phẩm chạy thời gian thực trên máy ảnh số.

Hai con trỏ thì không có “người phát minh” — nó thuộc loại kỹ thuật hiển nhiên khi nhìn đúng góc, được dùng rải rác từ lâu, và chỉ được đặt tên và hệ thống hoá khi lập trình thi đấu cần một từ vựng chung để nói về nó.

### I Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                                                         | Bài toán thúc ép lúc đó                                                                             | Điều thay đổi sau đó                                                                                                                                |
|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Thập niên 1940 — máy tính có bộ nhớ đánh địa chỉ            | Cần một cách tổ chức dữ liệu khớp với phần cứng.                                                    | Mảng: không phải phát minh mà là hình dạng tự nhiên của bộ nhớ; mọi cấu trúc khác đều xây trên nó.                                                  |
| Trước cả máy tính — bảng lũy tích trong thống kê, thiên văn | Tra nhanh tổng của một khoảng trong bảng số.                                                        | Tổng tiền tố như một dạng tích phân rời rạc; mảng hiệu là sai phân, tức phép ngược.                                                                 |
| Thập niên 1980–90 — tính toán song song                     | Nhiều thuật toán song song cần cùng một nguyên thủy tích lũy.                                       | Phép <i>scan</i> song song hoá được trong $O(\log n)$ bước dù định nghĩa trông thuần tuần tự; trở thành khối xây dựng chuẩn của GPU (D/24).         |
| 2001 — Viola & Jones                                        | Phát hiện khuôn mặt thời gian thực: cần tổng cường độ trên hàng nghìn hình chữ nhật mỗi khung hình. | Ảnh tích phân — tổng tiền tố hai chiều — cho mỗi hình chữ nhật trong bốn phép truy cập; biến thuật toán từ không khả thi thành sản phẩm thương mại. |
| Thập niên 2000 — lập trình thi đấu                          | Cần từ vựng chung để dạy các mẫu lặp lại.                                                           | “Hai con trỏ”, “cửa sổ trượt” được đặt tên và hệ thống hoá; kỹ thuật cũ nhưng cái tên mới làm chúng dạy được.                                       |

## I 1.9 Liên hệ ngang

### I Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực                 | Ý tưởng tương ứng                              | Giống ở đâu, khác ở đâu                                                                                                                                                                                                      |
|--------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Thị giác máy tính        | Ảnh tích phân, bộ lọc hộp, tính đặc trưng Haar | Cùng một cấu trúc với tổng tiền tố hai chiều. Bài học chung: khi cần rất nhiều truy vấn trên vùng chữ nhật, hãy trả trước một lượt quét. Ứng dụng gần: tính tổng cường độ trên các ô lưới khi lọc điểm đặc trưng trong SLAM. |
| Giải tích                | Tích phân và đạo hàm là hai phép ngược nhau    | Tổng tiền tố và mảng hiệu là bản rời rạc chính xác của cặp này. Nhận ra sự tương ứng giúp nhớ được cả hai và biết khi nào đổi qua lại.                                                                                       |
| Xử lý tín hiệu           | Trung bình trượt, bộ lọc chạy                  | Cùng khuôn cửa sổ trượt, chỉ khác là ở đó cửa sổ có độ dài cố định nên còn đơn giản hơn. Cả hai đều dựa vào việc cập nhật gia tăng thay vì tính lại.                                                                         |
| Mạng máy tính            | Cửa sổ trượt trong TCP                         | Cùng tên và cùng ý tưởng cơ bản: duy trì một khoảng hợp lệ, tiến mép phải khi nhận được xác nhận, kéo mép trái theo. Khác: ở đó cửa sổ điều tiết tốc độ, không phải để tìm cực trị.                                          |
| Tính toán song song, GPU | Nguyên thủy <i>scan</i> và <i>reduce</i>       | Tổng tiền tố là nguyên thủy song song cơ bản. Điểm bất ngờ đáng nhớ: một phép định nghĩa hoàn toàn tuần tự vẫn có thể song song hoá trong $O(\log n)$ bước bằng cây tổng.                                                    |
| Cơ sở dữ liệu            | Chỉ mục tổng hợp, bảng tổng lũy tích           | Cùng đánh đổi: trả trước chi phí xây dựng để truy vấn rẻ. Cùng nhược điểm: dữ liệu thay đổi thì phải xây lại — và đó chính là lý do tồn tại của cây chỉ số ở B/10.                                                           |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì ba lý do vật lý mà ký hiệu tiệm cận không nhìn thấy: truy cập là phép tính địa chỉ chứ không phải lần theo con trỏ; các phần tử nằm liên tiếp nên phần cứng nạp sẵn phần tử kế tiếp vào cache; và không có chi phí cấp phát cho từng phần tử. Với dữ liệu vừa cỡ vài triệu phần tử, ba yếu tố này cộng lại có thể tạo chênh lệch cả chục lần — đủ để lật ngược một khác biệt tiệm cận nhỏ như thừa số log.
- A2. Lập luận “tổng quãng đường của con trỏ”. Vòng lặp ngoài chỉ tăng  $l$ , vòng lặp trong chỉ giảm  $r$ , và không chỉ số nào quay lui. Mỗi chỉ số vì thế đi tối đa  $n$  bước trong suốt cả thuật toán, nên tổng số bước tối đa là  $2n$ . Cấu trúc lồng nhau của code đánh lừa: cái quyết định chi phí không phải độ sâu lồng nhau mà là tổng số lần các biến chỉ số thay đổi.
- A3. Nó hỏng khi mất tính đơn điệu. Ví dụ: tìm đoạn con dài nhất có tổng  $\leq S$  trong mảng có số âm. Với mảng  $[5, -6, 5]$  và  $S = 4$ : cửa sổ trượt sẽ co bỏ phần tử đầu ngay khi tổng vượt  $S$  và bỏ sót đoạn  $[5, -6, 5]$  có tổng 4. Nguyên nhân: khi có số âm, việc mở rộng cửa sổ có thể làm tổng giảm, nên điều kiện không còn đơn điệu theo độ dài. Bài đó phải giải bằng tổng tiền tố cộng cấu trúc tìm kiếm.
- A4. Nó cho một mẫu tổng quát: nếu phép toán A rẻ trên biểu diễn gốc còn phép toán B đắt, hãy tìm biểu diễn mà trong đó B rẻ, làm hết các thao tác B rồi chuyển đổi ngược một lần. Cụ thể: tổng tiền tố cho “truy vấn đoạn rẻ, cập nhật đắt”; mảng hiệu cho “cập nhật đoạn rẻ, đọc kết quả một lần”. Khi bài cần cả hai đều nhiều thì không biểu diễn tĩnh nào đủ, và đó chính là lúc cần cây chỉ số ở B/10.
- A5. Vì sắp xếp tạo ra tính đơn điệu, và tính đơn điệu là điều kiện đầu vào của hầu hết kỹ thuật rẻ: hai con trỏ, tìm kiếm nhị phân, tham lam, gộp các phần tử bằng nhau. Chi phí  $O(n \log n)$  gần như luôn nằm trong ngân sách, nên đây là một trong những nước đi có tỉ lệ lợi ích trên rủi ro cao nhất khi bí. Điểm cần cẩn thận: đôi khi thứ tự gốc mang thông tin (bài về đoạn liên tiếp), và lúc đó sắp xếp làm mất bài toán.

## NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn quản lý sổ thu chi của một quán ăn và mỗi ngày có người hỏi “từ ngày 5 tới ngày 20 quán thu bao nhiêu?”. Cách vụng là mỗi lần hỏi lại cộng lại từ đầu. Cách khôn là ngay từ đầu bạn ghi thêm một cột: tổng thu từ đầu tháng tới hết ngày này. Khi có người hỏi, bạn lấy con số ở ngày 20 trừ con số ở ngày 4. Đó là tổng tiền tố, và toàn bộ mẹo chỉ là *trả trước một lần để mọi câu hỏi sau chỉ tốn một phép trừ*.

Ngược lại, nếu sếp nói “cộng thêm 10 nghìn vào mọi ngày từ 5 tới 20” rất nhiều lần trong tháng, thì sửa từng ngày là phí. Bạn chỉ cần ghi “từ ngày 5 tăng thêm 10” và “từ ngày 21 giảm đi 10”, rồi cuối tháng cộng dồn một lượt. Đó là mảng hiệu.

Còn hai con trỏ thì giống hai người cùng đi trên một hàng ghế từ hai đầu, và không ai được quay lui. Vì không ai quay lui nên dù họ đi lâu tới đâu, tổng số bước cũng chỉ bằng chiều dài hàng ghế. Điều kiện để cách này đúng là hàng ghế phải được sắp có trật tự: bước sang phải thì giá trị luôn tăng, bước sang trái thì luôn giảm. Nếu trật tự đó không có — ví dụ trong dãy có số âm làm tổng lúc tăng lúc giảm — thì hai người sẽ bỏ sót chỗ cần tìm, và tệ hơn là chương trình vẫn chạy ngon lành, chỉ đưa ra kết quả sai.

**Một câu:** *Đừng tính lại từ đầu; hãy cập nhật đúng phần vừa thay đổi.*

**Chỗ không rút gọn được:** Việc kiểm tra “tính chất này có đơn điệu không” vẫn phải làm bằng lập luận cho từng bài; không có dấu hiệu bề ngoài nào chắc chắn.

## Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                     | Trả lời                                                                                                                                       |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Ý chung của cả chương               | Khi vòng ngoài tiến một bước, vòng trong đừng quay lại từ đầu — hãy cập nhật phần thay đổi. Biến $O(n^2)$ thành $O(n)$ .                      |
| ••• Tổng tiền tố                        | $P_i = P_{i-1} + a_i$ với $P_0 = 0$ ; tổng đoạn $[l, r] = P_r - P_{l-1}$ . Luôn để ô đệm $P_0$ để không có trường hợp riêng.                  |
| ••• Điều kiện dùng được tổng tiền tố    | Phép toán phải có phép ngược: tổng và XOR được, cực đại thì không — cực đại cần sparse table hoặc cây phân đoạn (B/10).                       |
| ••• Mảng hiệu                           | Cộng $v$ vào $[l, r]$ : $D_l += v$ , $D_{r+1} -= v$ ; cuối cùng lấy tổng tiền tố. Dùng khi cập nhật đoạn nhiều, đọc kết quả một lần.          |
| ••• Vì sao hai con trỏ tuyến tính       | Mỗi chỉ số chỉ đi một chiều và không quay lui, nên tổng quãng đường $\leq 2n$ . Độ sâu lồng nhau của code không quyết định chi phí.           |
| ••• Điều kiện sống còn của cửa sổ trượt | Tính đơn điệu: đoạn vi phạm thì mọi đoạn chứa nó cũng vi phạm. Mất điều kiện này thì kết quả sai một cách im lặng.                            |
| ••• Phân ví dụ kinh điển                | “Đoạn dài nhất có tổng $\leq S$ ” với mảng có số âm: $[5, -6, 5]$ , $S = 4$ — cửa sổ trượt bỏ sót đáp án.                                     |
| •• Khuôn cửa sổ trượt                   | Mở rộng $r$ một bước; trong khi vi phạm thì co $l$ ; cập nhật đáp án. Ba dòng, dùng cho hàng trăm bài.                                        |
| •• Đếm đoạn con có tổng bằng $k$        | $P_r - P_{l-1} = k$ : với mỗi $r$ , đếm số $P$ trước đó bằng $P_r - k$ bằng bảng băm. Mẫu “biến điều kiện đoạn thành điều kiện hai điểm nút”. |
| •• Tổng tiền tố hai chiều               | Tổng hình chữ nhật bằng bốn giá trị theo bao hàm–loại trừ; trong thị giác máy tính gọi là ảnh tích phân.                                      |
| •• Vì sao mảng thẳng con trỏ            | Truy cập trực tiếp, dữ liệu liên tiếp thân thiện cache, không cấp phát từng phần tử. Thua khi phải chèn/xóa giữa mảng nhiều.                  |
| •• Sắp xếp như nước mở đầu              | Nó tạo ra tính đơn điệu — điều kiện đầu vào của hai con trỏ, nhị phân và tham lam. $O(n \log n)$ hầu như luôn trong ngân sách.                |
| • Đặc thù của chuỗi                     | So sánh chuỗi tốn theo độ dài chứ không $O(1)$ ; bảng chữ cái nhỏ nên đếm bằng mảng 26 phần tử nhanh hơn bảng băm.                            |

••• phải thuộc    •• nên nắm    • biết là có

## Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

## 1. Khi nào dùng tổng tiền tố, khi nào phải dùng cây phân đoạn?

| Quyết định bằng hai câu hỏi. Dữ liệu có thay đổi giữa các truy vấn không? Nếu không, tổng tiền tố là đủ và luôn tốt hơn vì hằng số nhỏ hơn nhiều. Nếu có thay đổi, tổng tiền tố phải xây lại  $O(n)$  mỗi lần nên hỏng, và bạn cần Fenwick hoặc cây phân đoạn (B/10). Câu hỏi thứ hai: phép toán có phép ngược không? Tổng và XOR có nên tổng tiền tố dùng được; cực đại và cực tiểu thì không, phải dùng sparse table cho dữ liệu tĩnh hoặc cây phân đoạn cho dữ liệu động.

## 2. Làm sao biết chắc một bài giải được bằng cửa sổ trượt trước khi cài?

| Kiểm tính đơn điệu bằng cách hỏi hai câu cụ thể. Một: nếu đoạn  $[l, r]$  đã thỏa điều kiện, mọi đoạn con của nó có chắc chắn thỏa không? Hai: nếu đoạn  $[l, r]$  vi phạm, mọi đoạn chứa nó có chắc chắn vi phạm không? Cả hai câu đều “có” thì cửa sổ trượt dùng được. Nếu bạn không chắc, hãy thử ngay ba dữ liệu đặc biệt: có số âm, có số 0, và mọi phần tử bằng nhau — ba ca này bắt được phần lớn lỗi đơn điệu.

## 3. Hai con trỏ và tìm kiếm nhị phân đều cần dữ liệu đã sắp. Chọn cái nào?

| Chọn theo số lượng truy vấn. Nếu bạn cần trả lời một câu hỏi cho mỗi vị trí trái (kiểu “với mỗi  $l$ , tìm  $r$  lớn nhất thỏa điều kiện”) thì hai con trỏ cho  $O(n)$  tổng cộng, tốt hơn  $O(n \log n)$  của việc nhị phân từng vị trí. Nhưng hai con trỏ chỉ dùng được khi  $r$  là hàm *không giảm* theo  $l$ ; nếu không, phải nhị phân. Trong thực tế, viết nhị phân dễ hơn và ít lỗi hơn, nên khi giới hạn cho phép thì đó là lựa chọn an toàn.

## 4. Tổng tiền tố có bị tràn số không?

| Rất hay bị, và đây là lỗi im lặng kinh điển. Với  $n = 10^5$  phần tử, mỗi phần tử tới  $10^9$ , tổng có thể tới  $10^{14}$  — vượt xa số nguyên 32 bit. Quy tắc: mảng tổng tiền tố gần như luôn phải dùng kiểu 64 bit ngay cả khi mảng gốc là 32 bit. Trong bài lấy modulo, còn phải cẩn thận thêm: hiệu  $P_r - P_{l-1}$  có thể âm sau khi lấy dư, nên phải cộng thêm  $m$  rồi lấy dư lần nữa.

## 5. Kỹ thuật này còn dùng ở đâu ngoài bài tập luyện?

| Rất nhiều nơi trong hệ thống thật. Đếm số sự kiện trong một khoảng thời gian trong hệ thống giám sát chính là tổng tiền tố trên trục thời gian. Giới hạn tốc độ theo cửa sổ trượt trong máy chủ web là đúng khuôn ở mục 4. Trong thị giác máy tính, ảnh tích phân dùng để tính nhanh thống kê kê trên vùng ảnh. Trong xử lý dữ liệu cảm biến, trung bình trượt để lọc nhiễu chính là cửa sổ trượt với độ dài cố định.

## 6. Có mẹo nào tránh lỗi lệch một đơn vị với tổng tiền tố không?

| Ba quy ước cụ thể giúp gần như hết hẳn. Một: luôn để  $P_0 = 0$  và đánh số mảng từ 1, khi đó công thức  $P_r - P_{l-1}$  đúng cho mọi  $l$  kể cả  $l = 1$ . Hai: viết bất biến ra thành chú thích — “ $P_i$  = tổng của  $i$  phần tử đầu tiên” — và mọi chỉ số suy ra từ đó. Ba: kiểm tra bằng một ví dụ ba phần tử đã tính tay trước khi chạy dữ liệu lớn. Đây là ứng dụng trực tiếp của kỷ luật ở A/04.

## 7. Bài yêu cầu cả cập nhật đoạn lẫn truy vấn đoạn thì sao?

| Không cấu trúc tính nào trong chương này giải được, vì tổng tiền tố mạnh ở truy vấn còn mảng hiệu mạnh ở cập nhật, và ghép trực tiếp thì cái này phá cái kia. Đó chính xác là lý do tồn tại của cây phân đoạn có lazy propagation ở B/10: nó chấp nhận  $O(\log n)$  cho *cả hai* phía thay vì  $O(1)$  cho một phía và  $O(n)$  cho phía kia.

## Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                      | Nguồn                         | Điều cần nhận ra                                                      |
|------------------------------------------|-------------------------------|-----------------------------------------------------------------------|
| ● Tổng đoạn với truy vấn tĩnh            | CSES “Range Sum Queries I”[1] | Tổng tiền tố với ô đệm; chú ý kiểu 64 bit.                            |
| ● Cập có tổng bằng $x$ trong mảng đã sắp | CSES “Sum of Two Values”[1]   | Hai con trỏ; viết bất biến “mọi cặp đã loại đều không thể là đáp án”. |

| Bài                                                      | Nguồn                                                     | Điều cần nhận ra (tiếp)                                                                                                         |
|----------------------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| • Tổng đoạn con lớn nhất                                 | CSES “Maximum Subarray Sum”[1]                            | So sánh cách Kadane với cách tổng tiền tố cộng cực tiểu chạy — hai góc nhìn cho cùng bài.                                       |
| •• Đoạn con dài nhất không có ký tự lặp                  | LeetCode “Longest Substring Without Repeating Characters” | Cửa sổ trượt với mảng đếm; kiểm tính đơn điệu trước khi cài.                                                                    |
| •• Đếm số đoạn con có tổng bằng $x$                      | CSES “Subarray Sums II”[1]                                | Tổng tiền tố cộng bằng băm; mẫu “điều kiện đoạn thành điều kiện hai điểm nút”.                                                  |
| •• Cập nhật đoạn, truy vấn điểm                          | CSES “Range Update Queries”[1]                            | Mảng hiệu; sau đó thử lại bằng Fenwick để thấy khi nào cần cấu trúc mạnh hơn.                                                   |
| •• Tổng hình chữ nhật trong ma trận tĩnh                 | CSES “Forest Queries”[1]                                  | Tổng tiền tố hai chiều; vẽ hình bao hàm—loại trừ ra giấy trước khi viết công thức.                                              |
| •• Gộp các đoạn giao nhau                                | LeetCode “Merge Intervals”                                | Sắp theo điểm bắt đầu; đây là bài mẫu cho “chọn khoá sắp xếp chính là lời giải”.                                                |
| ••• Cửa sổ nhỏ nhất chứa đủ các ký tự cần thiết          | LeetCode “Minimum Window Substring”                       | Cửa sổ trượt hai chiều với bộ đếm; bài kinh điển để luyện việc co cửa sổ đúng lúc.                                              |
| ••• Nước mưa mắc kẹt                                     | LeetCode “Trapping Rain Water”                            | Giải cả ba cách (tiền tố cực đại, ngăn xếp <a href="#">B/07</a> , hai con trỏ) và so sánh — bài tập bước 4 tốt nhất của chương. |
| ••• Số đoạn con có đúng $k$ phần tử phân biệt            | Bài tập kinh điển                                         | Mẹo “đúng $k$ ” = “nhiều nhất $k$ ” trừ “nhiều nhất $k - 1$ ” — một mẫu biến bài không đơn điệu thành hai bài đơn điệu.         |
| ••• Đoạn con dài nhất có tổng $\leq S$ khi mảng có số âm | Bài tập kinh điển                                         | Bài mà cửa sổ trượt sai; cần tổng tiền tố cộng cấu trúc tìm kiếm — hãy tự chỉ ra chính xác chỗ tính đơn điệu gãy.               |

## Kết chương và đọc tiếp

Chương này chỉ có một ý và bốn cách hiện thực hoá nó: đừng tính lại thứ vừa tính. Tổng tiền tố trả trước cho truy vấn; mảng hiệu trả trước cho cập nhật; hai con trỏ và cửa sổ trượt cập nhật gia tăng khi biên di chuyển. Điều kiện dùng được của cả bốn đều quy về tính đơn điệu, và mất tính đơn điệu thì lỗi xảy ra một cách im lặng — nên câu hỏi kiểm tra ở khối *Cần nhớ* đáng thành phần xạ.

Chương tiếp theo mở rộng đúng ý đó cho tình huống mà thứ tự xử lý mới là cái quan trọng: khi bạn cần lấy ra “phần tử đúng lúc” — mới nhất, nhỏ nhất, hay lớn hơn kế tiếp — thay vì quét theo chỉ số. Đó là ngăn xếp, hàng đợi và heap ([B/07](#)), và bạn sẽ thấy ngăn xếp đơn điệu là họ hàng gần của hai con trỏ, với cùng lập luận “mỗi phần tử vào ra đúng một lần”.

# Nguồn

---

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [4] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [5] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [6] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [7] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [8] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [9] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [10] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [11] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] AtCoder. “AC Library: Lazy Segtree.” (), address: [https://atcoder.github.io/ac-library/master/document\\_en/lazysegtree.html](https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html) (visited on 09/07/2026).
- [13] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [14] R. E. Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets,” *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [15] M. L. Fredman and M. E. Saks, “The Cell Probe Complexity of Dynamic Data Structures,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [16] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [17] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [18] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-line String Searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

## Ngăn xếp, hàng đợi và heap (Stacks, Queues, Heaps)

**PHỤ THUỘC** B/06 cho kỹ thuật quét tuyến tính; A/03 cho phân tích khẩu hao — ngăn xếp đơn điệu là ví dụ khẩu hao đẹp nhất trong cả quyển sách.

### TÓM TẮT

Ba cấu trúc trong chương này trả lời cùng một câu hỏi với ba câu trả lời khác nhau: *trong đồng dư liệu đang chờ, phần tử nào nên xử lý tiếp?* Ngăn xếp nói “cái mới nhất”, hàng đợi nói “cái cũ nhất”, heap nói “cái nhỏ nhất”. Ba lựa chọn ấy sinh ra ba họ thuật toán rất khác nhau: đệ quy và duyệt sâu, duyệt rộng và mô phỏng theo thời gian, và các thuật toán tham lam kiểu Dijkstra hay Huffman. Chương cũng dựng hai kỹ thuật có tỉ lệ sức mạnh trên độ dài code rất cao: ngăn xếp đơn điệu và hàng đợi hai đầu đơn điệu. Nếu chỉ nhớ một điều: heap cổ tình duy trì *ít trật tự hơn* mảng đã sắp — nó chỉ biết ai nhỏ nhất — và chính sự tiết chế đó làm nó rẻ.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **ngăn xếp (stack)** — cấu trúc vào sau ra trước (LIFO): thêm và lấy đều ở một đầu, cả hai  $O(1)$ .
- **hàng đợi (queue)** — vào trước ra trước (FIFO): thêm ở một đầu, lấy ở đầu kia.
- **hàng đợi hai đầu (deque)** — thêm và lấy được ở cả hai đầu, mỗi thao tác  $O(1)$ .
- **heap nhị phân** — cây nhị phân gần đầy, mỗi nút không lớn hơn các con của nó, lưu bằng mảng: con của  $i$  là  $2i$  và  $2i + 1$ . Lấy nhỏ nhất  $O(1)$ , thêm và xóa  $O(\log n)$ .
- **hàng đợi ưu tiên (priority queue)** — giao diện trừu tượng “thêm phần tử, lấy phần tử ưu tiên nhất”; heap là cách cài phổ biến nhất nhưng không phải duy nhất.
- **ngăn xếp đơn điệu** — ngăn xếp mà các phần tử trong nó luôn giữ thứ tự tăng (hoặc giảm); dùng để tìm “phần tử lớn hơn kế tiếp” cho mọi vị trí trong  $O(n)$ .

### CÂU HỎI MỞ ĐẦU

- Q1. Ngăn xếp đơn điệu có vòng lặp lồng nhau và trông như  $O(n^2)$ . Vì sao nó thật sự là  $O(n)$ ?
- Q2. Vì sao heap chỉ cho biết phần tử nhỏ nhất mà không cho biết phần tử thứ hai một cách trực tiếp — và vì sao đó lại là *ưu điểm* chứ không phải khuyết điểm?
- Q3. Xây heap từ  $n$  phần tử tốn  $O(n)$  chứ không phải  $O(n \log n)$ , dù mỗi lần chèn tốn  $O(\log n)$ . Chỗ nào trong lập luận “ $n$  lần chèn” bị sai?
- Q4. Duyệt sâu dùng ngăn xếp, duyệt rộng dùng hàng đợi — chỉ khác một dòng code. Vì sao hai thuật toán lại có tính chất khác nhau đến vậy?
- Q5. Khi nào một mảng đã sắp xếp tốt hơn heap, và ngược lại?

Chương trước quét dữ liệu theo thứ tự chỉ số. Nhưng rất nhiều bài toán không cho phép làm vậy: bạn xử lý một phần tử, việc đó sinh ra công việc mới, và công việc mới phải xếp vào một chỗ nào đó để xử lý sau. Câu hỏi trung tâm trở thành: *trong đồng công việc đang chờ, cái nào làm tiếp?*

Ba cấu trúc trong chương này là ba câu trả lời, và điều đáng chú ý là chúng chỉ khác nhau ở đúng chỗ đó — cùng một khung “lấy ra, xử lý, đẩy vào” nhưng đổi quy tắc lấy ra thì được ba thuật toán khác hẳn nhau về hành vi. Duyệt sâu, duyệt rộng và Dijkstra là cùng một chương trình với ba cấu trúc chờ khác nhau, và nhận ra điều đó làm ba thuật toán ấy sụp thành một.

## 2.1 Ngăn xếp: cái mới nhất trước

Ngăn xếp là cấu trúc gắn liền với *sự lồng nhau*. Mỗi khi bài toán có cấu trúc “mở rồi phải đóng theo đúng thứ tự ngược lại” thì ngăn xếp là công cụ tự nhiên: dấu ngoặc trong biểu thức, thẻ trong tài liệu đánh dấu, lời gọi hàm trong chương trình, thao tác hoàn tác trong trình soạn thảo.

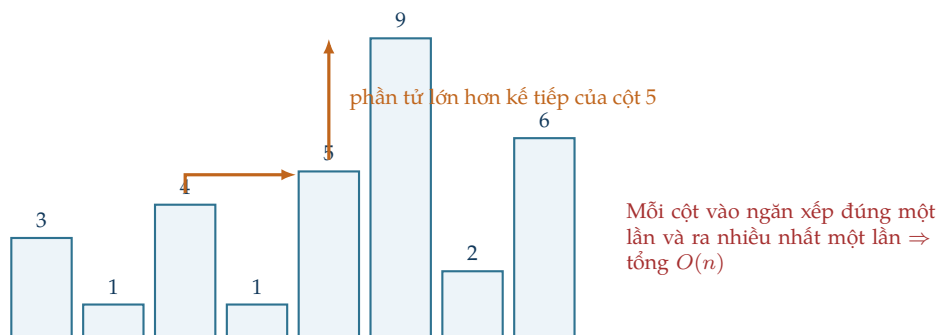
Điều đáng để ý về mặt khái niệm: đệ quy và ngăn xếp là cùng một thứ. Máy tính thực hiện đệ quy bằng cách đẩy trạng thái lời gọi vào ngăn xếp phân cứng. Vì vậy mọi thuật toán đệ quy đều viết lại được thành vòng lặp với ngăn xếp tường minh — một kỹ thuật đáng biết vì đôi khi nó là cách duy nhất tránh tràn ngăn xếp khi độ sâu lên tới hàng trăm nghìn (E/25).

### Ngăn xếp đơn điệu: kỹ thuật đáng giá nhất của chương

Bài toán: với mỗi vị trí  $i$  trong mảng, tìm vị trí đầu tiên bên phải có giá trị lớn hơn  $a_i$ . Vết cặn  $O(n^2)$ . Ngăn xếp đơn điệu cho  $O(n)$ .

Ý tưởng: quét từ trái sang phải, duy trì một ngăn xếp chứa các vị trí mà *chưa tìm được phần tử lớn hơn*. Khi tới vị trí  $i$ , lấy ra khỏi ngăn xếp mọi vị trí có giá trị nhỏ hơn  $a_i$  — với mỗi vị trí lấy ra, đáp án của nó chính là  $i$ . Rồi đẩy  $i$  vào.

Bất biến: *ngăn xếp luôn chứa các giá trị giảm dần từ đáy lên đỉnh, và đó đúng là những vị trí còn đang chờ đáp án*. Vì sao  $O(n)$ ? Mỗi vị trí được đẩy vào đúng một lần và lấy ra nhiều nhất một lần, nên tổng số thao tác không quá  $2n$  — lập luận khấu hao y hệt lập luận “tổng quãng đường con trỏ” ở chương B/06. Vòng lặp trong có thể chạy nhiều bước ở một số lần lặp, nhưng khi đó nó vừa xoá đi nhiều phần tử mà ai đó đã trả tiền lúc đẩy vào.



**Hình 2.1.** Ngăn xếp đơn điệu tìm “phần tử lớn hơn kế tiếp”. Các cột đang nằm trong ngăn xếp luôn giảm dần từ trái sang phải; khi gặp một cột cao hơn, tất cả các cột thấp hơn nó bị lấy ra và nhận cột này làm đáp án. Cùng cấu trúc giải bài “hình chữ nhật lớn nhất trong biểu đồ cột” — ở đó mỗi cột bị lấy ra chính là lúc ta biết được cả biên trái lẫn biên phải của hình chữ nhật cao bằng nó.

Họ bài giải bằng ngăn xếp đơn điệu rộng hơn tương ứng: phần tử lớn hơn/nhỏ hơn kế tiếp hai phía, hình chữ nhật lớn nhất trong biểu đồ cột, nước mưa mắc kẹt, số đoạn con mà một phần tử là cực đại, và cả việc dựng cây Cartesian. Dấu hiệu nhận biết trên đề: *với mỗi phần tử, cần biết phần tử gần nhất về một phía thoả một quan hệ so sánh*.

## 2.2 Hàng đợi và hàng đợi hai đầu

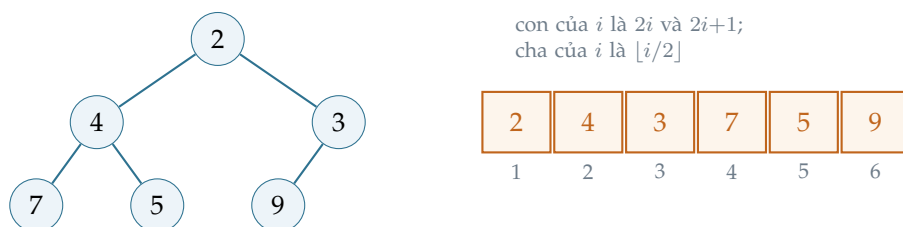
Hàng đợi gắn với việc xử lý theo *lớp*: duyệt rộng trên đồ thị xử lý hết các đỉnh cách 1 bước rồi mới tới các đỉnh cách 2 bước, và chính tính chất đó làm duyệt rộng cho đường đi ngắn nhất trên đồ thị không trọng số (C/17). Trong hệ thống thật, hàng đợi là cấu trúc của mọi thứ có “xếp hàng chờ”: hàng đợi tin nhắn, bộ đệm, lịch tiến trình.

Hàng đợi hai đầu cho một kỹ thuật quan trọng: *cực đại trên cửa sổ trượt* trong  $O(n)$ . Duy trì một deque chứa các chỉ số có giá trị giảm dần; khi cửa sổ tiến, bỏ ở đầu những chỉ số đã rơi ra ngoài cửa sổ, và bỏ ở cuối những chỉ số có giá trị nhỏ hơn phần tử mới — vì chúng không bao giờ còn cơ hội làm cực đại nữa. Đây là kết hợp của cửa sổ trượt (B/06) với ý tưởng đơn điệu của mục trước, và nó cũng là bước tăng tốc chuẩn cho một họ bài quy hoạch động (C/16).

## 2.3 Heap: chỉ biết ai nhỏ nhất, và chỉ cần thế

Heap nhị phân là một cây nhị phân gần đầy lưu trong mảng, với tính chất: mỗi nút không lớn hơn các con của nó. Chú ý điều *không* được đảm bảo: hai anh em không có quan hệ gì với nhau, và mảng biểu diễn heap hoàn toàn không sắp xếp. Heap chỉ biết đúng một điều — gốc là nhỏ nhất.

Sự tiết chế đó chính là nguồn sức mạnh. Duy trì trật tự toàn phần (mảng đã sắp) tốn  $O(n)$  cho mỗi lần chèn; duy trì trật tự bộ phận của heap chỉ tốn  $O(\log n)$ . Nếu bài toán chỉ cần biết ai nhỏ nhất — và rất nhiều bài chỉ cần thế — thì trả tiền cho trật tự toàn phần là lãng phí.



**Hình 2.2.** Heap nhị phân và biểu diễn mảng của nó. Không có con trỏ nào — quan hệ cha con là phép tính chỉ số, nên heap vừa tiết kiệm bộ nhớ vừa thân thiện với cache. Chú ý mảng *không* sắp xếp: 4 đứng trước 3. Heap chỉ hứa mỗi nút nhỏ hơn các con của nó, không hứa gì về quan hệ giữa các nhánh.

Ba thao tác và chi phí: lấy nhỏ nhất  $O(1)$ ; thêm phần tử  $O(\log n)$  bằng cách đặt vào cuối rồi đẩy lên; xoá nhỏ nhất  $O(\log n)$  bằng cách đưa phần tử cuối lên gốc rồi đẩy xuống. Còn một thao tác đáng nhớ vì nó phản trực giác: *xây heap từ  $n$  phần tử có sẵn tốn  $O(n)$ , không phải  $O(n \log n)$* . Lý do: nếu đi từ dưới lên và đẩy xuống, thì các nút ở tầng đáy — chiếm một nửa số nút — gần như không phải làm gì, các nút ở tầng trên thì ít. Tổng chi phí là  $\sum_h (n/2^h) \cdot h$ , một chuỗi hội tụ về  $O(n)$ . Đây là ứng dụng trực tiếp của “cấp số nhân giảm bị chặn bởi hằng số” ở chương A/05.

### Ba mẫu dùng heap đáng thuộc

*Chọn  $k$  phần tử lớn nhất trong  $n$  phần tử.* Giữ một heap nhỏ nhất kích thước  $k$ ; mỗi phần tử mới, nếu lớn hơn gốc thì thay gốc. Chi phí  $O(n \log k)$  và bộ nhớ  $O(k)$  — tốt hơn hẳn việc sắp toàn bộ khi  $k \ll n$ , và quan trọng hơn, nó chạy được cả khi dữ liệu là luồng không lưu hết được (D/24).

*Trộn  $k$  danh sách đã sắp.* Heap chứa phần tử đầu của mỗi danh sách; mỗi lần lấy ra cái nhỏ nhất rồi nạp phần tử kế tiếp của danh sách đó. Chi phí  $O(N \log k)$ . Đây là thuật toán nền của việc sắp xếp ngoài (dữ liệu lớn hơn bộ nhớ) và của phép trộn trong nhiều hệ cơ sở dữ liệu.

*Trung vị của luồng dữ liệu.* Hai heap: một heap lớn nhất giữ nửa nhỏ, một heap nhỏ nhất giữ nửa lớn, giữ cho kích thước chênh nhau tối đa 1. Trung vị luôn ở đỉnh của một trong hai. Mẫu này — tách dữ liệu thành hai nửa, mỗi nửa một heap quay ngược nhau — còn dùng cho nhiều bài “duy trì phân vị”.

| Cấu trúc            | Lấy nhỏ nhất | Chèn        | Tìm phần tử thứ $k$ | Nên chọn khi                            |
|---------------------|--------------|-------------|---------------------|-----------------------------------------|
| Mảng chưa sắp       | $O(n)$       | $O(1)$      | $O(n)$              | Dữ liệu tĩnh, ít truy vấn               |
| Mảng đã sắp         | $O(1)$       | $O(n)$      | $O(1)$              | Xây một lần, truy vấn nhiều, không chèn |
| Heap nhị phân       | $O(1)$       | $O(\log n)$ | <i>không hỗ trợ</i> | Chỉ cần cực trị, có chèn liên tục       |
| Cây cân bằng (B/09) | $O(\log n)$  | $O(\log n)$ | $O(\log n)$         | Cần cả thứ tự lẫn cực trị               |

Bảng trên trả lời câu hỏi mở đầu thứ năm: heap thắng khi có chèn liên tục và chỉ cần cực trị; mảng đã sắp thắng khi dữ liệu tĩnh và cần truy vấn theo hạng; cây cân bằng đắt hơn cả hai nhưng làm được cả hai việc.

### CẠM BẤY

Heap nhị phân chuẩn *không* hỗ trợ hai thao tác mà người ta hay tưởng là có: xoá một phần tử bất kỳ, và giảm khoá của một phần tử. Muốn có chúng cần lưu thêm chỉ số vị trí của từng phần tử, tức là một heap có đánh dấu. Trong lập trình thi đấu, mẹo thông dụng hơn là *xoá lười*: cứ đẩy bản ghi mới vào và khi lấy ra thì bỏ qua những bản ghi đã lỗi thời. Đây chính là lý do bản Dijkstra viết bằng hàng đợi ưu tiên thường có dòng kiểm tra “nếu khoảng cách lấy ra lớn hơn khoảng cách đã biết thì bỏ qua” (C/17).

## 2.4 Cùng một khung, ba cấu trúc, ba thuật toán

Đây là chỗ chương này trả công nhiều nhất về mặt tư duy. Xét khung chương trình sau: lấy một phần tử từ kho chờ, xử lý nó, đẩy các phần tử liên quan vào kho chờ, lặp lại. Đổi kho chờ, bạn được ba thuật toán khác nhau:

- **Ngăn xếp** → duyệt sâu. Đi tới cùng một nhánh rồi mới quay lại. Dùng để tìm thành phần liên thông, phát hiện chu trình, sắp thứ tự tô-pô.
- **Hàng đợi** → duyệt rộng. Xử lý theo lớp khoảng cách. Cho đường đi ngắn nhất trên đồ thị không trọng số.
- **Hàng đợi ưu tiên** → Dijkstra và A\*. Xử lý theo thứ tự chi phí tăng dần. Cho đường đi ngắn nhất với trọng số không âm.

Nhận ra ba thuật toán này là một giúp bạn nhớ ít hơn và suy được nhiều hơn. Ví dụ, câu hỏi “vì sao Dijkstra cần trọng số không âm” trở thành hiển nhiên khi nhìn theo khung này: thuật toán chốt một đỉnh ngay khi lấy nó ra khỏi hàng đợi ưu tiên, tức là nó tin rằng không có đường nào rẻ hơn sẽ tới sau — và niềm tin ấy chỉ đúng nếu mọi cạnh đều làm chi phí tăng lên. Chương C/17 khai thác đầy đủ cách nhìn này.

## 2.5 Giảm khoá và bài học Fibonacci heap: khi cận và máy thật không cùng kết luận

Khối *Cạm bẫy* ở trên nói heap nhị phân không có thao tác giảm khoá. Câu hỏi tự nhiên là: có cấu trúc nào có không, và nó đáng giá bao nhiêu? Câu trả lời dẫn tới một trong những câu chuyện đáng học nhất của ngành cấu trúc dữ liệu.

Bắt đầu từ chỗ thao tác này thực sự tốn tiền. Dijkstra (C/17) mỗi lần thư giãn một cạnh là một lần muốn hạ nhãn của một đỉnh. Với heap nhị phân, cách làm phổ biến là đẩy thêm một bản ghi

mới rồi bỏ qua bản ghi lỗi thời khi lấy ra — nghĩa là hàng đợi chứa tới  $m$  phần tử và tổng chi phí là  $O(m \log n)$ . Fredman và Tarjan[2] đặt câu hỏi: nếu giảm khoá rẽ đi thì Dijkstra rẽ đi bao nhiêu? Với  $O(1)$  cho giảm khoá và  $O(\log n)$  cho lấy nhỏ nhất, tổng thành  $O(m + n \log n)$  — tốt hơn hẳn khi đồ thị dày, vì lúc đó  $m$  lớn hơn  $n \log n$  nhiều lần.

Fibonacci heap đạt hai cận đó theo nghĩa *khấu hao*, và cơ chế của nó là một minh hoạ đẹp cho phương pháp thể năng ở chương A/03: hoãn mọi việc dọn dẹp tới lúc bắt buộc. Chèn chỉ là thêm một cây vào danh sách gốc, không so sánh gì cả. Giảm khoá chỉ là *cắt* nút đó ra khỏi cha và biến nó thành một gốc mới — cũng không so sánh gì. Toàn bộ việc gộp các cây cùng bậc bị dồn sang lần lấy nhỏ nhất kế tiếp, và để cây không bị cắt tới mức xơ xác, mỗi nút bị mất con lần thứ hai sẽ bị cắt lên theo dây chuyền. Hàm thể năng đếm số cây gốc cùng số nút đã bị đánh dấu, và chính nó trả tiền cho những lần dọn dẹp đất đỏ.

Đến đây lý thuyết rất đẹp. Nhưng câu hỏi thực dụng — *có nên dùng không* — lại được trả lời bằng máy đo chứ không bằng chứng minh, và câu trả lời không đơn giản như hai phe vẫn nói. Larkin, Sen và Tarjan[3] cài mười biến thể heap theo cùng một chuẩn “không tối ưu hoá lắt léo” rồi đo trên ba loại tải: dãy thao tác nhân tạo, Dijkstra trên bản đồ đường thật, và một bộ mô phỏng sự kiện rời rạc. Bốn kết luận của họ đáng nhớ hơn bất kỳ bảng độ phức tạp nào.

Thứ nhất, thời gian chạy tương quan mạnh với *tỉ lệ trượt cache*, đặc biệt là cache L1 — nghĩa là quyết định thiết kế ở tầng cao nhất, “mảng hay con trỏ”, chi phối kết quả nhiều hơn cận khấu hao. Thứ hai, pairing heap thắng Fibonacci heap trên mọi tải mà họ thử: cùng ý tưởng hoãn dọn dẹp nhưng ít siêu dữ liệu hơn nên ít trượt cache hơn. Thứ ba, cả hai điều mà “ai cũng biết” đều sai ở một số tải: heap  $d$  phân lưu trong mảng *không* phải luôn tốt nhất, và Fibonacci heap *không* phải luôn chậm thảm hại — có tải nó vượt heap mảng. Thứ tư, và mỉa mai nhất, những biến thể được đề xuất về sau nhằm *đơn giản hoá* Fibonacci heap thường chạy chậm hơn chính bản gốc.

Bài học rút ra không phải “đừng tin lý thuyết” mà là một câu chính xác hơn: cận khấu hao đếm *số thao tác trên cấu trúc*, còn máy tính tiền theo *cách bố trí bộ nhớ*. Hai thước đo ấy chỉ trùng nhau khi mỗi thao tác có giá như nhau, mà điều đó không đúng từ khi có phân cấp cache (A/03). Vì vậy quy tắc thực hành gọn lại thành ba dòng: trong thi đấu, dùng heap nhị phân của thư viện kèm mẹo xoá lười; trong hệ thống thật mà hàng đợi ưu tiên nằm trên đường nóng, hãy đo với đúng tải của mình (E/26); và chỉ nghĩ tới Fibonacci heap khi đồ thị dày tới mức số lần giảm khoá lớn hơn hẳn số lần lấy nhỏ nhất — một điều kiện hiếm gặp hơn nhiều so với ấn tượng mà giáo trình để lại.

## 2.6 Gốc rễ: từ máy tính không có đệ quy tới lịch trình hệ điều hành

Ngăn xếp có một lịch sử gắn liền với việc làm cho đệ quy trở nên khả thi. Những máy tính đầu tiên không có cơ chế gọi hàm lồng nhau: địa chỉ trở về được ghi đè vào một ô cố định, nên hàm không thể gọi lại chính nó. Cuối thập niên 1950, khi ngành trình biên dịch cần đánh giá biểu thức toán học có dấu ngoặc lồng nhau, hai nhà nghiên cứu người Đức là Friedrich Bauer và Klaus Samelson đề xuất dùng một cấu trúc vào sau ra trước để lưu các phép toán đang chờ — và cùng thời gian đó, việc thiết kế ngôn ngữ Algol 60 với đệ quy đã biến ngăn xếp thành một phần của kiến trúc máy. Ngày nay “ngăn xếp lời gọi” là khái niệm mà mọi lập trình viên gặp mỗi lần đọc thông báo lỗi, nhưng nó là một phát minh có ngày tháng, không phải điều hiển nhiên.

Heap ra đời năm 1964 từ một nhu cầu rất cụ thể: J. W. J. Williams muốn một thuật toán sắp xếp tại chỗ với đảm bảo  $O(n \log n)$  ở trường hợp xấu nhất — thứ mà quicksort không có — và ông thiết kế cấu trúc heap để phục vụ heapsort. Ngay sau đó, Robert Floyd đưa ra cách xây heap từ dưới lên trong  $O(n)$ . Điều thú vị là cấu trúc sinh ra để phục vụ sắp xếp lại trở nên nổi tiếng hơn ở vai trò khác: hàng đợi ưu tiên cho mô phỏng sự kiện rời rạc, cho lịch trình hệ điều hành, và cho các thuật toán tham lam trên đồ thị.

Câu chuyện đó lặp lại một lần nữa hai mươi năm sau. Fredman và Tarjan thiết kế Fibonacci heap [2] với mục tiêu rất hẹp: giảm độ phức tạp của Dijkstra từ  $O(m \log n)$  xuống  $O(m + n \log n)$ , bằng cách làm cho thao tác giảm khoá tốn  $O(1)$  khấu hao. Họ đạt được mục tiêu lý thuyết, nhưng hằng số lớn tới mức trong thực tế heap nhị phân thường vẫn nhanh hơn — một ví dụ kinh điển cho bài học “tiệm cận không phải tất cả” ở chương A/03, và cũng là ví dụ cho thấy một kết quả lý thuyết vẫn có giá trị dù ít khi được cài đặt: nó xác lập đâu là giới hạn.

**I Gốc rễ: ý tưởng này ra đời từ bài toán nào**

| Mốc                                                        | Bài toán thúc ép lúc đó                                                        | Điều thay đổi sau đó                                                                                                   |
|------------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Cuối thập niên 1950 — Bauer, Samelson và Algol 60          | Đánh giá biểu thức có ngoặc lồng nhau; và làm cho hàm gọi được chính nó.       | Ngăn xếp trở thành cấu trúc nền của trình biên dịch và của kiến trúc máy; “ngăn xếp lời gọi” ra đời.                   |
| 1964 — Williams                                            | Cần thuật toán sắp xếp tại chỗ có đảm bảo $O(n \log n)$ ở trường hợp xấu nhất. | Heap nhị phân và heapsort; cấu trúc sau đó nổi tiếng hơn ở vai trò hàng đợi ưu tiên.                                   |
| 1964 — Floyd                                               | Xây heap bằng $n$ lần chèn tốn $O(n \log n)$ , có cách rẻ hơn không?           | Xây từ dưới lên trong $O(n)$ ; ví dụ mẫu mực cho việc tổng của cấp số nhân giảm là hằng số.                            |
| Thập niên 1960–70 — mô phỏng sự kiện rời rạc, hệ điều hành | Cần lấy sự kiện sắp xảy ra sớm nhất trong hàng triệu sự kiện đang chờ.         | Hàng đợi ưu tiên trở thành cấu trúc lõi của mô phỏng và của bộ lập lịch.                                               |
| 1984–87 — Fredman & Tarjan                                 | Dijkstra bị chặn bởi chi phí giảm khoá.                                        | Fibonacci heap: giảm khoá $O(1)$ khấu hao, cho $O(m + n \log n)$ ; nhưng hằng số lớn nên hiếm khi thắng trong thực tế. |

## I 2.7 Liên hệ ngang

**I Liên hệ ngang: ý này còn sống ở đâu nữa**

| Lĩnh vực                        | Ý tưởng tương ứng                                        | Giống ở đâu, khác ở đâu                                                                                                                                                                            |
|---------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kiến trúc máy tính              | Ngăn xếp lời gọi, khung ngăn xếp, tràn ngăn xếp          | Không phải chỉ tương tự — đệ quy <i>được cài đặt</i> bằng đúng cấu trúc này. Hệ quả thực dụng: đệ quy sâu quá thì sập, và cách chữa là viết lại bằng ngăn xếp tường minh (E/25).                   |
| Trình biên dịch                 | Phân tích biểu thức, khớp ngoặc, ký pháp Ba Lan ngược    | Ứng dụng lịch sử đầu tiên của ngăn xếp. Mọi bài “kiểm tra dãy ngoặc hợp lệ” trong phòng vẫn là phiên bản thu nhỏ của bộ phân tích cú pháp thật.                                                    |
| Hệ điều hành                    | Lịch trình tiến trình theo độ ưu tiên, hàng đợi sẵn sàng | Cùng cấu trúc hàng đợi ưu tiên. Khác ở ràng buộc: bộ lập lịch phải tránh bỏ sót tiến trình, nên độ ưu tiên thay đổi theo thời gian chờ — một yêu cầu không có trong bài toán thuật toán thuần túy. |
| Mô phỏng sự kiện rời rạc        | Lịch sự kiện: luôn lấy sự kiện có thời điểm sớm nhất     | Đây là ứng dụng “đúng nghĩa đen” của hàng đợi ưu tiên, và là lý do cấu trúc này quan trọng ngoài phạm vi thuật toán đồ thị.                                                                        |
| Lập kế hoạch đường đi cho robot | $A^*$ với biên là hàng đợi ưu tiên                       | Cùng khung “lấy ra, mở rộng, đẩy vào” ở mục 4, chỉ đổi khoá ưu tiên thành $g + h$ . Bài học chuyển thẳng: chọn cấu trúc kho chờ chính là chọn thuật toán.                                          |
| Xử lý dòng dữ liệu              | Giữ $k$ phần tử lớn nhất bằng heap kích thước $k$        | Cùng mẫu ở mục 3.1. Ưu điểm quyết định trong bối cảnh luồng: bộ nhớ $O(k)$ không phụ thuộc độ dài luồng (D/24).                                                                                    |

| Lĩnh vực    | Ý tưởng tương ứng                                  | Giống ở đâu, khác ở đâu (tiếp)                                                                                                      |
|-------------|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Nén dữ liệu | Thuật toán Huffman: luôn gộp hai tần suất nhỏ nhất | Một thuật toán tham lam mà hàng đợi ưu tiên là công cụ thực thi trực tiếp. Xem C/15 cho phần chứng minh vì sao tham lam ở đây đúng. |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì mỗi phần tử được đẩy vào ngăn xếp đúng một lần và bị lấy ra nhiều nhất một lần trong suốt cả thuật toán, nên tổng số thao tác không quá  $2n$  bất kể vòng lặp trong chạy dài tới đâu ở một số lần lặp cụ thể. Đây là lập luận khấu hao kiểu tổng gộp ( $A/03$ ): một lần lấy ra nhiều phần tử chỉ xảy ra được sau khi đã có nhiều lần đẩy vào tương ứng. Cùng dạng lập luận với “tổng quãng đường con trỏ” ở B/06.
- A2. Vì duy trì ít trật tự hơn thì rẻ hơn. Sắp xếp toàn phần đòi  $O(n \log n)$  và mỗi lần chèn vào mảng đã sắp tồn  $O(n)$ ; heap chỉ duy trì quan hệ cha-con nên chèn chỉ  $O(\log n)$ . Nếu bài toán chỉ hỏi “ai nhỏ nhất” — và rất nhiều thuật toán tham lam chỉ hỏi đúng thế — thì trả tiền cho trật tự toàn phần là lãng phí. Đây là ví dụ tiêu biểu của nguyên tắc chung ở phần B: chọn cấu trúc duy trì *đúng* lượng trật tự mà bài cần, không hơn.
- A3. Sai ở chỗ giả định mọi phần tử đều phải đi hết  $\log n$  tầng. Khi xây từ dưới lên, một nửa số nút nằm ở tầng đáy và không phải đẩy xuống bước nào; một phần tư ở tầng kế và đi tối đa một bước; cứ thế. Tổng chi phí là  $\sum_h (n/2^h) \cdot h$ , một chuỗi hội tụ, cho  $O(n)$ . Còn cách chèn lần lượt thì ngược lại: phần tử vào sau đi từ đáy lên nên đúng là tốn  $O(\log n)$  mỗi cái.
- A4. Vì thứ tự lấy ra quyết định thứ tự khám phá. Ngăn xếp lấy cái mới nhất nên thuật toán lao sâu theo một nhánh trước khi quay lại — hợp cho việc phát hiện chu trình và tính thứ tự tô-pô, vì thời điểm vào/ra của mỗi đỉnh mang thông tin cấu trúc. Hàng đợi lấy cái cũ nhất nên khám phá theo lớp khoảng cách tăng dần — đó chính xác là lý do duyệt rộng cho đường đi ngắn nhất trên đồ thị không trọng số. Cùng chương trình, đổi một dòng, đổi hẳn tính chất được đảm bảo.
- A5. Mảng đã sắp thẳng khi dữ liệu tĩnh: nó cho cả cực trị lẫn truy vấn phần tử thứ  $k$  trong  $O(1)$ , còn heap không trả lời được câu thứ hai. Heap thẳng khi có chèn xen kẽ liên tục, vì chèn vào mảng đã sắp tồn  $O(n)$  còn heap chỉ  $O(\log n)$ ; và heap còn thẳng khi bạn chỉ cần  $k$  phần tử đầu của một luồng rất dài, vì bộ nhớ chỉ  $O(k)$ . Cần cả hai khả năng thì phải trả giá cao hơn bằng cây cân bằng ở B/09.

### NÓI LẠI THẬT ĐƠN GIẢN

Ba cấu trúc này chỉ khác nhau ở một câu hỏi: trong đồng việc đang chờ, làm việc nào trước? Ngăn xếp giống chồng giấy tờ trên bàn: bạn luôn xử lý tờ trên cùng, tức là tờ mới đặt xuống gần nhất. Kiểu này hợp khi công việc lồng nhau — đang làm việc A thì phát sinh việc B phải xong trước, xong B mới quay lại A. Đó cũng đúng là cách máy tính chạy hàm gọi hàm.

Hàng đợi giống hàng người chờ mua vé: ai tới trước phục vụ trước. Kiểu này hợp khi bạn muốn lan toả đều ra xung quanh, ví dụ “tìm những chỗ cách đây đúng ba bước” — vì mọi chỗ cách hai bước sẽ được xử lý xong trước khi chạm tới chỗ cách ba bước.

Heap giống phòng cấp cứu: không xét ai tới trước, mà xét ai nặng nhất. Điều đáng nói là phòng cấp cứu không cần xếp hạng toàn bộ bệnh nhân từ nặng tới nhẹ; nó chỉ cần biết *ai nặng nhất ngay lúc này*. Chính vì đòi hỏi ít hơn nên nó làm nhanh hơn — và đó là bài học chung: đừng sắp xếp mọi thứ khi bạn chỉ cần biết cái đứng đầu.

Còn ngăn xếp đơn điệu thì giống việc bạn xếp các cột chờ “ai sẽ cao hơn tôi”. Mỗi cột mới cao lên sẽ giải quyết xong một loạt cột thấp đang chờ, và những cột đó rời hàng vĩnh viễn. Vì mỗi cột vào hàng một lần và rời hàng một lần, tổng công việc chỉ tỉ lệ với số cột, dù có lúc một cột mới giải quyết hết cả chục cột cũ.

**Một câu:** Chọn cấu trúc kho chờ chính là chọn thuật toán: mới nhất, cũ nhất, hay nhỏ nhất.

**Chỗ không rút gọn được:** Việc heap không hỗ trợ “xoá phần tử bất kỳ” là hạn chế thật, không phải chi tiết cài đặt; muốn có nó thì phải trả bằng cấu trúc phức tạp hơn hoặc bằng mẹo xoá lười.

| Thuật ngữ / câu hỏi                      | Trả lời                                                                                                                                                                                            |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● Ba cấu trúc, ba quy tắc lấy ra       | Ngăn xếp: mới nhất (LIFO). Hàng đợi: cũ nhất (FIFO). Heap: nhỏ nhất. Đối kho chờ trong cùng một khung thì được duyệt sâu, duyệt rộng, Dijkstra.                                                    |
| ●●● Ngăn xếp đơn điệu — ý tưởng          | Giữ các vị trí chưa tìm được đáp án theo thứ tự đơn điệu; phần tử mới giải quyết và loại bỏ mọi vị trí nhỏ hơn nó.                                                                                 |
| ●●● Ngăn xếp đơn điệu — vì sao $O(n)$    | Mỗi phần tử vào một lần, ra tối đa một lần; tổng $\leq 2n$ thao tác. Khẩu hao kiểu tổng gộp.                                                                                                       |
| ●●● Dấu hiệu dùng ngăn xếp đơn điệu      | “Với mỗi phần tử, tìm phần tử gần nhất về một phía thoả quan hệ so sánh” — gồm cả biểu đồ cột, nước mưa, số đoạn mà phần tử là cực đại.                                                            |
| ●●● Tính chất heap                       | Mỗi nút $\leq$ các con; lưu bằng mảng, con của $i$ là $2i, 2i+1$ . Không đảm bảo gì về quan hệ giữa hai nhánh; mảng không sắp xếp.                                                                 |
| ●●● Chi phí heap                         | Lấy nhỏ nhất $O(1)$ ; chèn và xoá gốc $O(\log n)$ ; xây từ $n$ phần tử có sẵn $O(n)$ .                                                                                                             |
| ●●● Vì sao xây heap là $O(n)$            | Xây từ dưới lên: nửa số nút ở đáy không phải làm gì; tổng $\sum_h (n/2^h)h$ hội tụ. Cấp số nhân giảm bị chặn bởi hằng số ( $A/05$ ).                                                               |
| ●● Cực đại trên cửa sổ trượt             | Deque chứa chỉ số có giá trị giảm dần; bỏ đầu khi rơi khỏi cửa sổ, bỏ cuối khi nhỏ hơn phần tử mới. $O(n)$ tổng cộng.                                                                              |
| ●● Ba mẫu dùng heap                      | $k$ lớn nhất bằng heap kích thước $k$ ( $O(n \log k)$ ), bộ nhớ $O(k)$ ; trộn $k$ danh sách đã sắp; trung vị luồng bằng hai heap ngược nhau.                                                       |
| ●● Heap không hỗ trợ gì                  | Xoá phần tử bất kỳ và giảm khoá — cần heap có đánh dấu vị trí, hoặc dùng mẹo xoá lười (kiểm tra bản ghi lỗi thời khi lấy ra).                                                                      |
| ●● Vì sao Dijkstra cần trọng số không âm | Vì nó chốt đỉnh ngay khi lấy ra khỏi hàng đợi ưu tiên; niềm tin “không có đường rẻ hơn tới sau” chỉ đúng khi mọi cạnh làm chi phí tăng.                                                            |
| ●● Heap so với mảng đã sắp               | Mảng đã sắp: dữ liệu tĩnh, cần truy vấn theo hạng. Heap: có chèn liên tục, chỉ cần cực trị. Cần cả hai thì dùng cây cân bằng ( $B/09$ ).                                                           |
| ● Fibonacci heap                         | Giảm khoá $O(1)$ khẩu hao, cho Dijkstra $O(m + n \log n)$ ; hằng số lớn nên hiếm khi thắng trong thực tế.                                                                                          |
| ●● Fibonacci heap có đáng dùng không     | Cận $O(m + n \log n)$ cho Dijkstra là thật, nhưng đo thực nghiệm cho thấy pairing heap thắng nó trên mọi tải và thời gian chạy bám theo tỉ lệ trượt cache L1. Thi đấu: heap nhị phân kèm xoá lười. |

●●● phải thuộc    ●● nên nắm    ● biết là có

## I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào tôi nên viết lại đệ quy thành vòng lặp với ngăn xếp tường minh?

| Chủ yếu khi độ sâu đệ quy có thể lớn — duyệt sâu trên đồ thị  $10^5$  đỉnh dạng đường thẳng sẽ làm tràn ngăn xếp trong nhiều môi trường. Lý do phụ nhưng cũng quan trọng: khi bạn cần lưu trạng thái tùy biến ở mỗi mức, hoặc cần tạm dừng và tiếp tục việc duyệt. Cái giá là code dài hơn và dễ sai hơn, nên đừng làm khi không cần: đệ quy dễ đọc hơn và khớp với chứng minh quy nạp ( $A/04$ ).

2. Ngăn xếp đơn điệu tăng hay giảm — làm sao khỏi nhầm?

| Đừng nhớ theo tên, hãy suy từ mục tiêu. Nếu bạn tìm “phần tử lớn hơn kế tiếp”, thì những phần tử còn chờ đáp án phải là những phần tử chưa gặp ai lớn hơn — và nếu  $a_i \leq a_j$  với  $i < j$  thì  $a_i$  không thể còn chờ khi  $a_j$  đã vào, nên ngăn xếp giữ giá trị giảm dần từ đáy lên đỉnh. Cách kiểm tra nhanh: viết mảng bốn phần tử ra giấy và chạy tay — nhanh hơn nhớ quy tắc.

3. Tôi dùng hàng đợi ưu tiên cho Dijkstra và bị quá thời gian. Nguyên nhân hay gặp?

| Ba nguyên nhân, theo thứ tự tần suất. Một, quên bỏ qua bản ghi lỗi thời khi lấy ra — không có dòng kiểm tra đó thì mỗi đỉnh bị xử lý nhiều lần và số cạnh duyệt tăng vọt. Hai, đẩy vào hàng đợi mọi cạnh kể cả khi không cải thiện được khoảng cách. Ba, dùng cấu trúc nặng làm khoá (chuỗi, cặp lớn) khiến hàng số so sánh cao. Kiểm điểm một trước vì nó là lỗi phổ biến nhất.

4. Deque và hai heap — khi nào dùng cái nào cho bài cửa sổ trượt?

| Nếu bạn chỉ cần cực trị của cửa sổ, deque đơn điệu cho  $O(n)$  và hằng số rất nhỏ — luôn chọn nó. Nếu bạn cần trung vị hoặc phần tử thứ  $k$  của cửa sổ thì deque không làm được, và bạn cần hai heap có xoá lười hoặc một cây cân bằng (B/09), chấp nhận  $O(n \log n)$ . Nguyên tắc chung: dùng cấu trúc yếu nhất còn trả lời được câu hỏi của bạn.

5. Vì sao heapsort ít được dùng dù có đảm bảo  $O(n \log n)$  ở trường hợp xấu nhất?

| Vì nó truy cập bộ nhớ theo kiểu nhảy cóc giữa các tầng của cây, rất không thân thiện với cache, trong khi quicksort quét gần như tuần tự. Kết quả là quicksort thường nhanh hơn đáng kể trên máy thật dù tệ hơn trên giấy. Giải pháp thực tế của các thư viện là kết hợp: bắt đầu bằng quicksort, chuyển sang heapsort khi độ sâu đệ quy vượt ngưỡng — đúng ý tưởng introsort[4], một ví dụ đẹp cho chương E/26.

6. Có cấu trúc nào thay heap được không?

| Có vài lựa chọn tùy bối cảnh. Nếu khoá là số nguyên nhỏ và tăng dần theo thời gian — ví dụ Dijkstra với trọng số nhỏ — thì hàng đợi theo nhóm (bucket queue) cho  $O(1)$  mỗi thao tác. Nếu chỉ có hai loại trọng số 0 và 1, deque thay được hoàn toàn và cho thuật toán 0–1 BFS (C/17). Nếu cần hợp nhất hai hàng đợi thường xuyên, các cấu trúc heap có thể hợp nhất như pairing heap sẽ hợp hơn. Câu hỏi luôn là: cấu trúc yếu nhất còn đủ dùng ở đây là gì.

7. Trong hệ thống thật, hàng đợi ưu tiên có nhược điểm gì mà bài tập không nhắc?

| Có, và nó quan trọng: sự bỏ đói. Nếu luôn phục vụ phần tử ưu tiên cao nhất, các phần tử ưu tiên thấp có thể chờ vô hạn. Hệ điều hành và bộ lập lịch tác vụ giải quyết bằng cách tăng dần độ ưu tiên theo thời gian chờ, tức là biến khoá thành hàm của cả nội dung lẫn thời gian. Đây là ví dụ điển hình của việc bài toán thật có thêm ràng buộc công bằng mà mô hình thuật toán thuần túy bỏ qua — chủ đề của E/26.

**Bài tập** (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                              | Nguồn                                                              | Điều cần nhận ra                                                                                   |
|--------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| ● Kiểm tra dãy ngoặc hợp lệ                      | LeetCode “Valid Parentheses”                                       | Ứng dụng ngăn xếp cổ điển nhất; chú ý trường hợp còn dư ngoặc mở khi hết chuỗi.                    |
| ● Trộn $k$ danh sách đã sắp                      | LeetCode “Merge k Sorted Lists”                                    | Heap kích thước $k$ ; so sánh với cách trộn đôi một để thấy $O(N \log k)$ từ đâu ra.               |
| ● $k$ phần tử lớn nhất trong luồng               | LeetCode “Kth Largest Element in a Stream”                         | Heap nhỏ nhất kích thước $k$ ; giải thích vì sao không dùng heap lớn nhất kích thước $n$ .         |
| ●● Phần tử lớn hơn kế tiếp cho mọi vị trí        | CSES “Nearest Smaller Values”[1]                                   | Ngăn xếp đơn điệu; viết bất biến ra trước khi cài.                                                 |
| ●● Cực đại trên mọi cửa sổ độ dài $k$            | CSES “Sliding Window Cost”-loại; LeetCode “Sliding Window Maximum” | Deque đơn điệu; chú ý lưu chỉ số chứ không lưu giá trị.                                            |
| ●● Trung vị của luồng dữ liệu                    | LeetCode “Find Median from Data Stream”                            | Hai heap cân bằng; bất biến về chênh lệch kích thước là chỗ dễ sai nhất.                           |
| ●● Nhiệm vụ và hạn chót, tối đa số việc làm được | Bài tập kinh điển                                                  | Tham lam kết hợp heap: nhận hết rồi bỏ việc tệ nhất khi quá tải — mẫu “tham lam có hối tiếc”.      |
| ●●● Hình chữ nhật lớn nhất trong biểu đồ cột     | LeetCode “Largest Rectangle in Histogram”                          | Ngăn xếp đơn điệu; mẫu chót là hiểu rằng lúc một cột bị lấy ra thì cả hai biên của nó đã xác định. |

| Bài                                                         | Nguồn                               | Điều cần nhận ra (tiếp)                                                                                                  |
|-------------------------------------------------------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| ●●● Hình chữ nhật toàn số 1 lớn nhất trong ma trận nhị phân | LeetCode “Maximal Rectangle”        | Quy về bài biểu đồ cột trên từng hàng — ví dụ đẹp về việc quy bài lạ về bài đã giải.                                     |
| ●●● Nước mưa mắc kẹt bằng ngăn xếp                          | LeetCode “Trapping Rain Water”      | Đã gặp ở B/06 bằng hai con trỏ; lần này bắt buộc dùng ngăn xếp để thấy hai góc nhìn.                                     |
| ●●● Tổng của cực tiểu trên mọi đoạn con                     | LeetCode “Sum of Subarray Minimums” | Với mỗi phần tử, đếm số đoạn mà nó là cực tiểu bằng ngăn xếp đơn điệu; cẩn thận với phần tử bằng nhau để khỏi đếm trùng. |
| ●●● Sắp xếp ngoài: trộn nhiều tệp đã sắp lớn hơn bộ nhớ     | Bài tập hệ thống                    | Heap $k$ đường; nối trực tiếp với mô hình bộ nhớ ngoài ở D/24.                                                           |

## Kết chương và đọc tiếp

Ba cấu trúc, ba quy tắc chọn phần tử tiếp theo, và một bài học chung đáng mang theo cả phần B: hãy dùng cấu trúc *yếu nhất mà vẫn trả lời được câu hỏi của bạn*. Heap rẻ hơn cây cân bằng vì nó biết ít hơn; deque rẻ hơn heap vì nó biết còn ít hơn nữa. Mỗi lần bạn đòi hỏi thêm một khả năng, bạn trả thêm tiền — và rất nhiều lời giải chậm là do đòi hỏi những khả năng không dùng tới.

Hai chương tiếp theo là hai câu trả lời đối lập cho bài toán tra cứu. Chương B/08 hỏi “làm sao tìm được phần tử theo khoá thật nhanh nếu tôi *không* cần thứ tự”, và câu trả lời là băm — kèm một cam bẫy an ninh mà rất ít giáo trình nói tới. Chương B/09 hỏi câu ngược lại: nếu tôi *cần* thứ tự thì phải trả thêm bao nhiêu.

**Đọc thêm: công trình gốc** (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

| Mức         | Công trình và tác giả                                                                          | Nơi công bố, năm                                     | Vì sao đọc, đọc phần nào                                                                                                                                |
|-------------|------------------------------------------------------------------------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| nền tảng    | Fredman & Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms” | Journal of the ACM, 1987                             | Nguồn của cận $O(m + n \log n)$ cho Dijkstra. Đọc mục mô tả cấu trúc và hàm thế năng; phần ứng dụng cho cây khung nhỏ nhất đọc sau cũng được.           |
| nền tảng    | Larkin, Sen, Tarjan, “A Back-to-Basics Empirical Study of Priority Queues”                     | ALENEX (SIAM), 2014                                  | Số đo thật cho mười biến thể heap. Đọc phần kết luận và bảng Dijkstra trên bản đồ đường — đây là tài liệu chữa bệnh “chọn cấu trúc theo bảng tiệm cận”. |
| tra khi cần | Tarjan, “Amortized Computational Complexity”                                                   | SIAM Journal on Algebraic and Discrete Methods, 1985 | Đọc khi muốn hiểu phương pháp thế năng đủ kỹ để tự phân tích một cấu trúc hoãn việc.                                                                    |
| tra khi cần | Sedgewick & Wayne, <i>Algorithms</i> (4th ed.), chương 2.4                                     | Addison-Wesley, 2011                                 | Bản trình bày heap nhị phân kèm số đo và mã chạy được; hợp để đối chiếu cài đặt của bạn.                                                                |

# Nguồn

---

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [4] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [5] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [6] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [7] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [8] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [9] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [10] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [11] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] AtCoder. “AC Library: Lazy Segtree.” (), address: [https://atcoder.github.io/ac-library/master/document\\_en/lazysegtree.html](https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html) (visited on 09/07/2026).
- [13] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [14] R. E. Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets,” *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [15] M. L. Fredman and M. E. Saks, “The Cell Probe Complexity of Dynamic Data Structures,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [16] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [17] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [18] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-line String Searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

## Băm và tập hợp (Hashing and Sets)

**PHỤ THUỘC** A/05 cho xác suất và nghịch lý ngày sinh; B/06 cho mẫu "tổng tiền tố cộng bảng băm". Chuẩn bị cho B/12 (băm chuỗi) và C/20 (ngẫu nhiên hoá).

### TÓM TẮT

Băm là cách đổi một khoá bất kỳ thành một chỉ số mảng, để tra cứu trở thành truy cập trực tiếp. Nó cho  $O(1)$  trung bình cho thêm, xoá và tìm — rẻ hơn hẳn cây tìm kiếm — nhưng đánh đổi ba thứ: không có thứ tự, không có đảm bảo trường hợp xấu nhất, và dễ bị tấn công có chủ đích. Chương này đi qua cách chọn hàm băm, hai cách xử lý va chạm, ý nghĩa của hệ số tải, nghịch lý ngày sinh và hệ quả của nó với băm chuỗi, cùng hai cấu trúc xác suất là Bloom filter và bộ đếm số phần tử phân biệt. Nếu chỉ nhớ một điều: bảng băm nhanh vì nó *giả định dữ liệu không thù địch*, và trong lập trình thi đấu cũng như trong hệ thống công khai, giả định đó bị phá bằng cách trộn một hạt giống ngẫu nhiên vào hàm băm.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm băm (hash function)** — hàm biến một khoá thành một số nguyên trong khoảng  $[0, m)$ . Yêu cầu duy nhất về mặt đúng đắn: khoá bằng nhau phải cho cùng giá trị; yêu cầu về hiệu năng: các khoá khác nhau nên rải đều.
- **va chạm (collision)** — hai khoá khác nhau cho cùng giá trị băm. Không tránh được vì số khoá khả dĩ luôn lớn hơn số ô.
- **hệ số tải (load factor)** — tỉ số giữa số phần tử và số ô của bảng,  $\alpha = n/m$ . Quyết định chi phí trung bình.
- **ngịch lý ngày sinh** — trong  $k$  giá trị lấy ngẫu nhiên từ  $N$  khả năng, xác suất có hai giá trị trùng nhau trở nên đáng kể khi  $k \approx \sqrt{N}$ .
- **băm đa thức cho chuỗi** — coi chuỗi như đa thức theo cơ số  $b$ , lấy giá trị theo modulo  $m$ :  $H(s) = \sum s_i b^i \bmod m$ . Cho phép tính băm của mọi đoạn con trong  $O(1)$  sau tiền xử lý.
- **Bloom filter** — cấu trúc trả lời “phần tử này có trong tập không” với bộ nhớ rất nhỏ, không bao giờ báo sai “không có”, nhưng đôi khi báo nhầm “có”.

### CÂU HỎI MỞ ĐẦU

- Q1. Bảng băm cho  $O(1)$  trung bình còn cây cân bằng cho  $O(\log n)$ . Vì sao không phải lúc nào cũng dùng bảng băm?
- Q2. Vì sao một lời giải dùng `unordered_map` có thể bị đánh trượt trên Codeforces trong khi cùng lời giải đó dùng `map` thì qua?
- Q3. Băm chuỗi bằng một modulo  $10^9 + 7$  nghe rất an toàn vì xác suất trùng chỉ  $10^{-9}$ . Vì sao trong bài có  $10^5$  chuỗi thì lập luận đó sai?
- Q4. Bloom filter có thể báo nhầm “có” nhưng không bao giờ báo nhầm “không có”. Cấu trúc nào của nó tạo ra sự bất đối xứng đó, và vì sao bất đối xứng lại hữu ích?
- Q5. Trong hệ phân tán, thêm một máy chủ vào cụm làm hầu hết khoá phải chuyển chỗ nếu dùng `hash mod n`. Cách sửa là gì?

Bài toán tra cứu là bài toán phổ biến nhất trong lập trình: cho một khoá, tìm giá trị đi kèm. Nếu khoá là số nguyên nhỏ, lời giải hiển nhiên là dùng mảng — khoá chính là chỉ số, tra cứu tốn một phép truy cập. Toàn bộ ý tưởng băm là mở rộng lời giải hiển nhiên ấy cho khoá bất kỳ: nếu khoá không phải chỉ số, hãy ép nó thành chỉ số bằng một hàm.

Cái giá của việc ép đó là va chạm, vì số khoá khả dĩ luôn nhiều hơn số ô. Toàn bộ kỹ thuật băm — chọn hàm, xử lý va chạm, chọn kích thước bảng — chỉ là cách sống chung với va chạm sao cho chi phí trung bình vẫn gần  $O(1)$ . Và có một tình tiết mà giáo trình hay bỏ qua nhưng lại quyết định trong thực tế: mọi phân tích “trung bình” ở đây đều giả định dữ liệu không được chọn bởi người muốn hại bạn.

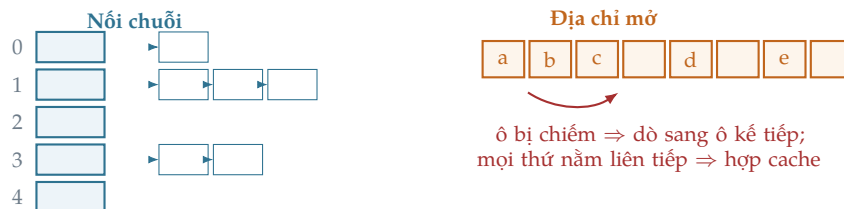
### 3.1 Hàm băm và hai cách sống chung với va chạm

Một hàm băm tốt cần hai tính chất: tính được nhanh, và rải các khoá thực tế đều trên các ô. Tính chất thứ hai quan trọng hơn về ngoài của nó, vì các khoá thực tế thường có cấu trúc — toạ độ trên lưới, chuỗi có tiền tố chung, số bội của một hằng — và một hàm băm ngây thơ sẽ dồn chúng vào ít ô.

Khi va chạm xảy ra, có hai cách xử lý.

**Nối chuỗi:** mỗi ô giữ một danh sách các phần tử băm vào đó. Đơn giản, chịu được hệ số tải lớn hơn 1, nhưng mỗi phần tử tốn thêm một con trỏ và việc lần theo danh sách không thân thiện với cache.

**Địa chỉ mở:** mọi phần tử nằm ngay trong mảng; nếu ô đã bị chiếm thì dò sang ô kế tiếp theo một quy tắc. Bộ nhớ gọn, truy cập tuần tự nên rất hợp cache — các bảng băm nhanh nhất trong thực tế đều dùng cách này. Cái giá: hệ số tải phải giữ dưới khoảng 0,7 nếu không chi phí bùng nổ, và việc xoá phần tử phức tạp hơn (phải đánh dấu “đã xoá” thay vì để trống).



**Hình 3.1.** Hai cách xử lý va chạm. Nối chuỗi đơn giản và chịu được tải cao, nhưng mỗi lần lần theo con trỏ là một lần có nguy cơ trượt cache. Địa chỉ mở giữ mọi thứ trong một mảng liên tiếp nên nhanh hơn đáng kể trên máy thật, đổi lại phải giữ hệ số tải thấp và xử lý xoá bằng cách đánh dấu.

Về chi phí: với nối chuỗi, độ dài danh sách trung bình là  $\alpha$ , nên chi phí tìm là  $O(1 + \alpha)$ . Với địa chỉ mở và dò tuyến tính, chi phí trung bình tăng vọt khi  $\alpha$  tiến gần 1 — đó là lý do bảng băm phải *tự mở rộng*: khi vượt ngưỡng tải, cấp phát bảng lớn gấp đôi và băm lại toàn bộ. Chi phí băm lại là  $O(n)$  nhưng khấu hao thành  $O(1)$  mỗi thao tác, đúng lập luận nhân đôi ở chương A/03.

### 3.2 Chỗ mà giáo trình nói “trung bình” và thực tế nói “kẻ tấn công”

Đây là mục quan trọng nhất chương, và cũng là mục hay bị bỏ.

Mọi phân tích trên đều giả định các khoá rải đều trên các ô. Nhưng hàm băm là một hàm *tất định*: nếu ai đó biết hàm băm của bạn, họ có thể chuẩn bị một tập khoá mà tất cả cùng băm vào

một ô. Khi đó bảng băm thoái hoá thành một danh sách liên kết, và mọi thao tác thành  $O(n)$ . Với  $n = 10^5$  phần tử, lời giải  $O(n)$  trên mỗi thao tác trở thành  $10^{10}$  — quá thời gian chắc chắn.

Điều này không phải chuyện lý thuyết. Trong lập trình thi đấu, các bộ test chống `unordered_map` của C++ được viết ra chính bằng cách này và rất phổ biến. Trong hệ thống thật, hiện tượng tương tự từng được dùng để tấn công máy chủ web: gửi một yêu cầu chứa hàng nghìn tham số cùng băm vào một ô làm máy chủ tiêu tốn thời gian gấp hàng nghìn lần bình thường.

Cách chữa giống nhau ở cả hai bối cảnh: *làm cho hàm băm không đoán trước được*, bằng cách trộn vào một hạt giống ngẫu nhiên chọn lúc chạy. Về mặt lý thuyết, ý tưởng này có tên là *băm phổ dụng* — chọn ngẫu nhiên một hàm từ một họ hàm có tính chất “với hai khoá bất kỳ, xác suất va chạm nhỏ” — và điểm mấu chốt là ngẫu nhiên nằm ở *lựa chọn của bạn*, không ở dữ liệu. Nhờ vậy đảm bảo vẫn đúng ngay cả khi dữ liệu do đối thủ chọn, miễn là họ không biết hạt giống. Đây là một ví dụ mẫu mực cho toàn bộ chương C/20: ngẫu nhiên hoá không phải để hy vọng gặp may, mà để *tước bỏ khả năng dự đoán của đối thủ*.

#### CẠM BẤY

Trong C++, `unordered_map` với khoá số nguyên dùng một hàm băm rất đơn giản và có thể đoán được, nên nó bị đánh trượt thường xuyên trên các kỳ thi. Cách chữa tiêu chuẩn là cung cấp hàm băm riêng có trộn hạt giống lấy từ đồng hồ hệ thống. Trong ngôn ngữ khác, hãy kiểm tra xem thư viện chuẩn có ngẫu nhiên hoá hạt giống theo tiến trình hay không — nhiều ngôn ngữ hiện đại đã làm sẵn việc này sau các đợt tấn công vào máy chủ web.

### 3.3 Bảo đảm mạnh hơn: băm cuckoo và băm bảng tra

Băm phổ dụng cứu được trường hợp xấu *kỳ vọng*, nhưng vẫn không hứa gì về một thao tác cụ thể: một lần tra vẫn có thể phải lần qua một chuỗi dài. Với hệ thống có hạn kỳ thời gian — bộ định tuyến, bộ điều khiển, phần cứng — kỳ vọng là không đủ. Hai kết quả sau cho thấy có thể đòi hỏi nhiều hơn, và cả hai đều đáng biết vì chúng đổi cách nghĩ chứ không chỉ thêm một cấu trúc.

*Băm cuckoo*[5] lật ngược câu hỏi: thay vì cho phép một khoá nằm ở nhiều chỗ tùy theo lịch sử va chạm, hãy quy định mỗi khoá chỉ được nằm ở *đúng hai* vị trí, cho bởi hai hàm băm. Khi đó tra cứu là hai lần đọc bộ nhớ, không hơn —  $O(1)$  trong *trường hợp xấu nhất*, và xoá cũng đơn giản vì không có ô “đã xoá” nào phải đánh dấu. Cái giá đền hết sang phép chèn: nếu cả hai vị trí đều bị chiếm, ta *đá* kẻ đang ngồi ở một trong hai ô sang vị trí thay thế của nó, và kẻ bị đá lại đá tiếp người khác. Chuỗi đá này kết thúc nhanh nếu bảng còn đủ rộng — điều kiện quen thuộc là hệ số tải dưới  $1/2$  — còn nếu nó dài quá ngưỡng thì ta bốc lại hai hàm băm mới và xây lại bảng. Kết quả: chèn tốn  $O(1)$  kỳ vọng theo nghĩa khấu hao, đổi lấy bộ nhớ rộng gấp đôi và một cái đuôi hiếm hoi rất đắt.

Điểm đáng học nằm ở chỗ đánh đổi được *dời* chứ không biến mất: cùng một bài toán, nhưng chi phí bị đẩy hết từ phía đọc sang phía ghi. Đó là lựa chọn đúng cho các bảng viết một lần đọc rất nhiều lần, và là lựa chọn sai cho tài ghi liên tục.

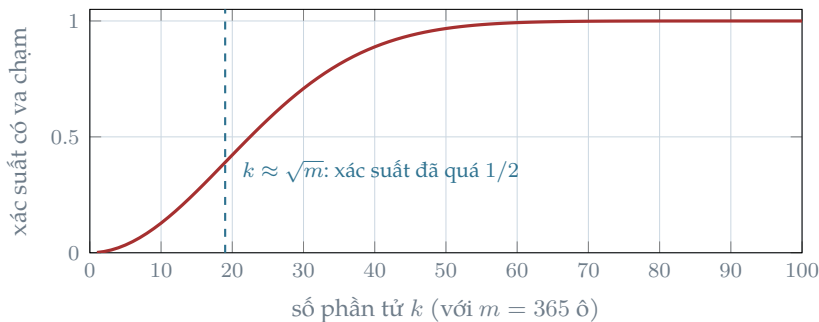
*Băm bảng tra*[6] trả lời một câu hỏi khác: hàm băm phải “ngẫu nhiên” tới mức nào thì các phân tích lý thuyết mới còn đúng? Cách làm đơn giản tới mức đáng ngờ — cất khoá thành vài ký tự, mỗi ký tự tra một bảng số ngẫu nhiên đã dựng sẵn, rồi XOR tất cả lại. Hàm như vậy chỉ đọc lập bậc ba, thấp hơn nhiều so với mức mà các chứng minh cổ điển đòi hỏi, nên theo lý thuyết cũ thì nó không đủ tốt. Pătraşcu và Thorup chứng minh rằng nó vẫn cho các chặn tập trung kiểu Chernoff, và nhờ đó dò tuyến tính lẫn băm cuckoo chạy đúng như phân tích với một hàm băm chỉ tốn vài phép tra bảng và một phép XOR.

Ba dòng đáng mang theo từ mục này. Một, “ngẫu nhiên hoá” không phải một mức duy nhất mà là cả một thang, và câu hỏi kỹ thuật luôn là *cần độc lập tới bậc mấy*. Hai, hàm băm rẻ kèm một phân tích tinh vi thường thắng hàm băm đắt kèm một phân tích thô. Ba, khi bài toán đòi bảo đảm cho *từng* thao tác chứ không cho trung bình, hãy hỏi xem có cấu trúc nào chuyển chi phí sang phía còn lại không — đây là mẫu tư duy dùng lại được, không riêng gì bảng băm.

### 3.4 Nghịch lý ngày sinh và băm chuỗi

Băm chuỗi là công cụ mạnh: coi chuỗi như một đa thức, ta tính được giá trị băm của *mọi* đoạn con trong  $O(1)$  sau khi tiền xử lý  $O(n)$ , nên so sánh hai đoạn con bất kỳ trở thành  $O(1)$ . Rất nhiều bài chuỗi khó sụp xuống thành dễ nhờ điều đó (B/12).

Nhưng có một cái bẫy xác suất mà nhiều người tính sai. Nếu modulo là  $10^9 + 7$ , xác suất hai chuỗi cụ thể trùng băm là khoảng  $10^{-9}$  — nghe rất an toàn. Sai lầm là ở chỗ bài toán thường so sánh *rất nhiều* cặp. Với  $k$  chuỗi, số cặp là  $k^2/2$ , nên xác suất có ít nhất một va chạm xấp xỉ  $k^2/(2m)$ . Với  $k = 10^5$  và  $m = 10^9$ , con số đó là khoảng 5, tức là gần như chắc chắn có va chạm. Đây chính là nghịch lý ngày sinh: va chạm trở nên đáng kể khi số phần tử đạt cỡ  $\sqrt{m}$ , chứ không phải cỡ  $m$ .



**Hình 3.2.** Nghịch lý ngày sinh với  $m = 365$ . Trực giác nói phải cần cỡ  $m/2$  phần tử mới có va chạm; thực tế chỉ cần cỡ  $\sqrt{m}$ , tức khoảng 23 người. Hệ quả trực tiếp cho băm chuỗi: modulo  $10^9$  chỉ “an toàn” tới cỡ  $3 \cdot 10^4$  chuỗi, nên bài có  $10^5$  chuỗi cần hai modulo hoặc một modulo cỡ  $10^{18}$ .

Hai cách chữa. Dùng *hai modulo* và coi cặp giá trị là mã băm — xác suất va chạm trở thành tích của hai xác suất, đủ nhỏ cho mọi bài thực tế. Hoặc dùng *một modulo* cỡ  $10^{18}$  với phép nhân cẩn thận để không tràn. Và như mục trước: cơ sở của đa thức phải chọn *ngẫu nhiên lúc chạy*, vì băm chuỗi với cơ sở cố định cũng bị dựng test phá.

### 3.5 Cấu trúc xác suất: chấp nhận sai một chiều để tiết kiệm bộ nhớ

Khi tập dữ liệu quá lớn để lưu, có một hướng đi rất khác: chấp nhận trả lời sai với xác suất nhỏ, đổi lấy bộ nhớ nhỏ hơn nhiều bậc.

*Bloom filter* là ví dụ kinh điển. Ta giữ một mảng bit và  $k$  hàm băm. Thêm phần tử: bật  $k$  bit tương ứng. Hỏi phần tử: nếu có bit nào bằng 0 thì chắc chắn phần tử chưa từng được thêm; nếu mọi bit đều bằng 1 thì *có lẽ* đã thêm — có thể các bit đó do những phần tử khác bật lên. Sự bất đối xứng ấy chính là điều làm nó hữu ích: nó dùng ở vị trí bộ lọc đứng trước một phép tra cứu đắt tiền, nơi “chắc chắn không có” cho phép bỏ qua ngay, còn “có lẽ có” chỉ tốn thêm một lần kiểm tra thật. Cơ sở dữ liệu dùng nó để tránh đọc đĩa vô ích; hệ thống mạng dùng nó để lọc yêu cầu.

Đếm số phần tử phân biệt là bài toán họ hàng. Đếm chính xác cần bộ nhớ tỉ lệ với số phần tử phân biệt; nhưng nếu chấp nhận sai số vài phần trăm thì có thể dùng vài kilobyte cho hàng tỉ phần tử.

Ý tưởng gốc rất đẹp: nếu các giá trị băm rải đều, thì giá trị băm nhỏ nhất mà bạn nhìn thấy nói cho bạn biết bạn đã nhìn bao nhiêu phần tử phân biệt — càng nhiều phần tử thì cực tiểu càng nhỏ. Cấu trúc HyperLogLog[7] là bản tinh chỉnh của ý tưởng đó và được dùng rộng rãi trong các hệ thống đo lường.

### 3.6 Khi nào không dùng băm

Bảng băm mất ba thứ so với cấu trúc có thứ tự, và mỗi thứ đều là lý do đủ để chọn khác.

*Không có thứ tự.* Không hỏi được “khoá nhỏ nhất lớn hơn  $x$ ”, không duyệt được theo thứ tự tăng, không truy vấn được đoạn. Bài nào có chữ “gần nhất”, “lớn hơn hoặc bằng”, “trong khoảng” thì băm không dùng được, và bạn cần chương B/09 hoặc B/10.

*Không có đảm bảo trường hợp xấu nhất.* Trong hệ thống có hạn kỳ thời gian thực, một thao tác đột ngột tốn  $O(n)$  vì băm lại hoặc vì cụm va chạm là không chấp nhận được.

*Chi phí ẩn khi khoá lớn.* Băm một chuỗi dài tốn theo độ dài chuỗi, nên  $O(1)$  chỉ là  $O(1)$  phép băm, không phải  $O(1)$  thao tác cơ bản.

Ngược lại, có một trường hợp mà băm không phải lựa chọn tốt nhất mà là lựa chọn *sai*: khi khoá là số nguyên trong một khoảng nhỏ. Lúc đó mảng thường luôn thắng — nhanh hơn, đơn giản hơn, không có va chạm. Người mới hay dùng bảng băm cho khoá  $0..10^6$ , trong khi một mảng `int` một triệu phần tử chỉ tốn 4 MB và nhanh hơn nhiều lần.

### 3.7 Gốc rễ: từ bảng ký hiệu của trình dịch hợp ngữ tới hệ phân tán

Băm ra đời cùng lúc với những trình biên dịch đầu tiên, và vì một lý do rất cụ thể. Trình dịch hợp ngữ cần một *bảng ký hiệu*: ánh xạ tên biến sang địa chỉ. Tên biến là chuỗi, bộ nhớ thì đánh số, nên cần một câu nối — và câu nối đó là hàm băm. Ý tưởng được ghi nhận trong một ghi chú nội bộ của Hans Peter Luhn tại IBM năm 1953, và kỹ thuật địa chỉ mở xuất hiện ngay sau đó trong các trình dịch hợp ngữ giữa thập niên 1950. Đây là một ví dụ điển hình cho mẫu lặp đi lặp lại trong lịch sử ngành: cấu trúc dữ liệu quan trọng thường sinh ra từ nhu cầu của công cụ hệ thống, không từ lý thuyết.

Phần lý thuyết đến sau và giải một vấn đề thật. Suốt hai thập niên, người ta biết bảng băm nhanh “trung bình” nhưng không có cách nào đảm bảo điều đó khi dữ liệu không ngẫu nhiên. Năm 1979, Carter và Wegman đề xuất băm phổ dụng: thay vì hy vọng dữ liệu ngẫu nhiên, hãy *chọn hàm băm một cách ngẫu nhiên*. Đây là một chuyển dịch tư duy quan trọng và nó lặp lại ở nhiều nơi khác trong quyển sách này — quicksort với chốt ngẫu nhiên (C/14) là cùng một ý.

Bloom filter (1970) sinh ra từ ràng buộc bộ nhớ của thời đó: kiểm tra một từ có nằm trong từ điển gạch nối hay không, khi từ điển không nạp hết vào bộ nhớ được. Bài toán cũ đó biến mất, nhưng cấu trúc thì sống rất khoẻ trong các hệ thống hiện đại, ở đúng vai trò “lọc trước một phép tra cứu đắt tiền”.

Cuối cùng, một gốc rễ hiện đại hơn và có ảnh hưởng lớn: *băm nhất quán*, do nhóm của Karger đề xuất năm 1997 cho bài toán bộ nhớ đệm web phân tán. Vấn đề rất thực tế: nếu ánh xạ khoá tới máy chủ bằng  $\text{hash mod } n$  thì thêm hay bớt một máy làm gần như *toàn bộ* khoá đổi chỗ. Băm nhất quán xếp cả khoá lẫn máy chủ lên một vòng tròn và giao mỗi khoá cho máy chủ kế tiếp theo chiều kim đồng hồ; khi thêm một máy, chỉ các khoá thuộc một cung bị ảnh hưởng. Kỹ thuật này về sau trở thành nền của nhiều hệ cơ sở dữ liệu phân tán, và nó là câu trả lời cho câu hỏi mở đầu thứ năm.

### I Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                                          | Bài toán thúc ép lúc đó                                                   | Điều thay đổi sau đó                                                                                              |
|----------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 1953 — Luhn (IBM)                            | Bảng ký hiệu của trình dịch: ánh xạ tên biến (chuỗi) sang địa chỉ (số).   | Ý tưởng băm; ra đời từ nhu cầu công cụ hệ thống chứ không từ lý thuyết.                                           |
| Giữa thập niên 1950 — các trình dịch hợp ngữ | Bộ nhớ rất hẹp, không đủ chỗ cho danh sách liên kết mỗi ô.                | Địa chỉ mở với dò tuyến tính; cách cài vẫn nhanh nhất trên phần cứng hiện đại vì hợp cache.                       |
| 1970 — Bloom                                 | Kiểm tra từ có trong từ điển không, khi từ điển không nạp hết vào bộ nhớ. | Bloom filter: chấp nhận sai một chiều để giảm bộ nhớ nhiều bậc; nay dùng làm bộ lọc trước truy cập đĩa.           |
| 1979 — Carter & Wegman                       | “Trung bình nhanh” không còn đúng khi dữ liệu do người khác chọn.         | Băm phổ dụng: ngẫu nhiên hoá <i>lựa chọn hàm</i> , không phụ thuộc dữ liệu; nền cho cách chống test phá ngày nay. |
| 1997 — Karger và cộng sự                     | Bộ nhớ đệm web phân tán: thêm một máy chủ làm mọi khoá đối chỗ.           | Băm nhất quán; trở thành cơ sở phân mảnh dữ liệu của nhiều hệ phân tán hiện đại.                                  |
| 2007 — Flajolet và cộng sự                   | Đếm số phần tử phân biệt trong luồng hàng tỉ bản ghi với vài kilobyte.    | HyperLogLog; hoàn thiện dòng nghiên cứu ước lượng lực lượng bắt đầu từ thập niên 1980.                            |

### I 3.8 Liên hệ ngang

#### I Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực                     | Ý tưởng tương ứng                                                | Giống ở đâu, khác ở đâu                                                                                                                                                                                                                                                |
|------------------------------|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hệ phân tán                  | Băm nhất quán, phân mảnh dữ liệu                                 | Cùng hàm băm, khác mục tiêu: ở đây không phải để tra cứu nhanh mà để <i>ổn định ánh xạ</i> khi số máy thay đổi. Bài học: khi tập ô có thể thay đổi, mod $n$ là lựa chọn tệ.                                                                                            |
| Hệ thống kiểm soát phiên bản | Git định danh mọi đối tượng bằng mã băm nội dung                 | Băm dùng làm <i>địa chỉ</i> chứ không làm chỉ số: nội dung giống nhau thì tự động chung một tên. Điều kiện để dùng được là hàm băm mật mã, tức va chạm phải khó tạo ra có chủ đích — yêu cầu mạnh hơn hẳn băm thông thường.                                            |
| Mật mã                       | Hàm băm mật mã, cây Merkle, chữ ký                               | Cùng tên gọi nhưng yêu cầu khác hẳn: ở đây bắt buộc <i>không ai tạo được va chạm</i> , kể cả khi cố tình và có nhiều tài nguyên. Vì vậy chúng chậm hơn nhiều và không nên dùng làm hàm băm cho bảng băm.                                                               |
| Cơ sở dữ liệu                | Nối bằng băm (hash join), chỉ mục băm, Bloom filter chặn đọc đĩa | Cùng đánh đổi với chương này: chỉ mục băm nhanh hơn chỉ mục cây cho truy vấn bằng, nhưng vô dụng cho truy vấn khoảng — đó là lý do B-tree vẫn là mặc định (B/09).                                                                                                      |
| Nhận dạng ảnh, tìm gần đúng  | Băm nhảy cảm cục bộ (LSH): khoá gần nhau thì băm giống nhau      | Đảo ngược yêu cầu của chương này. Băm thường muốn rải đều để tránh va chạm; LSH cố ý tạo va chạm cho các phần tử tương tự, để tìm hàng xóm gần trong không gian nhiều chiều.                                                                                           |
| SLAM và nhận dạng địa điểm   | Túi từ thị giác, cây từ vựng, mô tả nhị phân                     | Cùng ý tưởng LSH: gán mỗi đặc trưng ảnh vào một “ô” để so khớp nhanh giữa hàng nghìn khung hình. Điểm giống quan trọng: cả hai đều chấp nhận sai sót và bù lại bằng bước kiểm chứng hình học phía sau — kiến trúc “lọc rê trước, kiểm đất sau” giống hệt Bloom filter. |

| Lĩnh vực                | Ý tưởng tương ứng                          | Giống ở đâu, khác ở đâu (tiếp)                                                                                                                             |
|-------------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chống trùng lặp dữ liệu | Lưu trữ theo nội dung, phát hiện tệp trùng | Băm để nhận diện thay vì để tra cứu. Cùng lưu ý về nghịch lý ngày sinh: số đối tượng càng lớn thì độ dài mã băm cần thiết càng phải tăng theo bình phương. |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì  $O(1)$  của bảng băm là trung bình chứ không phải đảm bảo, và vì nó mất thứ tự. Ba tình huống buộc phải dùng cây: bài cần truy vấn “khoá gần nhất”, “lớn hơn hoặc bằng  $x$ ” hay duyệt theo thứ tự; hệ thống có hạn kỳ thời gian thực không chịu được một thao tác đột ngột  $O(n)$ ; và trường hợp khoá do đối thủ chọn mà bạn không ngẫu nhiên hoá được hàm băm. Ngoài ra, khi khoá là số nguyên trong khoảng nhỏ thì mảng thường thắng cả hai.
- A2. Vì hàm băm mặc định cho số nguyên trong nhiều cài đặt là hàm đồng nhất hoặc gần như vậy, và kích thước bảng có thể đoán được. Người viết test chọn một tập khoá cùng số dư theo kích thước bảng, làm mọi khoá dồn vào một ô; bảng băm thoái hoá thành danh sách và mỗi thao tác thành  $O(n)$ . Cây cân bằng không có điểm yếu này vì  $O(\log n)$  của nó là đảm bảo cho mọi dữ liệu. Cách chữa là trộn hạt giống ngẫu nhiên lấy lúc chạy vào hàm băm.
- A3. Vì cái đáng lo không phải một cặp mà là *mọi cặp*. Với  $k$  chuỗi có  $k^2/2$  cặp, nên xác suất có ít nhất một va chạm xấp xỉ  $k^2/(2m)$ . Với  $k = 10^5$  và  $m = 10^9$  thì con số này lớn hơn 1, tức gần như chắc chắn có va chạm. Đây là nghịch lý ngày sinh: ngưỡng nguy hiểm là  $\sqrt{m}$ , không phải  $m$ . Chữa bằng hai modulo hoặc một modulo cỡ  $10^{18}$ .
- A4. Bất đối xứng đến từ chỗ Bloom filter chỉ *bật* bit chứ không bao giờ tắt: nếu một bit cần thiết đang bằng 0 thì chắc chắn phần tử chưa từng được thêm, còn mọi bit bằng 1 thì có thể do các phần tử khác bật. Điều đó hữu ích vì nó đặt cấu trúc vào đúng vai trò bộ lọc: câu trả lời “chắc chắn không” cho phép bỏ qua ngay một phép tra cứu đắt tiền, còn câu trả lời “có lẽ có” chỉ tốn thêm một lần kiểm tra thật — sai lầm không gây kết quả sai, chỉ gây thêm việc.
- A5. Băm nhất quán. Thay vì  $\text{hash} \bmod n$ , xếp cả khoá lẫn máy chủ lên một vòng tròn giá trị băm, và giao mỗi khoá cho máy chủ đầu tiên gặp theo chiều kim đồng hồ. Thêm một máy chủ làm các khoá trong một cung phải chuyển, tức trung bình  $1/n$  số khoá thay vì gần như toàn bộ. Trong thực tế người ta còn đặt nhiều “nút ảo” cho mỗi máy để tải rải đều hơn.

### NÓI LẠI THẬT ĐƠN GIẢN

Băm chỉ là cách biến một cái tên thành một số ngắn tử. Nếu tử của bạn có 100 ngắn và bạn có quy tắc “lấy tên, tính ra một số từ 0 tới 99”, thì cất và tìm đồ đều chỉ mất một bước: tính số rồi mở đúng ngăn. Nhanh hơn hẳn việc mở lần lượt từng ngăn.

Rắc rối duy nhất là hai cái tên khác nhau có thể ra cùng một số ngắn. Có hai cách sống chung. Cách thứ nhất: mỗi ngăn treo một cái túi, ai vào cùng ngăn thì bỏ chung túi, lúc tìm thì lục túi. Cách thứ hai: nếu ngăn đã có người, đi sang ngăn kế bên cho tới khi gặp ngăn trống. Cách thứ hai nhanh hơn trong thực tế vì bạn đi bộ dọc dãy tử, không phải lục túi.

Có hai điều dễ nhầm và đáng nhớ. Thứ nhất, đụng ngăn xảy ra sớm hơn ta tưởng rất nhiều: trong một lớp 23 người thì đã có khoảng một nửa cơ hội có hai người trùng ngày sinh, dù một năm có 365 ngày. Nói cách khác, muốn an toàn với  $k$  món đồ thì số ngăn phải cỡ  $k^2$ , chứ không phải cỡ  $k$ . Thứ hai, nếu có người biết quy tắc đánh số ngắn của bạn, họ có thể cố ý mang tới toàn những cái tên rơi vào cùng một ngăn, và tử của bạn thành một cái túi khổng lồ phải lục từng món. Cách chống rất đơn giản: mỗi lần mở tủ buổi sáng, bạn trộn thêm một con số bí mật ngẫu nhiên vào quy tắc. Họ không đoán được nữa.

*Một câu: Băm biến tên thành ngắn tử; mọi kỹ thuật còn lại chỉ để sống chung với chuyện hai tên rơi vào một ngăn.*

**Chỗ không rút gọn được:** Không có hàm băm nào tránh được va chạm — số tên luôn nhiều hơn số ngắn; ta chỉ chọn được cách phân bố và cách phản ứng.

| Thuật ngữ / câu hỏi               | Trả lời                                                                                                                                                                                                                   |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● Ý tưởng cốt lõi của băm       | Ép khoá bất kỳ thành chỉ số mảng bằng một hàm, để tra cứu thành truy cập trực tiếp. Va chạm là không tránh được.                                                                                                          |
| ●●● Hai cách xử lý va chạm        | Nối chuỗi (mỗi ô một danh sách, chịu tải > 1) và địa chỉ mở (dò sang ô kế tiếp, hợp cache, phải giữ tải < 0,7).                                                                                                           |
| ●●● Vì sao bảng băm bị đánh trượt | Hàm băm tốt định $\Rightarrow$ đối thủ dựng được tập khoá cùng ô $\Rightarrow$ mọi thao tác thành $O(n)$ . Chứa bằng hạt giống ngẫu nhiên lúc chạy.                                                                       |
| ●●● Băm phổ dụng — ý chính        | Ngẫu nhiên nằm ở <i>lựa chọn hàm băm của bạn</i> , không ở dữ liệu; nhờ đó đảm bảo vẫn đúng với dữ liệu thù địch.                                                                                                         |
| ●●● Nghịch lý ngày sinh           | Va chạm đáng kể khi số phần tử đạt cỡ $\sqrt{m}$ , không phải $m$ . Xác suất có va chạm $\approx k^2/(2m)$ .                                                                                                              |
| ●●● Hệ quả cho băm chuỗi          | Modulo $10^9$ chỉ an toàn tới cỡ $3 \cdot 10^4$ chuỗi; bài lớn cần hai modulo hoặc modulo cỡ $10^{18}$ , và cơ sở phải ngẫu nhiên.                                                                                        |
| ●● Hệ số tải và mở rộng bảng      | $\alpha = n/m$ ; vượt ngưỡng thì cấp phát bảng gấp đôi và băm lại, chi phí $O(n)$ nhưng khấu hao $O(1)$ .                                                                                                                 |
| ●● Bloom filter                   | Mảng bit + $k$ hàm băm; “chắc chắn không có” hoặc “có lẽ có”. Dùng làm bộ lọc rẻ trước một phép tra cứu đắt.                                                                                                              |
| ●● Ba thứ băm không làm được      | Không có thứ tự (không hỏi cận trên/dưới, không duyệt tăng dần); không đảm bảo trường hợp xấu nhất; $O(1)$ chỉ tính theo số phép băm, khoá dài vẫn tồn theo độ dài.                                                       |
| ●● Khi nào băm là lựa chọn sai    | Khi khoá là số nguyên trong khoảng nhỏ — mảng thường luôn thắng; nhanh hơn, đơn giản hơn, không va chạm.                                                                                                                  |
| ●● Băm nhất quán                  | Xếp khoá và máy chủ lên vòng tròn, khoá thuộc về máy kế tiếp theo chiều kim đồng hồ; thêm máy chỉ làm $1/n$ số khoá đổi chỗ.                                                                                              |
| ● Đếm phần tử phân biệt           | Giá trị băm nhỏ nhất nhìn thấy được nói lên số phần tử phân biệt; HyperLogLog tinh chỉnh ý tưởng đó, vài KB cho hàng tỉ bản ghi.                                                                                          |
| ● Băm mật mã khác băm thường      | Yêu cầu không ai tạo được va chạm kể cả khi cố tình; chậm hơn nhiều nên không dùng cho bảng băm, nhưng cần cho định danh nội dung như trong git.                                                                          |
| ●● Băm cuckoo và băm bảng tra     | Cuckoo: mỗi khoá chỉ nằm ở một trong hai ô nên tra cứu là $O(1)$ trường hợp xấu nhất, giá dồn sang phép chen và đòi tải dưới $1/2$ . Bảng tra: hàm băm rẻ, chỉ đọc lập bậc ba, nhưng vẫn có chặn tập trung đã chứng minh. |

●●● phải thuộc    ●● nên nắm    ● biết là có

## I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên chọn kích thước bảng băm là số nguyên tố hay lũy thừa của 2?

| Lũy thừa của 2 cho phép thay phép chia lấy dư bằng phép AND bit, nhanh hơn đáng kể — nhưng nó chỉ giữ lại các bit thấp của mã băm, nên đòi hỏi hàm băm phải trộn tốt các bit. Số nguyên tố tha thứ hơn cho hàm băm kém vì phép lấy dư trộn mọi bit, nhưng phép chia chậm. Thực hành hiện đại nghiêng về lũy thừa của 2 kèm một bước trộn bit sau khi băm. Trong thi đấu, dùng lũy thừa của 2 kèm trộn hạt giống là lựa chọn an toàn.

2. Khi nào nên dùng map thay unordered\_map trong C++?

| Bốn tình huống. Khi cần thứ tự hoặc truy vấn cận (lower\_bound). Khi số phần tử nhỏ — dưới vài trăm thì hằng số của map không đáng kể và code an toàn hơn. Khi bài thi có nguy cơ bị dựng test phá và bạn không muốn viết hàm băm riêng. Và khi cần đảm bảo trường hợp xấu nhất. Ngược lại, với hàng triệu thao tác tra cứu thuần bằng khoá và có hàm băm tự viết, unordered\_map hoặc một bảng địa chỉ mở tự cài sẽ nhanh hơn nhiều lần.

3. Băm chuỗi có thể thay hoàn toàn KMP và suffix array không?

| Thay được rất nhiều bài và thường ngắn hơn hẳn, nhưng có ba giới hạn thật. Một, nó là thuật toán xác suất — xác suất sai nhỏ nhưng khác không, và trong bài yêu cầu kết quả chắc chắn đúng thì đó là điều cần cân nhắc. Hai, một số bài cần cấu trúc bên trong của chuỗi (biên tuần hoàn, số chuỗi con phân biệt) mà băm không cho, phải dùng công cụ ở [B/12](#). Ba, băm chuỗi so sánh được “bằng nhau” nhưng không so sánh được “lớn hơn” theo thứ tự từ điển, trừ khi kết hợp thêm tìm kiếm nhị phân trên độ dài tiền tố chung.

#### 4. Bloom filter chọn bao nhiêu hàm băm là tốt nhất?

| Có sự đánh đổi rõ: ít hàm băm thì mỗi phần tử bật ít bit nên mảng thưa hơn, nhưng mỗi truy vấn kiểm tra ít điều kiện nên dễ báo nhầm; nhiều hàm băm thì ngược lại. Với  $m$  bit và  $n$  phần tử, số hàm băm tối ưu tỉ lệ với  $m/n$  — cụ thể là  $(m/n) \ln 2$ , và tại điểm tối ưu thì khoảng một nửa số bit được bật. Điều đáng nhớ về mặt trực giác: mảng đầy quá nửa thì cấu trúc bắt đầu mất tác dụng.

#### 5. Vì sao không dùng luôn hàm băm mật mã cho bảng băm để khỏi lo bị tấn công?

| Vì nó chậm hơn nhiều bậc — được thiết kế để chống lại đối thủ có nhiều tài nguyên, nên có tính tốn nhiều phép toán. Với bảng băm, thứ bạn cần chỉ là đối thủ *không đoán trước được*, mà điều đó đạt được rẻ hơn nhiều bằng một hàm băm nhanh cộng một hạt giống ngẫu nhiên bí mật. Dùng đúng công cụ cho đúng mức đe dọa là nguyên tắc chung, và dùng quá mức cũng là một dạng sai.

#### 6. Trong hệ thống thật, lỗi liên quan tới băm hay gặp là gì?

| Ba lỗi phổ biến. Một, dùng mã băm làm định danh lâu dài rồi gặp va chạm khi dữ liệu lớn lên — đúng bấy ngày sinh, và nó xuất hiện muộn nên rất khó truy. Hai, giả định thứ tự duyệt của cấu trúc băm là ổn định giữa các lần chạy; nhiều thư viện hiện đại cố tình xáo thứ tự đó, và code phụ thuộc vào nó sẽ hỏng không đều. Ba, dùng hash mod  $n$  cho phân mảnh rồi phải mở rộng cụm máy — lúc đó gần như toàn bộ dữ liệu phải di chuyển, và cách sửa là băm nhất quán.

#### 7. Có cấu trúc nào cho $O(1)$ trường hợp xấu nhất chứ không chỉ trung bình không?

| Có: băm hoàn hảo. Với tập khoá *tĩnh* biết trước, người ta xây được một hàm băm không có va chạm nào, cho tra cứu  $O(1)$  đảm bảo; kỹ thuật hai tầng cổ điển đạt được điều đó với bộ nhớ tuyến tính. Hạn chế là tập khoá phải cố định, nên nó dùng cho từ khoá ngôn ngữ lập trình, bảng tra cố định trong hệ thống nhúng — những nơi dữ liệu không đổi sau khi biên dịch.

### | Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                                                 | Nguồn                      | Điều cần nhận ra                                                                                          |
|---------------------------------------------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------|
| ● Đếm số phần tử phân biệt trong mảng                               | CSES “Distinct Numbers”[1] | So sánh ba cách: sắp xếp, tập hợp có thứ tự, tập hợp băm — và đo thời gian thật.                          |
| ● Hai số có tổng bằng $x$ , mảng chưa sắp                           | LeetCode “Two Sum”         | Bảng băm một lượt; đối chiếu với cách hai con trỏ ở <a href="#">B/06</a> và nêu điều kiện chọn mỗi cách.  |
| ● Nhóm các chuỗi đảo chữ với nhau                                   | LeetCode “Group Anagrams”  | Thiết kế khoá băm là bước quyết định — chuỗi đã sắp hay vector đếm 26 phần tử.                            |
| ●● Đếm số đoạn con có tổng bằng $x$                                 | CSES “Subarray Sums II”[1] | Tổng tiền tố cộng bảng băm; chú ý tổng có thể âm và phải dùng kiểu 64 bit.                                |
| ●● Đoạn con dài nhất có tổng bằng 0                                 | Bài tập kinh điển          | Cùng mẫu: lưu lần xuất hiện <i>đầu tiên</i> của mỗi giá trị tổng tiền tố.                                 |
| ●● Cài một bảng băm địa chỉ mở của riêng bạn                        | Bài tập cài đặt            | So thời gian với thư viện chuẩn trên $10^7$ thao tác; đây là bài tập cho thấy rõ nhất tác động của cache. |
| ●● Dựng bộ test làm chậm unordered_map                              | Bài tập an ninh            | Chọn khoá cùng số dư theo kích thước bảng; sau đó chứa bằng hàm băm có hạt giống và đo lại.               |
| ●●● So sánh mọi cặp đoạn con bằng băm chuỗi                         | CSES nhóm bài chuỗi[1]     | Hai modulo; tính xác suất va chạm cho $k$ chuỗi trước khi cài, đừng đoán.                                 |
| ●●● Tìm chuỗi con lặp lại dài nhất bằng băm và nhị phân trên độ dài | Bài tập kinh điển          | Kết hợp băm với nhị phân trên đáp án ( $C/19$ ); chú ý tính đơn điệu của điều kiện.                       |
| ●●● Cài Bloom filter và đo tỉ lệ báo nhầm theo số hàm băm           | Bài tập thực nghiệm        | Vẽ đường cong tỉ lệ báo nhầm theo $k$ ; so với dự đoán lý thuyết và giải thích chỗ lệch.                  |

| Bài                                                         | Nguồn            | Điều cần nhận ra (tiếp)                                                      |
|-------------------------------------------------------------|------------------|------------------------------------------------------------------------------|
| ●●● Cài băm nhất quán với nút ảo, mô phỏng thêm/bớt máy chủ | Bài tập hệ thống | Đo tỉ lệ khoá phải di chuyển; so với mod $n$ để thấy khác biệt bằng số liệu. |

### Kết chương và đọc tiếp

Băm đổi ba thứ — thứ tự, đảm bảo xấu nhất, và tính không đoán trước — để lấy  $O(1)$  trung bình. Hai điều đáng mang theo: nghịch lý ngày sinh nói ngưỡng an toàn là  $\sqrt{m}$  chứ không phải  $m$ ; và mọi phân tích “trung bình” đều sụp nếu dữ liệu do đối thủ chọn, nên hạt giống ngẫu nhiên không phải tùy chọn mà là một phần của lời giải.

Chương tiếp theo trả lời câu hỏi ngược lại: nếu bạn cần thứ tự thì phải trả thêm bao nhiêu, và các cấu trúc cây tìm kiếm giữ cân bằng bằng cách nào (B/09). Bạn sẽ thấy một mẫu quen thuộc lặp lại: các cấu trúc thắng cuộc trong thực tế thường là các cấu trúc ngẫu nhiên hoá, vì lý do giống hết lý do trong chương này.

**Đọc thêm: công trình gốc** (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

| Mức         | Công trình và tác giả                                               | Nơi công bố, năm                           | Vì sao đọc, đọc phần nào                                                                                                                  |
|-------------|---------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| nền tảng    | Pagh & Rodler, “Cuckoo Hashing”                                     | Journal of Algorithms, 2004                | Bản viết rất gọn của một ý tưởng đẹp. Đọc phần mô tả thuật toán chèn và điều kiện hệ số tải; phần chứng minh đọc sau.                     |
| nền tảng    | Pătraşcu & Thorup, “The Power of Simple Tabulation Hashing”         | Journal of the ACM, 2012                   | Đọc phần mở đầu và phát biểu các định lý — đủ để hiểu vì sao một hàm băm chỉ đọc lập bậc ba lại dùng được ở nơi lý thuyết cũ đòi bậc cao. |
| tra khi cần | Bloom, “Space/Time Trade-offs in Hash Coding with Allowable Errors” | Communications of the ACM, 1970            | Bốn trang, bài gốc của Bloom filter; đọc để thấy động cơ ban đầu là tiết kiệm bộ nhớ cho từ điển chính tả.                                |
| tra khi cần | Flajolet, Fusy, Gandouet, Meunier, “HyperLogLog”                    | Conference on Analysis of Algorithms, 2007 | Đọc khi cần đếm phần tử phân biệt ở quy mô lớn; phần phân tích là ví dụ mẫu mực về xác suất trong thiết kế cấu trúc.                      |

# Nguồn

---

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [4] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [5] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [6] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [7] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [8] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [9] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [10] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [11] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] AtCoder. “AC Library: Lazy Segtree.” (), address: [https://atcoder.github.io/ac-library/master/document\\_en/lazysegtree.html](https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html) (visited on 09/07/2026).
- [13] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [14] R. E. Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets,” *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [15] M. L. Fredman and M. E. Saks, “The Cell Probe Complexity of Dynamic Data Structures,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [16] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [17] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [18] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-line String Searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

## Cây tìm kiếm và cân bằng (Search Trees and Balance)

**PHỤ THUỘC** B/08 cho phía đối lập — băm không thứ tự; A/04 cho bất biến, vì cân bằng chính là một bất biến được duy trì sau mỗi thao tác.

### TÓM TẮT

Cây tìm kiếm là câu trả lời cho câu hỏi mà băm không trả lời được: *giữ dữ liệu có thứ tự với chi phí cập nhật rẻ*. Ý tưởng gốc rất đơn giản — mỗi nút chia dữ liệu thành nhỏ hơn và lớn hơn — nhưng nó thoái hoá thành danh sách nếu dữ liệu vào theo thứ tự sắp sẵn, và toàn bộ chương này là các cách khác nhau để chống lại sự thoái hoá đó: quay để giữ chiều cao (AVL, đỏ-đen), làm nút to ra cho hợp với đĩa (B-cây), hoặc dùng ngẫu nhiên để cân bằng gần như miễn phí (treap, danh sách bỏ qua). Chương cũng nói về *tăng cường* — gắn thêm thông tin vào mỗi nút để cây trả lời được câu hỏi mới, kỹ thuật đáng giá nhất ở đây. Nếu chỉ nhớ một điều: chiều cao cây là bất biến cần duy trì, và mọi cấu trúc chỉ khác nhau ở cách trả tiền cho bất biến đó.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **cây tìm kiếm nhị phân (BST)** — cây nhị phân trong đó mọi nút ở cây con trái nhỏ hơn nút gốc, mọi nút ở cây con phải lớn hơn. Duyệt giữa cho dãy tăng dần.
- **chiều cao cây** — số cạnh trên đường dài nhất từ gốc tới lá; quyết định chi phí mọi thao tác. Cây cân bằng có chiều cao  $O(\log n)$ , cây thoái hoá có chiều cao  $O(n)$ .
- **phép quay (rotation)** — thao tác cục bộ đổi chỗ một nút với con của nó, giữ nguyên thứ tự nhưng đổi chiều cao. Đây là công cụ cơ bản của mọi cây tự cân bằng.
- **tăng cường (augmentation)** — lưu thêm thông tin ở mỗi nút — kích thước cây con, tổng, cực đại — để trả lời được truy vấn mới mà vẫn giữ  $O(\log n)$ .
- **B-cây** — cây tìm kiếm nhiều nhánh, mỗi nút chứa nhiều khoá, thiết kế để mỗi nút vừa một khối đĩa. Cơ sở của mọi chỉ mục cơ sở dữ liệu.
- **treap** — cây kết hợp tính chất BST theo khoá và tính chất heap theo một độ ưu tiên *ngẫu nhiên*; cân bằng theo nghĩa kỳ vọng mà không cần luật quay phức tạp.

### CÂU HỎI MỞ ĐẦU

- Q1. Cây tìm kiếm nhị phân đơn giản chạy  $O(\log n)$  “trung bình”. Vì sao trong thực tế nó lại nguy hiểm, và tình huống nào làm nó tệ nhất?
- Q2. Cây đỏ-đen và AVL cùng cho  $O(\log n)$ . Vì sao thư viện chuẩn thường chọn đỏ-đen còn cơ sở dữ liệu lại chọn B-cây?
- Q3. Treap dùng số ngẫu nhiên để cân bằng và không có luật quay phức tạp nào. Vì sao nó vẫn đảm bảo được chiều cao  $O(\log n)$ ?
- Q4. Vì sao “tăng cường” là kỹ thuật đáng giá nhất của chương này, hơn cả việc thuộc luật cân bằng?
- Q5. Khi nào một cây cân bằng  $O(\log n)$  lại thua một mảng đã sắp  $O(n)$  trên máy thật?

Chương trước cho ta tra cứu  $O(1)$  nhưng không có thứ tự. Rất nhiều bài toán cần thứ tự: tìm phần tử gần nhất, liệt kê mọi phần tử trong một khoảng, tìm phần tử nhỏ nhất lớn hơn  $x$ , hỏi phần tử đứng thứ  $k$ . Nếu dữ liệu tĩnh thì mảng đã sắp cộng tìm kiếm nhị phân giải hết — rẻ và

nhANH. Bài toán chỉ trở nên khó khi dữ liệu *vừa cần thứ tự vừa thay đổi liên tục*, vì chèn vào mảng đã sắp tốn  $O(n)$ .

Cây tìm kiếm là câu trả lời cho đúng tình huống đó, và ý tưởng thì hiển nhiên: thay vì giữ dữ liệu thành một dãy, hãy giữ nó thành một cấu trúc phân nhánh, để việc chèn chỉ động tới một đường từ gốc xuống lá. Cái không hiển nhiên — và là toàn bộ nội dung của chương — là làm sao giữ cho đường đó ngắn.

#### 4.1 Bài toán thoái hoá và ý nghĩa của cân bằng

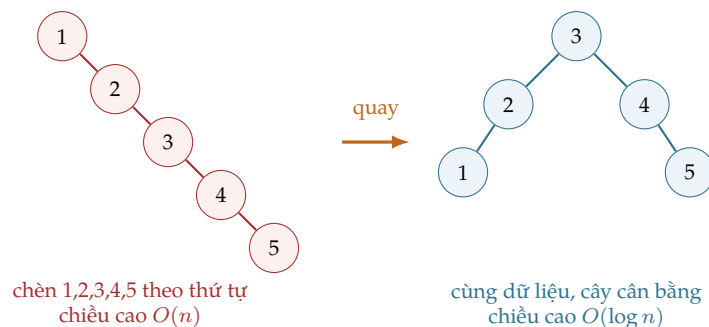
Cây tìm kiếm nhị phân thô sơ chèn phần tử mới bằng cách đi từ gốc, rẽ trái nếu nhỏ hơn, rẽ phải nếu lớn hơn, cho tới khi gặp chỗ trống. Nếu dữ liệu tới theo thứ tự ngẫu nhiên, chiều cao kỳ vọng là  $O(\log n)$  và mọi thứ đều tốt.

Nhưng nếu dữ liệu tới theo thứ tự *đã sắp* — tình huống cực kỳ phổ biến trong thực tế, vì dữ liệu thật thường có trật tự sẵn — thì mọi phần tử đều rẽ phải và cây thành một danh sách liên kết dài  $n$ . Mọi thao tác thành  $O(n)$ , và cấu trúc mà bạn chọn để tăng tốc lại làm chậm chương trình so với mảng thường.

Đây là cùng một câu chuyện với chương B/08: một cấu trúc nhanh “trung bình” nhưng có trường hợp xấu mà dữ liệu thực tế lại rất dễ rơi vào. Và có hai hướng chữa, hết như ở đó.

**Hướng tất định:** thêm một bất biến về hình dạng và duy trì nó sau mỗi thao tác. AVL yêu cầu chiều cao hai cây con của mọi nút chênh nhau không quá 1 — rất chặt, nên cây thấp và tìm kiếm nhanh, nhưng phải quay nhiều khi chèn/xoá. Cây đỏ-đen yêu cầu lỏng hơn — không đường nào từ gốc tới lá dài quá hai lần đường ngắn nhất — nên cây cao hơn một chút nhưng số phép quay ít hơn. Sự khác nhau đó giải thích lựa chọn của các thư viện: khi tỉ lệ chèn/xoá cao, đỏ-đen tổng thể rẻ hơn.

**Hướng ngẫu nhiên:** không ép hình dạng, mà làm cho hình dạng *không phụ thuộc thứ tự dữ liệu vào*. Đây là ý tưởng của treap và danh sách bỏ qua, và mục 3 sẽ nói kỹ vì nó rất đẹp.



**Hình 4.1.** Cùng năm khoá, hai hình dạng. Cây trái là kết quả của việc chèn dữ liệu đã sắp sẵn — trường hợp rất hay gặp trong thực tế, không phải trường hợp hiếm. Mọi cấu trúc trong chương này chỉ làm một việc: ngăn hình dạng bên trái xuất hiện, bằng phép quay hoặc bằng ngẫu nhiên hoá.

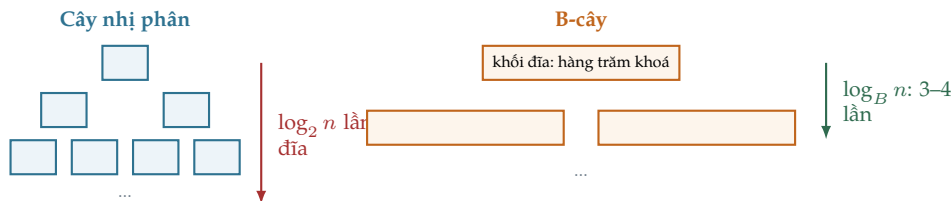
#### 4.2 Phép quay: thao tác cơ bản duy nhất

Mọi cây tự cân bằng đều dựa trên một thao tác duy nhất là phép quay: đổi chỗ một nút với con của nó, nối lại ba cây con cho đúng. Điều quan trọng nhất về phép quay — và là chỗ đáng dừng lại một phút — là nó *bảo toàn thứ tự duyệt giữa* nhưng *thay đổi chiều cao*. Nói theo ngôn ngữ chương A/04: phép quay giữ nguyên bất biến ngữ nghĩa (tính chất BST) trong khi sửa bất biến cấu trúc

(chiều cao). Đó chính là lý do nó là công cụ đúng: bạn cần sửa hình dạng mà không được làm hỏng nội dung.

Từ đó, các cấu trúc chỉ khác nhau ở *khi nào quay* và *quay bao nhiêu*: AVL quay khi chênh lệch chiều cao vượt 1; đỏ-đen quay và đổi màu theo một bộ luật dài hơn nhưng ít quay hơn; treap quay để khôi phục tính chất heap theo độ ưu tiên ngẫu nhiên; splay quay đưa nút vừa truy cập lên gốc.

Cây splay đáng nhắc riêng vì nó đại diện cho một triết lý khác[8]: không đảm bảo gì cho từng thao tác, chỉ đảm bảo  $O(\log n)$  *khấu hao*, đổi lại có tính chất rất hay là các phần tử hay truy cập tự động nổi lên gần gốc. Trong hệ thống có tính cục bộ truy cập cao, điều đó làm nó nhanh hơn cây cân bằng chặt. Nhược điểm: mỗi lần *đọc* cũng làm thay đổi cấu trúc, nên nó khó dùng trong môi trường nhiều luồng.



**Hình 4.2.** Đổi đơn vị chi phí thì đổi hình dạng cấu trúc. Khi chi phí thật là *lần đọc khối* chứ không phải phép so sánh, việc đăng nào cũng phải trả tiền cho một lần đọc khiến ta muốn đọc thật nhiều khoá một lúc — và cây từ nhị phân biến thành nhiều nhánh với nút vừa một khối đĩa.

### 4.3 Ngẫu nhiên hoá: cách rẻ nhất để cân bằng

Treap là một trong những ý tưởng đẹp nhất trong cấu trúc dữ liệu, và nó chỉ cần một câu để phát biểu: mỗi nút có khoá (do dữ liệu quyết định) và một *độ ưu tiên ngẫu nhiên* (do bạn sinh ra); cây được giữ sao cho theo khoá thì nó là BST, còn theo độ ưu tiên thì nó là heap.

Vì sao điều đó cho cân bằng? Vì hình dạng của một treap chỉ phụ thuộc vào *thứ tự các độ ưu tiên*, không phụ thuộc thứ tự chèn. Mà độ ưu tiên do bạn sinh ngẫu nhiên, nên hình dạng của treap giống hệt hình dạng của một BST được chèn theo thứ tự ngẫu nhiên — và chiều cao kỳ vọng của cây đó là  $O(\log n)$ . Nói cách khác: bạn không thể kiểm soát thứ tự dữ liệu tới, nhưng bạn có thể tự tạo ra một thứ tự ngẫu nhiên khác và để hình dạng phụ thuộc vào thứ tự đó.

Đây chính xác là ý tưởng của băm phổ dụng ở chương B/08 và của chốt ngẫu nhiên trong quick-sort ở chương C/14, xuất hiện lần thứ ba trong quyển sách này. Nếu bạn thấy mẫu ấy lặp lại thì đó không phải trùng hợp mà là một nguyên lý: *khi trường hợp xấu phụ thuộc vào dữ liệu, hãy chuyển sự phụ thuộc đó sang một nguồn ngẫu nhiên mà bạn kiểm soát*. Chương C/20 dành trọn để khai thác nguyên lý này.

Danh sách bỏ qua (skip list) là một hiện thực hoá khác của cùng ý tưởng, dùng nhiều tầng danh sách liên kết với xác suất lên tầng là  $1/2$ . Nó không phải cây, nhưng cho cùng đảm bảo kỳ vọng, dễ cài hơn cây cân bằng và dễ làm phiên bản nhiều luồng — đó là lý do một số hệ cơ sở dữ liệu chọn nó cho phần dữ liệu nằm trong bộ nhớ.

### 4.4 Tăng cường: kỹ thuật đáng giá nhất của chương

Đây là mục mà nếu chỉ đọc một mục thì nên đọc mục này. Ý tưởng: gắn thêm thông tin vào mỗi nút, miễn là thông tin đó *tính được từ hai con*, thì bạn giữ được nó qua mọi thao tác với chi phí không đổi, và cây trả lời được câu hỏi hoàn toàn mới.

Ví dụ quan trọng nhất: lưu ở mỗi nút *kích thước cây con* của nó. Với thông tin đó, cây trả lời được hai câu hỏi mà băm và heap đều chịu: “phần tử đứng thứ  $k$  là ai” và “có bao nhiêu phần tử nhỏ hơn  $x$ ” — cả hai trong  $O(\log n)$ , bằng cách đi xuống từ gốc và trừ dần. Cấu trúc này gọi là cây thống kê thứ tự, và nó là công cụ chuẩn cho các bài đếm nghịch thế hoặc đếm phần tử nhỏ hơn trong đoạn.

Ví dụ thứ hai: lưu *giá trị lớn nhất của điểm kết thúc* trong cây con, ta được cây khoảng — trả lời “có đoạn nào chứa điểm  $x$  không” hoặc “liệt kê mọi đoạn giao với  $[l, r]$ ”. Đây là cấu trúc dùng trong hình học tính toán (D/22) và trong các hệ thống lập lịch.

Quy tắc chung đáng nhớ: *thông tin tăng cường phải tổng hợp được từ các con trong  $O(1)$* . Kích thước, tổng, cực đại, cực tiểu đều thỏa; “phần tử phổ biến nhất trong cây con” thì không, vì không suy ra được từ hai con. Cùng điều kiện này sẽ xuất hiện lại ở chương B/10 với cây phân đoạn, và ở đó ta sẽ thấy nó chính là điều kiện “phép toán có tính kết hợp”.

#### 4.5 B-cây: khi chi phí thật không phải là số phép so sánh

Mọi phân tích trên đếm số phép so sánh. Nhưng nếu dữ liệu nằm trên đĩa, chi phí thật là số lần đọc khối — và một lần đọc đĩa đắt hơn hàng triệu phép so sánh trong bộ nhớ. Với mô hình chi phí đó, cây nhị phân là lựa chọn tệ: chiều cao  $\log_2 n$  nghĩa là  $\log_2 n$  lần đọc đĩa, mà với một tỉ bản ghi thì đó là 30 lần đọc cho một truy vấn.

B-cây giải quyết bằng cách làm nút to ra: mỗi nút chứa hàng trăm khoá và vừa đúng một khối đĩa. Chiều cao trở thành  $\log_{100} n$ , tức là ba tới bốn tầng cho một tỉ bản ghi. Trong nút thì tìm kiếm bằng so sánh — rẻ, vì nút đã nằm trong bộ nhớ sau khi đọc.

Đây là ví dụ rõ nhất trong cả quyển sách cho luận điểm ở chương A/03: *đổi mô hình chi phí thì đổi luôn cấu trúc tối ưu*. Cùng bài toán, cùng yêu cầu, nhưng khi đơn vị chi phí chuyển từ “phép so sánh” sang “lần chuyển khối” thì lời giải tốt nhất đổi hẳn hình dạng. Và điều đó vẫn đúng khi dữ liệu nằm trong RAM: vì bộ nhớ cũng có phân cấp cache, các cấu trúc “B-cây thu nhỏ” với nút vừa một dòng cache thường nhanh hơn cây nhị phân thuần túy.

| Cấu trúc                    | Đảm bảo                    | Cài đặt    | Chọn khi                                |
|-----------------------------|----------------------------|------------|-----------------------------------------|
| AVL                         | $O(\log n)$ chặt, cây thấp | Khó        | Đọc nhiều hơn ghi rất nhiều             |
| Đỏ-đen                      | $O(\log n)$ , ít quay hơn  | Khó        | Thư viện chuẩn, ghi và đọc cân bằng     |
| Treap                       | $O(\log n)$ kỳ vọng        | Dễ         | Tự cài trong thi đấu; cần tách/ghép cây |
| Danh sách bỏ qua            | $O(\log n)$ kỳ vọng        | Dễ         | Cần phiên bản nhiều luồng               |
| Splay                       | $O(\log n)$ khẩu hao       | Trung bình | Truy cập có tính cục bộ cao             |
| B-cây / B <sup>+</sup> -cây | $O(\log_B n)$ lần đọc khối | Khó        | Dữ liệu trên đĩa; chỉ mục cơ sở dữ liệu |

##### CẠM BẮY

Trong lập trình thi đấu, đừng cài AVL hay đỏ-đen. Nếu cần cây cân bằng tự viết — thường là để có thao tác tách và ghép, thứ mà thư viện chuẩn không cho — hãy dùng treap: ngắn hơn nhiều lần, ít lỗi hơn hẳn, và đảm bảo kỳ vọng là đủ. Còn nếu chỉ cần tập hợp có thứ tự thông thường thì dùng thẳng thư viện chuẩn; viết lại chỉ tăng nguy cơ sai.

## 4.6 Splay và câu hỏi mở nổi tiếng nhất của ngành cấu trúc dữ liệu

Bảng ở mục trước xếp splay tree vào ô “ $O(\log n)$  khấu hao, hợp khi truy cập có tính cục bộ”. Dòng đó đúng nhưng giấu mất phần thú vị nhất, và phần ấy đáng kể lại vì nó dạy một cách nghĩ chứ không chỉ một cấu trúc.

Splay tree không lưu *bất kỳ* thông tin cân bằng nào — không chiều cao, không màu, không độ ưu tiên. Luật duy nhất: mỗi lần chạm vào một nút, hãy quay nó lên tận gốc theo một khuôn cố định gồm các bước hai tầng. Sleator và Tarjan[8] chứng minh rằng chỉ luật đó đã đủ cho  $O(\log n)$  khấu hao — nhưng kết quả đáng nhớ hơn là hai tính chất mạnh hơn cận ấy.

*Tối ưu tĩnh*: trên bất kỳ dãy truy cập nào, tổng chi phí của splay chỉ hơn một hằng số lần so với *cây tĩnh tốt nhất* dựng bởi một người biết trước tần suất truy cập của mọi khoá. Nói cách khác, splay đạt được cận entropy của phân phối mà *không hề biết* phân phối đó — nó học từ chính dãy thao tác. *Tính chất tập làm việc*: chi phí chạm vào một khoá phụ thuộc số khoá phân biệt đã được chạm kể từ lần trước chạm nó, chứ không phụ thuộc  $n$ . Vì vậy một chương trình liên tục quay lại vài trăm khoá quen sẽ trả giá theo vài trăm chứ không theo hàng triệu — đúng hành vi mà bộ nhớ đệm mong muốn.

Câu hỏi mở nằm ở bước tiếp theo. Sleator và Tarjan phỏng đoán rằng splay không chỉ tối ưu so với cây tĩnh tốt nhất, mà tối ưu trong vòng một hằng số so với *mọi* thuật toán trên cây tìm kiếm nhị phân — kể cả thuật toán được phép quay tùy ý và được biết trước toàn bộ dãy truy cập. Đó là *phỏng đoán tối ưu động*, và sau bốn thập kỷ nó vẫn mở. Tiến bộ lớn đầu tiên là Tango tree của Demaine, Harmon, Iacono và Patrăşcu[9]: một cây trực tuyến có tỉ lệ cạnh tranh  $O(\log \log n)$ , dựng trên một cận dưới cho *mọi* cây nhị phân. Trước đó, tỉ lệ tốt nhất người ta biết chỉ là  $O(\log n)$  — tức là “không tệ hơn việc chẳng thích nghi gì cả”.

Có hai điều đáng mang theo, và cả hai đều lớn hơn bản thân splay tree. Thứ nhất là cách đặt câu hỏi: thay vì hỏi “cấu trúc này tốn bao nhiêu”, ta hỏi “nó tốn bao nhiêu so với lời giải tốt nhất có thể trên cùng dãy thao tác”. Đó chính là phân tích cạnh tranh, thước đo chuẩn của các bài toán trực tuyến ở chương D/24, và ở đây nó được áp cho một cấu trúc dữ liệu. Thứ hai là lời nhắc rằng chữ “tối ưu” luôn kèm một lớp đối thủ và một mô hình: tối ưu so với cây tĩnh, so với mọi cây nhị phân, hay so với mọi cấu trúc bất kỳ — ba câu hỏi khác nhau với ba câu trả lời khác nhau.

Còn phần thực dụng thì ngắn. Đừng cài splay trong kỳ thi: hằng số lớn, một thao tác đơn lẻ có thể tốn  $O(n)$ , và mọi lần *đọc* đều *ghi* vào cây — điều làm nó khó dùng trong môi trường nhiều luồng và bất lợi với bộ nhớ đệm chỉ đọc. Giá trị của nó với người học nằm ở chỗ khác: nó là bằng chứng rằng một cấu trúc không giữ chút siêu dữ liệu cân bằng nào vẫn có thể tự điều chỉnh theo dữ liệu, và nó là cửa vào cho cả một nhánh nghiên cứu vẫn còn sống.

## 4.7 Gốc rễ: từ bài toán chiều cao tới bài toán đĩa quay

Cây tìm kiếm nhị phân là ý tưởng cổ và không có tác giả rõ ràng — nó xuất hiện tự nhiên ở nhiều nơi trong thập niên 1950. Điều *có* tác giả rõ ràng là bài toán cân bằng, và cây tự cân bằng đầu tiên ra đời năm 1962 do hai nhà toán học Liên Xô, Adelson-Velsky và Landis, đề xuất — tên AVL là chữ đầu của họ. Đóng góp thực sự của họ không phải cấu trúc mà là *ý niệm*: rằng có thể duy trì một bất biến hình dạng sau mỗi thao tác với chi phí không đổi, và nhờ đó biến một đảm bảo “trung bình” thành đảm bảo tuyệt đối.

B-cây ra đời năm 1970 tại phòng nghiên cứu của Boeing, từ một bài toán hoàn toàn thực tế: chỉ mục cho các tệp lớn nằm trên đĩa quay. Rudolf Bayer và Edward McCreight nhận ra điều mà mô hình đếm phép so sánh che mất — rằng chi phí đọc một khối đĩa gần như không phụ thuộc kích thước khối, nên nếu đăng nào cũng phải trả tiền cho một lần đọc thì hãy đọc thật nhiều khoá một

lúc. Từ nhận xét đó ra cả một họ cấu trúc, và ngày nay gần như mọi hệ cơ sở dữ liệu quan hệ và mọi hệ tệp hiện đại đều dùng một biến thể của nó. Đây là ví dụ đẹp cho việc *đặt lại câu hỏi về đơn vị chi phí* có thể sinh ra cấu trúc mới.

Cây đỏ–đen có nguồn gốc thú vị: nó khởi đầu như một cách biểu diễn B-cây bậc 4 bằng cây nhị phân, rồi được Guibas và Sedgewick trình bày lại vào năm 1978 dưới dạng luật màu mà ta biết ngày nay. Nghĩa là hai cấu trúc trông rất khác nhau thực ra là hai cách nhìn cùng một thứ — một liên hệ đáng nhớ vì nó giải thích vì sao luật đỏ–đen trông tuy tiện: chúng là dấu vết của cấu trúc nhiều nhánh gốc.

Cuối cùng, hai cấu trúc ngẫu nhiên hoá ra đời muộn hơn nhiều và vì một lý do rất người: cây cân bằng tất định quá khó cài đúng. Pugh đề xuất danh sách bỏ qua năm 1990 với luận điểm thẳng thắn rằng nó đơn giản hơn hẳn cây cân bằng mà vẫn cho cùng đảm bảo kỳ vọng; Seidel và Aragon đưa ra treap năm 1996 với cùng tinh thần. Sự chuyển dịch này — chấp nhận đảm bảo kỳ vọng để đổi lấy code ngắn và đúng — là một chủ đề lặp lại trong ngành, và nó gắn với chương C/20.

#### **I** Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                                 | Bài toán thúc ép lúc đó                                                         | Điều thay đổi sau đó                                                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Thập niên 1950 — BST                | Cần cấu trúc vừa tra cứu nhanh vừa chèn được, thay cho mảng đã sắp.             | Ý tưởng phân nhánh theo khoá; nhưng chưa ai lo chuyện thoái hoá.                                                               |
| 1962 — Adelson-Velsky & Landis      | BST thoái hoá thành danh sách khi dữ liệu vào có trật tự.                       | Cây AVL — cây tự cân bằng đầu tiên; xác lập ý niệm “duy trì bất biến hình dạng sau mỗi thao tác”.                              |
| 1970 — Bayer & McCreight (Boeing)   | Chỉ mục tệp lớn trên đĩa quay; mỗi lần đọc đĩa đắt hơn hàng triệu phép so sánh. | B-cây: đổi đơn vị chi phí từ phép so sánh sang lần đọc khối $\Rightarrow$ nút to, cây thấp. Nền của mọi chỉ mục cơ sở dữ liệu. |
| 1972–78 — Bayer, Guibas & Sedgewick | Muốn ưu điểm của B-cây bậc 4 nhưng ở dạng cây nhị phân.                         | Cây đỏ–đen; luật màu trông tuy tiện chính là dấu vết của cấu trúc nhiều nhánh gốc.                                             |
| 1985 — Sleator & Tarjan             | Có cần đảm bảo cho <i>từng</i> thao tác không, hay tổng thể là đủ?              | Splay tree: $O(\log n)$ khẩu hao, tự thích nghi với mẫu truy cập; kèm theo là cả lý thuyết phân tích khẩu hao.                 |
| 1990–96 — Pugh; Seidel & Aragon     | Cây cân bằng tất định quá khó cài đúng.                                         | Danh sách bỏ qua và treap: dùng ngẫu nhiên để đạt cùng đảm bảo kỳ vọng với code ngắn hơn nhiều lần.                            |

## **I** 4.8 Liên hệ ngang

### **I** Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực      | Ý tưởng tương ứng                                    | Giống ở đâu, khác ở đâu                                                                                                                                            |
|---------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cơ sở dữ liệu | Chỉ mục B <sup>+</sup> -cây, quét khoảng, khoá chính | Cùng cấu trúc, và lý do chọn nó chính là mục 6. Điểm đáng nhớ: chỉ mục bám nhanh hơn cho truy vấn bằng nhưng vô dụng cho BETWEEN — đó là vì sao B-cây là mặc định. |
| Hệ tệp        | Chỉ mục thư mục, ánh xạ khối                         | Cùng ràng buộc đĩa nên cùng lời giải. Ví dụ về việc một mô hình chi phí (chuyển khối) sinh ra một họ cấu trúc riêng — xem thêm D/24.                               |

| Lĩnh vực                   | Ý tưởng tương ứng                                       | Giống ở đâu, khác ở đâu (tiếp)                                                                                                                                                                  |
|----------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hình học tính toán         | Cây khoảng, cây phân đoạn, cây $k$ -d                   | Đều là cây tìm kiếm được <i>tăng cường</i> : thêm thông tin ở mỗi nút để trả lời câu hỏi hình học. Cùng điều kiện: thông tin phải tổng hợp được từ các con (D/22).                              |
| SLAM và xử lý đám mây điểm | Cây $k$ -d cho tìm điểm gần nhất trong ICP              | Cùng ý tưởng chia không gian theo ngưỡng, chỉ khác là chia luân phiên theo từng chiều toạ độ. Cùng điểm yếu: dữ liệu phân bố lệch làm cây mất cân bằng, nên trong thực tế phải xây lại định kỳ. |
| Đồ hoạ, dò tia             | Cây bao khối (BVH), cây phân vùng không gian            | Cùng mục tiêu: giảm số phép kiểm tra từ tuyến tính xuống lôgarit bằng cấu trúc phân cấp. Tiêu chí xây cây khác nhau (diện tích bề mặt thay vì cân bằng số nút).                                 |
| Lập trình hàm              | Cấu trúc bền vững, cây bất biến                         | Cây rất hợp với biểu diễn bền vững vì một thao tác chỉ dùng $O(\log n)$ nút, phần còn lại chia sẻ được giữa các phiên bản [10]. Đây là nền của cây bền vững ở B/10.                             |
| Hệ thống nhiều luồng       | Danh sách bỏ qua trong bộ nhớ đệm ghi của cơ sở dữ liệu | Lý do chọn không phải tốc độ mà là <i>để làm phiên bản đồng thời</i> : các thao tác chỉ động vào con trỏ cục bộ, không cần quay cả cây.                                                         |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì trường hợp xấu nhất của nó không hiếm mà lại rất phổ biến: dữ liệu thật thường tới theo trật tự nào đó — theo thời gian, theo mã tăng dần, theo thứ tự đã sắp từ nguồn khác. Khi đó mọi phần tử rẽ cùng một hướng và cây thành danh sách với chiều cao  $n$ . Đây là cùng dạng vấn đề với bảng băm ở B/08: “trung bình nhanh” vô nghĩa nếu phân phối thực tế nằm đúng vào vùng xấu.
- A2. Vì hai bối cảnh có đơn vị chi phí khác nhau. Trong bộ nhớ, chi phí là số phép so sánh và số lần lần theo con trỏ, nên cây nhị phân là hợp lý; giữa AVL và đỏ-đen thì đỏ-đen quay ít hơn nên tốt hơn khi có nhiều chèn/xoá, và đó là lựa chọn của thư viện chuẩn. Trên đĩa, chi phí là số lần đọc khối và một lần đọc đắt hơn hàng triệu phép so sánh; lúc đó phải làm nút thật to để cây thật thấp — B-cây. Cùng bài toán, đổi mô hình chi phí, đổi cấu trúc tối ưu.
- A3. Vì hình dạng của treap chỉ phụ thuộc vào thứ tự tương đối của các độ ưu tiên, chứ không phụ thuộc thứ tự chèn. Mà độ ưu tiên do bạn sinh ngẫu nhiên, nên treap có phân phối hình dạng giống hệt một BST được chèn theo thứ tự ngẫu nhiên đều — và chiều cao kỳ vọng của cây đó là  $O(\log n)$ . Điểm mấu chốt: đổi thủ chọn được dữ liệu nhưng không chọn được độ ưu tiên, nên không dựng được trường hợp xấu.
- A4. Vì luật cân bằng thì bạn hầu như luôn dùng lại từ thư viện, còn tăng cường là thứ bạn phải tự nghĩ cho từng bài. Nó biến một cấu trúc chỉ biết “có hay không” thành cấu trúc trả lời được “phần tử thứ  $k$ ”, “bao nhiêu phần tử nhỏ hơn  $x$ ”, “đoạn nào chứa điểm này”. Điều kiện áp dụng cũng rất dễ kiểm: thông tin thêm phải tổng hợp được từ hai con trong  $O(1)$  — và chính điều kiện đó sẽ trở lại ở B/10 dưới tên “phép toán có tính kết hợp”.
- A5. Khi dữ liệu tĩnh hoặc gần như tĩnh. Mảng đã sắp cho tìm kiếm nhị phân với truy cập gần như tuần tự, rất hợp cache, không có con trỏ và không có chi phí cấp phát; cây cân bằng lần theo con trỏ nên mỗi tầng có thể là một lần trượt cache. Với vài triệu phần tử, khác biệt hằng số này lớn hơn khác biệt giữa  $O(\log n)$  và  $O(n)$  chèn — miễn là số lần chèn nhỏ. Quy tắc thực dụng: ít cập nhật thì mảng đã sắp; cập nhật liên tục thì cây.

### NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng một cuốn danh bạ. Nếu tên được ghi lộn xộn, muốn tìm ai bạn phải đọc từ đầu. Nếu ghi theo thứ tự chữ cái, bạn mở giữa quyển rồi lật qua lật lại — đó là tìm kiếm nhị phân, rất nhanh, nhưng thêm một cái tên mới vào giữa thì phải chép lại cả quyển.

Cây tìm kiếm là cách sắp xếp danh bạ thành nhiều tờ rồi có mũi tên chỉ nhau, để thêm một tên chỉ phải sửa vài tờ. Vấn đề duy nhất: nếu bạn thêm tên theo đúng thứ tự chữ cái từ A tới Z, các mũi tên sẽ nối thành một hàng dài và bạn quay lại đúng tình cảnh đọc từ đầu.

Có hai cách chống. Cách thứ nhất là cứ vài lần thêm lại chỉnh cho cây khỏi lệch — giống việc sắp lại giá sách cho hai bên đều nhau. Cách thứ hai thông minh hơn và rất đáng nhớ: gán cho mỗi tờ một con số ngẫu nhiên và xếp cây theo con số đó thay vì theo thứ tự bạn thêm vào. Vì con số là do bạn tung ra, người khác không thể cố tình đưa dữ liệu theo thứ tự xấu để hại bạn.

Cuối cùng là một chuyện thực tế thú vị. Nếu quyn danh bạ để trên bàn, việc lật một trang gần như miễn phí. Nhưng nếu nó nằm trong kho và mỗi lần cần một trang thì phải chạy xuống kho lấy, thì bạn sẽ không muốn mỗi trang chỉ có một cái tên — bạn sẽ muốn mỗi lần chạy xuống kho mang lên một tập dày. Đó chính là lý do cơ sở dữ liệu không dùng cây nhị phân mà dùng cây có nút rất to.

**Một câu:** *Chiều cao là thứ phải giữ; cách trả tiền cho nó — quay tay hay tung ngẫu nhiên — là chỗ các cấu trúc khác nhau.*

**Chỗ không rút gọn được:** Việc chọn thông tin tăng cường cho một bài cụ thể vẫn là bước sáng tạo; điều kiện “tổng hợp được từ hai con” chỉ nói cái gì không dùng được.

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                | Trả lời                                                                                                                                                                                                           |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Vấn đề gốc của BST             | Dữ liệu vào theo thứ tự đã sắp làm cây thoái hoá thành danh sách, chiều cao $O(n)$ . Đây là trường hợp phổ biến, không hiếm.                                                                                      |
| ••• Phép quay làm gì               | Bảo toàn thứ tự duyệt giữa (bất biến ngữ nghĩa) trong khi thay đổi chiều cao (bất biến cấu trúc). Là thao tác cơ bản duy nhất của mọi cây tự cân bằng.                                                            |
| ••• Vì sao treap cân bằng          | Hình dạng chỉ phụ thuộc thứ tự các độ ưu tiên ngẫu nhiên, không phụ thuộc thứ tự chèn $\Rightarrow$ tương đương BST chèn ngẫu nhiên $\Rightarrow$ chiều cao kỳ vọng $O(\log n)$ .                                 |
| ••• Tăng cường — điều kiện         | Thông tin thêm phải tổng hợp được từ hai con trong $O(1)$ : kích thước, tổng, cực trị được; “phần tử phổ biến nhất” thì không.                                                                                    |
| ••• Cây thống kê thứ tự            | Lưu kích thước cây con $\Rightarrow$ trả lời “phần tử thứ $k$ ” và “bao nhiêu phần tử nhỏ hơn $x$ ” trong $O(\log n)$ .                                                                                           |
| ••• Vì sao B-cây tồn tại           | Chi phí thật trên đĩa là số lần đọc khối, không phải số phép so sánh. Nút to $\Rightarrow$ chiều cao $\log_B n \Rightarrow 3-4$ lần đọc cho một tỉ bản ghi.                                                       |
| •• AVL so với đỏ-đen               | AVL chặt hơn nên cây thấp hơn, tìm nhanh hơn, nhưng quay nhiều hơn khi cập nhật. Đỏ-đen lỏng hơn, ít quay hơn — lựa chọn của thư viện chuẩn.                                                                      |
| •• Splay tree                      | Không đảm bảo từng thao tác, chỉ $O(\log n)$ khấu hao; phần tử hay truy cập tự nổi lên gần gốc. Nhược điểm: thao tác đọc cũng làm đổi cấu trúc.                                                                   |
| •• Danh sách bỏ qua                | Nhiều tầng danh sách liên kết, xác suất lên tầng $1/2$ ; cùng đảm bảo kỳ vọng với cây, dễ cài và dễ làm phiên bản nhiều luồng.                                                                                    |
| •• Mẫu ngẫu nhiên hoá lặp lại      | Băm phổ dụng (B/08), treap (chương này), chốt quicksort (C/14) — cùng một nguyên lý: chuyển sự phụ thuộc từ dữ liệu sang nguồn ngẫu nhiên bạn kiểm soát.                                                          |
| •• Cây khoảng                      | Lưu cực đại của điểm kết thúc trong cây con $\Rightarrow$ hỏi “đoạn nào chứa $x$ ”, “đoạn nào giao $[l, r]$ ”.                                                                                                    |
| • Khi nào mảng đã sắp thẳng cây    | Dữ liệu tĩnh hoặc ít cập nhật: không con trỏ, hợp cache, không cấp phát. Cây chỉ thẳng khi có cập nhật liên tục.                                                                                                  |
| • Lời khuyên thi đấu               | Đừng cài AVL hay đỏ-đen; cần cây tự viết thì dùng treap (đặc biệt khi cần tách/ghép), còn lại dùng thư viện chuẩn.                                                                                                |
| •• Splay và phỏng đoán tối ưu động | Splay không lưu thông tin cân bằng nào, vẫn đạt tối ưu tĩnh và tính chất tập làm việc. Phỏng đoán “cạnh tranh hằng số với mọi cây nhị phân” vẫn mở; tiến bộ lớn nhất là Tango tree, $O(\log \log n)$ -cạnh tranh. |

● ● ● phải thuộc ● ● nên nắm ● biết là có

## I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

### 1. Khi nào tôi thực sự cần tự cài cây cân bằng thay vì dùng thư viện?

| Chủ yếu khi cần hai thao tác mà thư viện chuẩn không có: *tách* cây thành hai theo khoá và *ghép* hai cây, hoặc cần truy vấn “phần tử thứ  $k$ ”. Với nhu cầu thứ nhất, treap là lựa chọn gần như bắt buộc vì tách/ghép trên treap chỉ vài dòng. Với nhu cầu thứ hai, nếu khoá là số nguyên trong khoảng vừa phải thì Fenwick (B/10) đơn giản hơn nhiều và nhanh hơn — hãy kiểm tra khả năng đó trước khi viết cây.

### 2. Tôi nghe nói cây đỏ-đen “khó cài”. Khó ở chỗ nào?

| Ở phần xoá. Chèn có bốn trường hợp và còn dễ nhớ; xoá có nhiều trường hợp hơn, phải xử lý cả tình huống “nút thiếu màu đen” lan ngược lên gốc, và mỗi trường hợp lại chia theo màu của anh em và của các cháu. Sai một nhánh thì cây vẫn chạy đúng về mặt thứ tự nhưng mất cân bằng dần — một lỗi im lặng rất khó phát hiện. Đây là lý do thực tế khiến treap và danh sách bỏ qua được ưa chuộng ngoài các thư viện chuẩn.

### 3. Cây $k$ -d có phải là cây tìm kiếm không, và vì sao nó hay mất cân bằng?

| Nó là cây tìm kiếm nhị phân trên không gian nhiều chiều: mỗi tầng chia theo một chiều toạ độ khác nhau. Nó mất cân bằng vì chiều được chọn luân phiên chứ không theo phân bố dữ liệu; nếu điểm dồn về một mặt phẳng hoặc một trục thì nhiều lần chia gần như không tách được gì. Trong thực tế — ví dụ tìm điểm gần nhất trên đám mây điểm của cảm biến — cách xử lý phổ biến là chọn chiều có phương sai lớn nhất thay vì luân phiên, và xây lại cây định kỳ thay vì cập nhật từng điểm.

### 4. Cây cân bằng có dùng được trong môi trường nhiều luồng không?

| Được nhưng khó, vì phép quay động tới nhiều nút cùng lúc nên phải khoá một vùng lớn. Có ba hướng thực tế: dùng danh sách bỏ qua, vốn chỉ cần sửa vài con trỏ cục bộ và có bản không khoá; dùng cấu trúc bền vững kiểu lập trình hàm, ở đó người đọc luôn nhìn một ảnh chụp bất biến[10]; hoặc dùng B-cây với kỹ thuật khoá theo nút và tách trước khi đi xuống. Splay tree là lựa chọn tệ nhất ở đây vì ngay cả thao tác đọc cũng làm thay đổi cấu trúc.

### 5. Vì sao đếm nghịch thể lại là bài tiêu biểu cho chương này?

| Vì nó có ba lời giải bằng ba công cụ khác nhau của phần B, và so sánh chúng dạy nhiều hơn bản thân bài. Cách một: cây thống kê thứ tự, chèn dần từ trái sang phải và mỗi lần hỏi “bao nhiêu phần tử đã chèn lớn hơn giá trị này”. Cách hai: Fenwick trên giá trị đã nén (B/10) — ngắn hơn và nhanh hơn. Cách ba: sắp xếp trộn, đếm trong lúc trộn (C/14). Ba cách cùng  $O(n \log n)$  nhưng hằng số và độ dài code rất khác.

### 6. “Bền vững” nghĩa là gì và vì sao cây lại hợp với nó?

| Bền vững là khả năng giữ lại mọi phiên bản cũ sau khi cập nhật, để truy vấn được quá khứ. Cây rất hợp vì một thao tác chỉ đụng tới các nút trên một đường từ gốc xuống lá — chỉ  $O(\log n)$  nút. Thay vì sửa tại chỗ, ta tạo bản sao của đúng đường đó và cho các nhánh còn lại trở chung sang cây cũ; chi phí thêm là  $O(\log n)$  bộ nhớ mỗi phiên bản. Ý tưởng này sẽ quay lại với cây phân đoạn bền vững ở B/10.

### 7. Nếu khoá là số nguyên trong khoảng hẹp, có gì tốt hơn cây không?

| Có, và chênh lệch lớn. Với khoá trong  $[0, U)$ , cấu trúc van Emde Boas cho mọi thao tác trong  $O(\log \log U)$  — nhanh hơn hẳn  $O(\log n)$  khi  $U$  không quá lớn, đổi lại tốn bộ nhớ  $O(U)$  và cài đặt phức tạp. Trong thực hành thi đấu, thường chỉ cần một Fenwick trên mảng bit hoặc một cấu trúc phân tầng đơn giản là đủ nhanh. Bài học chung vẫn thế: biết thêm về dữ liệu — ở đây là “khoá là số nguyên nhỏ” — thì luôn mua được tốc độ.

## I Bài tập (mức: ● khởi động ● ● chuẩn ● ● ● khó)

| Bài                                                                 | Nguồn                                 | Điều cần nhận ra                                                                            |
|---------------------------------------------------------------------|---------------------------------------|---------------------------------------------------------------------------------------------|
| • Cài BST cơ bản, rồi chèn $1..n$ theo thứ tự và đo chiều cao       | Bài tập cài đặt                       | Thấy tận mắt sự thoái hoá; sau đó chèn theo thứ tự ngẫu nhiên và đo lại.                    |
| • Dùng tập hợp có thứ tự để tìm phần tử gần nhất với mỗi truy vấn   | CSES “Concert Tickets”[1]             | Bài mẫu cho việc dùng <code>lower_bound</code> — thứ mà bảng băm không làm được.            |
| •• Đếm số nghịch thế                                                | CSES “Inversions”-loại; bài kinh điển | Giải bằng cả ba cách (cây thống kê thứ tự, Fenwick, sắp xếp trộn) và so hằng số.            |
| •• Cài treap với chèn, xoá, tìm phần tử thứ $k$                     | Bài tập cài đặt                       | Bắt đầu bằng tách/ghép; mọi thao tác khác viết từ hai thao tác đó.                          |
| •• Duy trì trung vị của một tập thay đổi bằng cây thống kê thứ tự   | Bài tập kinh điển                     | So sánh với cách hai heap ở B/07 — cây tổng quát hơn, heap nhanh hơn cho đúng bài này.      |
| •• Lịch đặt phòng: kiểm tra chồng lấn khi thêm mỗi lịch             | LeetCode “My Calendar I/II”           | Cây khoảng hoặc tập hợp có thứ tự; chú ý xử lý biên đóng/mở cho đúng.                       |
| ••• Cài danh sách bỏ qua và so tốc độ với cây cân bằng của thư viện | Bài tập thực nghiệm                   | Đo cả thời gian lẫn bộ nhớ; giải thích chênh lệch bằng cache chứ không bằng độ phức tạp.    |
| ••• Dùng treap để chèn/xoá cả đoạn của một dãy                      | Bài tập thi đấu kinh điển             | Treap theo chỉ số ngẫu nhiên — ứng dụng mạnh nhất của treap và là lý do chính để tự cài nó. |
| ••• Cài B-cây bậc lớn và đo số lần truy cập nút so với cây nhị phân | Bài tập hệ thống                      | Mô phỏng chi phí đọc khối; đây là cách rẻ nhất để thấy vì sao cơ sở dữ liệu chọn B-cây.     |
| ••• Cây $k$ -d cho tìm điểm gần nhất trên đám mây điểm 3D           | Bài tập ứng dụng                      | So với vét cạn về thời gian; thử phân bố lệch để thấy cây mất cân bằng.                     |

## Kết chương và đọc tiếp

Cây tìm kiếm là câu chuyện về một bất biến: chiều cao phải là  $O(\log n)$ . Mọi cấu trúc trong chương chỉ khác nhau ở cách trả tiền cho bất biến đó — quay theo luật chặt, quay theo luật lỏng, quay theo độ ưu tiên ngẫu nhiên, hay làm nút to ra cho hợp với đơn vị chi phí của đĩa. Kỹ thuật đáng mang theo nhất không phải luật cân bằng mà là *tăng cường*, cùng điều kiện của nó: thông tin thêm phải tổng hợp được từ các con.

Chương tiếp theo lấy đúng ý tưởng tăng cường ấy và đẩy tới cùng cho một bài toán rất cụ thể: truy vấn trên đoạn kèm cập nhật. Bạn sẽ gặp lại điều kiện “tổng hợp được từ các con” dưới cái tên chính xác của nó — tính kết hợp — và sẽ thấy hai cấu trúc có tỉ lệ sức mạnh trên độ dài code cao nhất trong cả phần B là Fenwick và cây phân đoạn (B/10).

**Đọc thêm: công trình gốc** (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

| Mức      | Công trình và tác giả                                          | Nơi công bố, năm                | Vì sao đọc, đọc phần nào                                                                                                                  |
|----------|----------------------------------------------------------------|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| nền tảng | Sleator & Tarjan, “Self-Adjusting Binary Search Trees”         | Journal of the ACM, 1985        | Bài gốc của splay tree. Đọc phần phát biểu tối ưu tĩnh và tập làm việc; phần chứng minh là ví dụ mẫu mực cho phương pháp thế năng (A/03). |
| nền tảng | Demaine, Harmon, Iacono, Pătraşcu, “Dynamic Optimality—Almost” | SIAM Journal on Computing, 2007 | Tango tree và cận dưới đàn xen. Đọc mục giới thiệu để hiểu phát biểu của phỏng đoán tối ưu động, kể cả khi bỏ qua chi tiết kỹ thuật.      |

| Mức         | Công trình và tác giả                                          | Nơi công bố, năm                 | Vì sao đọc (tiếp)                                                                                                       |
|-------------|----------------------------------------------------------------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| tra khi cần | Knuth, “Optimum Binary Search Trees”                           | Acta Informatica, 1971           | Cây tìm tối ưu tính được bằng quy hoạch động $O(n^2)$ nhờ tính đơn điệu của điểm chia — cùng kết quả sẽ trở lại ở C/16. |
| tra khi cần | Demaine, khoá học MIT 6.851<br><i>Advanced Data Structures</i> | tài liệu khoá học công khai, MIT | Bài giảng về cây bền vững, tối ưu động và các cấu trúc gần biên; hợp để đi tiếp sau chương này.                         |

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [4] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [5] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [6] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [7] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [8] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [9] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [10] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [11] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] AtCoder. “AC Library: Lazy Segtree.” (), address: [https://atcoder.github.io/ac-library/master/document\\_en/lazysegtree.html](https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html) (visited on 09/07/2026).
- [13] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [14] R. E. Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets,” *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [15] M. L. Fredman and M. E. Saks, “The Cell Probe Complexity of Dynamic Data Structures,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [16] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [17] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [18] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-line String Searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

## Cây chỉ số và truy vấn đoạn (Fenwick, Segment Trees, Range Queries)

**PHỤ THUỘC** B/06 cho tổng tiền tố — chương này là câu trả lời khi mảng thay đổi; B/09 cho ý tưởng tăng cường; A/05 cho tính kết hợp.

### TÓM TẮT

Chương B/06 cho truy vấn đoạn  $O(1)$  với điều kiện mảng đứng yên. Chương này bỏ điều kiện đó, và cái giá phải trả là  $O(\log n)$  cho cả truy vấn lẫn cập nhật. Bốn cấu trúc: Fenwick — gọn cho tổng/XOR với cập nhật điểm; cây phân đoạn — tổng quát nhất, thêm lazy propagation thì cập nhật cả đoạn cũng  $O(\log n)$ ; sparse table —  $O(1)$  truy vấn nhưng mảng phải tĩnh; và phân rã căn bậc hai — chậm hơn nhưng dùng được cho những phép toán mà ba cấu trúc kia chịu. Các cấu trúc gộp đoạn cần trạng thái đủ thông tin và phép gộp kết hợp; Mo cần thao tác thêm/bớt ở biên đủ rẻ. Nếu chỉ nhớ một điều: chọn cấu trúc bằng cách trả lời hai câu — phép toán có kết hợp không, và mảng có thay đổi không.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **phép toán kết hợp (associative)** — phép  $\oplus$  thoả  $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ . Điều kiện để gộp kết quả của hai nửa thành kết quả của cả đoạn — nền của mọi cấu trúc trong chương.
- **phần tử trung hoà** — giá trị  $e$  với  $a \oplus e = e \oplus a = a$ ; cần để trả lời truy vấn trên đoạn rỗng mà không phải viết trường hợp riêng. Cùng với phép kết hợp, ta có một *vị nhóm* (monoid).
- **Fenwick (binary indexed tree)** — mảng phụ trong đó ô  $i$  giữ tổng của một đoạn có độ dài bằng bit thấp nhất của  $i$ ; cập nhật điểm và tổng tiền tố đều  $O(\log n)$ , code chỉ vài dòng.
- **cây phân đoạn (segment tree)** — cây nhị phân trong đó mỗi nút giữ kết quả gộp của một đoạn; lá là phần tử đơn, gốc là cả mảng.
- **lazy propagation** — kỹ thuật hoãn việc đẩy cập nhật xuống các nút con cho tới khi thật sự cần, để cập nhật cả đoạn cũng chỉ  $O(\log n)$ .
- **sparse table** — bảng tiền xử lý  $f[i][k] =$  kết quả gộp của đoạn dài  $2^k$  bắt đầu tại  $i$ ; truy vấn  $O(1)$  cho phép toán *luỹ đẳng* như cực trị, nhưng mảng phải tĩnh.

### CÂU HỎI MỞ ĐẦU

- Q1. Vì sao Fenwick không dùng được cho truy vấn cực đại, trong khi cây phân đoạn thì được?
- Q2. Sparse table trả lời cực đại trong  $O(1)$  còn cây phân đoạn cần  $O(\log n)$ . Vì sao sparse table không thay thế được cây phân đoạn?
- Q3. Lazy propagation hoãn việc cập nhật lại. Vì sao việc hoãn đó không làm sai kết quả, và lập luận nào bảo đảm điều đó?
- Q4. Cùng  $O(\log n)$  nhưng Fenwick thường nhanh hơn cây phân đoạn vài lần. Khác biệt nằm ở đâu?
- Q5. Có phép toán nào mà cả Fenwick lẫn cây phân đoạn đều chịu, nhưng phân rã căn bậc hai vẫn giải được không?

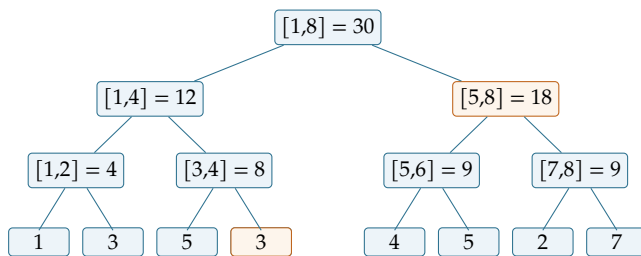
Cuối chương B/06 còn một câu hỏi bỏ ngỏ. Tổng tiền tố cho truy vấn đoạn trong  $O(1)$ , nhưng chỉ khi mảng đứng yên; mảng hiệu cho cập nhật đoạn trong  $O(1)$ , nhưng chỉ khi ta đọc kết quả một lần ở cuối. Bài toán thật thường trộn cả hai: cập nhật và truy vấn xen kẽ, mỗi loại hàng trăm nghìn lần.

Không có biểu diễn tĩnh nào làm được cả hai, và lý do khá sâu: tổng tiền tố nhanh vì mỗi ô của nó tóm tắt *rất nhiều* phần tử, nên sửa một phần tử làm hỏng rất nhiều ô. Đây là một đánh đổi thật sự, không phải thiếu sót của kỹ thuật. Lời giải của chương này là một sự thỏa hiệp có cấu trúc: thay vì mỗi ô tóm tắt một tiền tố dài, hãy để mỗi ô tóm tắt một đoạn có độ dài *lưu thừa của 2*, xếp thành cây. Khi đó mỗi phần tử chỉ nằm trong  $O(\log n)$  đoạn, và mỗi đoạn truy vấn ghép được từ  $O(\log n)$  mảnh.

## 5.1 Cây phân đoạn: ý tưởng tổng quát nhất

Chia mảng làm đôi, mỗi nửa lại chia đôi, cho tới khi còn một phần tử. Mỗi nút của cây giữ kết quả gộp của đoạn tương ứng. Với tổng, nút cha bằng tổng hai con; với cực đại, nút cha bằng cực đại hai con.

Truy vấn đoạn  $[l, r]$ : đi từ gốc xuống, nếu đoạn của nút nằm gọn trong  $[l, r]$  thì trả kết quả đã lưu, nếu không giao thì bỏ qua, nếu giao một phần thì đi tiếp cả hai con. Có thể chứng minh mỗi tầng đóng góp tối đa hai nút được “dùng nguyên”, nên chi phí là  $O(\log n)$ . Cập nhật một phần tử: đi từ lá lên gốc, cập nhật  $O(\log n)$  nút trên đường.



Truy vấn  $[4, 8]$  ghép từ hai nút đã tô:  $[4, 4]$  và  $[5, 8]$ , tổng  $3 + 18 = 21$ . Mỗi tầng gộp tối đa hai nút  $\Rightarrow O(\log n)$ .

**Hình 5.1.** Cây phân đoạn cho mảng tám phần tử. Điều đáng chú ý: truy vấn không đi tới lá mà dừng lại ngay khi gặp một nút nằm gọn trong đoạn hỏi. Đây cũng là chỗ giải thích vì sao lazy propagation hoạt động — nếu ta dừng ở một nút thì mọi thứ bên dưới nó không cần chính xác ngay lúc này.

Điều kiện để mọi thứ trên đúng là *tính kết hợp*: kết quả của cả đoạn phải ghép được từ kết quả hai nửa. Tổng, tích, cực đại, cực tiểu, GCD, XOR, số lượng, “đoạn con có tổng lớn nhất” đều thỏa. Ngược lại, “phần tử xuất hiện nhiều nhất trong đoạn” thì *không* — biết đáp án của hai nửa không đủ để suy ra đáp án của cả đoạn — và đó là lý do bài đó cần công cụ khác, ví dụ thuật toán Mo ở mục 5.

Nhận xét này là cùng một điều kiện với “thông tin tăng cường phải tổng hợp được từ hai con” ở chương B/09, nay được gọi bằng tên toán học của nó. Trong ngôn ngữ đại số, ta cần một *vị nhóm*: một phép kết hợp cộng một phần tử trung hoà. Phần tử trung hoà không phải chi tiết hình thức — nó chính là giá trị bạn trả về cho đoạn rỗng, và quên nó là nguồn lỗi cài đặt phổ biến (trả về 0 cho truy vấn cực đại là sai nếu mảng có số âm; phải trả về  $-\infty$ ).

### Lazy propagation: hoãn cho tới khi thật cần

Cập nhật một phần tử là  $O(\log n)$ , nhưng cập nhật cả một đoạn — “cộng 5 vào mọi phần tử trong  $[l, r]$ ” — nếu làm từng phần tử thì thành  $O(n \log n)$ . Lazy propagation giải quyết bằng một ý rất giống ý của mảng hiệu: *ghi nợ*.

Khi một cập nhật phủ trọn đoạn của một nút, ta cập nhật giá trị tổng hợp của nút đó và ghi một “khoản nợ” vào nút, rồi *dừng lại* — không đi xuống các con. Chỉ khi nào một truy vấn hay cập nhật sau này cần đi xuống dưới nút đó, ta mới đẩy khoản nợ xuống hai con.

Vì sao không sai? Vì bất biến được duy trì là: *giá trị lưu tại mỗi nút đã đúng, còn khoản nợ tại nút mô tả phần chưa được áp cho các con*. Mọi đường đi xuống đều đẩy nợ trước khi đọc con, nên không ai đọc dữ liệu cũ. Đây lại là một ví dụ của mẫu “trì hoãn công việc cho tới khi có người thật sự cần” — cùng ý với xoá lười ở chương B/07.

## I 5.2 Fenwick: ngắn hơn, nhanh hơn, hẹp hơn

Fenwick làm cùng việc với cây phân đoạn cho trường hợp tổng, nhưng bằng một mẹo chỉ số rất gọn: ô  $i$  giữ tổng của đoạn kết thúc tại  $i$  có độ dài bằng *bit thấp nhất* của  $i$ . Nhờ vậy tổng tiền tố tới  $i$  ghép từ các ô  $i$ ,  $i - \text{lowbit}(i)$ ,  $\dots$  — số ô bằng số bit 1 của  $i$ , tối đa  $\log n$ .

Ưu điểm là rất thực dụng: code khoảng năm dòng, bộ nhớ đúng  $n$  số, và thường có hằng số nhỏ. Các bước truy cập nhảy theo lowbit, không phải quét tuần tự; mức nhanh hơn phải đo trên cách cài và dữ liệu cụ thể.

Cái giá của khuôn Fenwick tổng/XOR: truy vấn đoạn dùng nghịch đảo để loại tiền tố trái, còn cập nhật điểm kiểu cộng dồn dùng tính giao hoán. Không nên suy rộng rằng mọi biến thể Fenwick đều cần nghịch đảo: cực tiểu tiền tố với cập nhật chỉ giảm vẫn làm được, nhưng không hỗ trợ gán tùy ý hay suy ra cực tiểu đoạn bằng hai tiền tố. Tổng và XOR có; cực đại thì không — biết cực đại của  $[1, r]$  và của  $[1, l - 1]$  không suy ra được cực đại của  $[l, r]$ . Đó là câu trả lời cho câu hỏi mở đầu thứ nhất, và cũng là ranh giới rõ ràng giữa hai cấu trúc.

Hai mở rộng đáng biết: Fenwick cũng làm được “cập nhật đoạn, truy vấn điểm” bằng cách chạy trên mảng hiệu; và làm được cả “cập nhật đoạn, truy vấn đoạn” bằng hai Fenwick với một chút đại số. Ngoài ra, có thể tìm “vị trí nhỏ nhất mà tổng tiền tố vượt  $x$ ” trong  $O(\log n)$  bằng cách đi xuống theo bit, với điều kiện các phần tử không âm để tổng tiền tố đơn điệu — kỹ thuật thay thế cho cây thống kê thứ tự ở B/09 và thường nhanh hơn nhiều.

## I 5.3 Sparse table: trả trước tất cả, với điều kiện mảng tĩnh

Nếu mảng không bao giờ thay đổi và phép toán *luỹ đẳng* — tức  $a \oplus a = a$ , đúng với cực đại, cực tiểu, GCD — thì ta có thể làm tốt hơn  $O(\log n)$ . Tiền xử lý  $f[i][k] =$  kết quả gộp của đoạn dài  $2^k$  bắt đầu tại  $i$ , tốn  $O(n \log n)$  thời gian và bộ nhớ. Truy vấn  $[l, r]$ : lấy  $k = \lfloor \log_2(r - l + 1) \rfloor$  và gộp hai đoạn dài  $2^k$ , một bắt đầu tại  $l$ , một kết thúc tại  $r$ . Hai đoạn này *chồng lên nhau*, và đó chính là chỗ cần tính lũy đẳng — với tổng thì cách này sai vì phần chồng bị đếm hai lần.

Vậy nên bảng so sánh dưới đây tóm tắt lựa chọn, và nó đáng thuộc vì đây là quyết định bạn phải ra rất nhiều lần.

| Cấu trúc            | Truy vấn      | Cập nhật                  | Điều kiện          | Chọn khi                                              |
|---------------------|---------------|---------------------------|--------------------|-------------------------------------------------------|
| Tổng tiền tố (B/06) | $O(1)$        | $O(n)$                    | có nghịch đảo      | Mảng tĩnh, phép có nghịch đảo                         |
| Sparse table        | $O(1)$        | $O(n)$ sửa ô bị ảnh hưởng | kết hợp, lũy đẳng  | Mảng tĩnh, hỏi cực trị rất nhiều                      |
| Fenwick             | $O(\log n)$   | $O(\log n)$               | tổng/XOR, cộng dồn | Tổng có cập nhật; ưu tiên vì ngắn và nhanh            |
| Cây phân đoạn       | $O(\log n)$   | $O(\log n)$               | kết hợp            | Phép toán bất kỳ có kết hợp; cần cập nhật đoạn (lazy) |
| Phân rã $\sqrt{n}$  | $O(\sqrt{n})$ | $O(1)$ hoặc $O(\sqrt{n})$ | gộp khối đủ rẻ     | Tổng/cực trị theo khối; kiểm tra chi phí gộp          |

## 5.4 Phân rã căn bậc hai và thuật toán Mo

Khi không tìm được trạng thái gọn để gộp hai đoạn, hãy xét xử lý theo khối; đây không phải lời giải tự động cho mọi phép toán không kết hợp. Lúc đó vẫn còn một hướng: chia mảng thành  $\sqrt{n}$  khối, mỗi khối  $\sqrt{n}$  phần tử, và giữ thông tin tổng hợp cho từng khối. Truy vấn đoạn xử lý các khối nằm trọn bằng thông tin đã lưu, và duyệt tay các phần tử ở hai đầu — tổng cộng  $O(\sqrt{n})$  nếu mỗi khối được gộp trong  $O(1)$ .

Cần thiết kế và phân tích thao tác gộp/cập nhật khối; đổi lại cấu trúc khá linh hoạt, và code thường ngắn hơn cây phân đoạn có lazy. Trong nhiều bài,  $O(n\sqrt{n})$  vẫn nằm trong ngân sách của chương A/03 và đó là lựa chọn hợp lý — đặc biệt khi bạn còn ít thời gian trong kỳ thi.

Một biến thể rất đẹp là *thuật toán Mo*: khi mọi truy vấn được biết trước (truy vấn ngoại tuyến), hãy sắp xếp chúng theo khối của đầu trái rồi theo đầu phải, và di chuyển hai con trỏ dần dần từ truy vấn này sang truy vấn kế tiếp. Tổng quãng đường của hai con trỏ là  $O((n+q)\sqrt{n})$ , và ta giải được những bài mà cây phân đoạn không đụng tới được — ví dụ “có bao nhiêu giá trị phân biệt trong đoạn” hay “phần tử xuất hiện nhiều nhất trong đoạn”. Chú ý điều kiện đánh đổi: kỹ thuật này đòi truy vấn phải biết trước, tức là bạn mua tốc độ bằng cách từ bỏ tính trực tuyến.

## 5.5 Ba mở rộng nên biết là có

*Cây phân đoạn bền vững*. Mỗi cập nhật chỉ đụng  $O(\log n)$  nút, nên thay vì sửa tại chỗ, ta tạo bản sao của đúng đường đó và chia sẻ phần còn lại với phiên bản cũ. Kết quả: giữ được mọi phiên bản với chi phí  $O(\log n)$  mỗi lần. Ứng dụng kinh điển là truy vấn “phần tử nhỏ thứ  $k$  trong đoạn  $[l, r]$ ” — giải bằng hiệu của hai phiên bản.

*Hai chiều*. Cây phân đoạn của các cây phân đoạn cho truy vấn trên hình chữ nhật với  $O(\log^2 n)$ . Tốn bộ nhớ, và trong thực tế thường thay bằng Fenwick hai chiều nếu chỉ cần tổng.

*Cây phân đoạn “beats”*. Xử lý được các cập nhật lạ như “gán mọi phần tử trong đoạn bằng  $\min(a_i, x)$ ” với phân tích khấu hao tinh vi. Xếp vào loại biết là có, tra khi gặp.

### CẠM BẮY

Lỗi cài đặt phổ biến nhất với cây phân đoạn không nằm ở logic cây mà ở *phần tử trung hoà*. Truy vấn cực đại trên đoạn rỗng phải trả về  $-\infty$ , không phải 0; truy vấn GCD trên đoạn rỗng phải trả về 0, không phải 1. Lỗi này ẩn kỹ vì nó chỉ lộ ra khi mảng có số âm hoặc khi truy vấn rơi vào nhánh không giao. Cách phòng: viết rõ phần tử trung hoà thành một hằng số có tên, và tự hỏi “nếu đoạn rỗng thì hàm này trả về gì”.

## 5.6 Biến cấu trúc tĩnh thành cấu trúc động: phép biến đổi Bentley–Saxe

Sparse table ở mục 3 trả lời truy vấn trong  $O(1)$  nhưng đòi hỏi mảng đứng yên. Đó không phải trường hợp cá biệt: rất nhiều cấu trúc mạnh đều là cấu trúc *tĩnh* — mảng đã sắp cộng tìm kiếm nhị phân, bao lồi dựng sẵn, cây phân hoạch không gian. Câu hỏi tự nhiên là có cách chung nào thêm phép chèn vào một cấu trúc tĩnh bất kỳ không, thay vì phải thiết kế lại từ đầu cho từng bài. Bentley và Saxe[11] trả lời là có, với một điều kiện rõ ràng, và lời giải của họ đơn giản tới mức nên thuộc.

Điều kiện: bài toán truy vấn phải *phân rã được*, nghĩa là khi tập dữ liệu tách thành hai phần rời nhau thì đáp án trên cả tập gộp được từ hai đáp án thành phần bằng một phép toán rẻ. Cụ thể, cực tiểu, cực đại, tổng, “có tồn tại phần tử thoả tính chất này không”, “điểm gần nhất tới  $q$ ” đều phân rã được. “Phần tử nhỏ thứ  $k$ ” thì không — biết phần tử thứ  $k$  của mỗi nửa không cho biết phần tử thứ  $k$  của cả tập.

Cách dựng đọc như một phép đếm nhị phân, và đó chính là trực giác nên giữ. Ta không nuôi một cấu trúc lớn mà nuôi một *bộ* cấu trúc tĩnh có kích thước 1, 2, 4, 8, ..., mỗi luỹ thừa của hai có nhiều nhất một cái — đúng như biểu diễn nhị phân của số phần tử hiện có. Chèn một phần tử là dựng một cấu trúc kích thước 1; để có hai cấu trúc cùng kích thước thì huỷ cả hai và dựng lại một cấu trúc tĩnh kích thước gấp đôi, đúng như việc nhớ 1 khi cộng nhị phân. Truy vấn thì hỏi cả  $O(\log n)$  cấu trúc rồi gộp các đáp án lại.

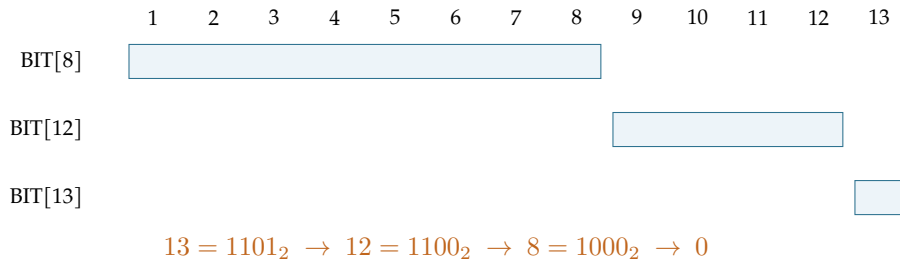
Chi phí suy ra trực tiếp từ hình ảnh đó. Mỗi phần tử chỉ bị dựng lại một lần cho mỗi tầng nó leo qua, tức là  $O(\log n)$  lần trong cả đời; nên nếu dựng cấu trúc tĩnh cho  $n$  phần tử tốn  $P(n)$  thì chèn tốn  $O(\frac{P(n)}{n} \log n)$  khấu hao, còn truy vấn tốn thêm một thừa số  $O(\log n)$  so với bản tĩnh. Lập luận này chính là lập luận bộ đếm nhị phân ở chương A/03, xuất hiện lại ở một vai trò hoàn toàn khác.

Ba hệ quả thực dụng. Một, khi bạn có một cấu trúc tĩnh viết sẵn và bài lại đòi thêm phần tử *trực tuyến*, đừng vội đi tìm cấu trúc động tương ứng — hãy hỏi bài toán có phân rã được không đã. Hai, phép biến đổi này không cho phép xoá; muốn xoá phải dùng các biến thể khác của cùng dòng nghiên cứu, và trong bài thi thì thường rẻ hơn nhiều nếu chuyển sang xử lý ngoại tuyến bằng chia để trị trên trục thời gian ( $C/14$ ). Ba, nếu đã có một cấu trúc động tự nhiên — cây phân đoạn cho các phép toán kết hợp là ví dụ ngay trong chương này — thì dùng nó, vì phép biến đổi tổng quát luôn phải trả thêm một thừa số lôgarit cho tính tổng quát.

Hình dạng “nhiều tầng, mỗi tầng gấp đôi, hợp nhất khi đầy, truy vấn phải hỏi mọi tầng” còn xuất hiện ở nơi khác hẳn: các động cơ lưu trữ ghi nhiều đọc ít trong cơ sở dữ liệu hiện đại tổ chức dữ liệu đúng theo kiểu đó. Sự trùng hình dạng ấy không phải quan hệ vay mượn có chứng cứ lịch sử, nhưng nó cho thấy cùng một ràng buộc — ghi phải rẻ, và cấu trúc tĩnh thì rẻ hơn cấu trúc động — dẫn tới cùng một lời giải ở hai ngành cách nhau ba chục năm.

## 5.7 Nhìn lowbit và kiểm hợp thành lazy bằng tay

Hình 5.2 triển khai trực tiếp truy vấn tiền tố 13. Mỗi bước xoá bit 1 thấp nhất nên các khối vừa khít, không chồng lấn. Để tự kiểm cài đặt, thử thêm  $\Delta$  tại vị trí 11: với  $n = 16$ , các ô bị cập nhật phải là 11, 12, 16.



**Hình 5.2.** Tiền tố  $[1, 13]$  là hợp rời của  $[13, 13]$ ,  $[9, 12]$ ,  $[1, 8]$ . Vì thế tổng cần đúng ba ô Fenwick:  $\text{BIT}[13] + \text{BIT}[12] + \text{BIT}[8]$ .

Lazy propagation còn đòi cập nhật tác động được lên trạng thái tổng hợp và hợp thành được trong thời gian hằng số [12]. Với cộng đoạn và hỏi tổng, lưu  $(sum, len)$ ; cộng  $d$  biến nó thành  $(sum + d \cdot len, len)$ . Không lưu độ dài thì không biết phải tăng tổng bao nhiêu. Với cả gán và cộng, thứ tự là phần của thuật toán: gán 5 rồi cộng 3 cho kết quả 8, còn cộng 3 rồi gán 5 cho kết quả 5. Hãy kiểm hai chuỗi thao tác này trên mảng hai phần tử trước khi thử ngẫu nhiên.

#### CẠM BÃY

Không phải mọi phép gộp kết hợp đều tự động hỗ trợ mọi cập nhật lazy. Ví dụ chỉ lưu tổng thì không đủ cập nhật  $a_i \leftarrow \min(a_i, x)$ : hai đoạn  $[0, 10]$  và  $[5, 5]$  cùng tổng 10, nhưng chặn trên ở 5 cho tổng 5 và 10. Muốn xử lý phải thêm thông tin và chứng minh chi phí, như segment tree beats.

### 5.8 Gốc rễ: từ nén dữ liệu và hình học tới bài toán truy vấn đoạn

Điều thú vị là hai cấu trúc chính của chương này ra đời ở hai ngành hoàn toàn khác nhau và không phải để giải bài “truy vấn đoạn” như ta biết ngày nay.

Cây phân đoạn xuất hiện trong *hình học tính toán* cuối thập niên 1970, gắn với tên Jon Bentley. Bài toán gốc là bài toán đâm xuyên: cho một tập đoạn thẳng trên đường thẳng, hỏi một điểm bị bao nhiêu đoạn phủ. Cấu trúc phân cấp theo đoạn ra đời từ đó, và mãi về sau cộng đồng thuật toán mới dùng nó cho bài toán truy vấn tổng/cực trị trên mảng — ứng dụng mà ngày nay hầu như là ứng dụng duy nhất người ta nhắc tới.

Fenwick ra đời năm 1994 từ một bài toán *nén dữ liệu*. Trong mã hoá số học, bộ mã cần liên tục cập nhật tần suất của từng ký hiệu và liên tục hỏi tổng tần suất lũy tích — đúng bài toán “cập nhật điểm, truy vấn tiền tố”. Peter Fenwick tìm ra cách biểu diễn bằng bit thấp nhất và công bố nó như một giải pháp cho bài toán nén; cấu trúc gọn tới mức ngày nay nó là thứ đầu tiên người ta viết ra khi cần tổng đoạn có cập nhật.

Cả hai câu chuyện minh hoạ cùng một điều đáng nhớ về cách kiến thức lan truyền: cấu trúc dữ liệu hay được sinh ra để phục vụ một ứng dụng rất hẹp, rồi được cộng đồng khác nhận ra là công cụ tổng quát. Với người học, hệ quả thực dụng là: khi bạn gặp một bài lạ, rất có thể công cụ cần dùng đã tồn tại ở một ngành khác dưới một cái tên khác — và đó cũng là lý do phần *Liên hệ ngang* có mặt trong mọi chương của quyển sách này.

#### Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                                                 | Bài toán thúc ép lúc đó                                               | Điều thay đổi sau đó                                                                                                 |
|-----------------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Cuối thập niên 1970 — Bentley và hình học tính toán | Bài toán đâm xuyên: một điểm bị bao nhiêu đoạn phủ.                   | Cây phân đoạn ra đời cho hình học; ứng dụng “truy vấn tổng trên mảng” đến sau và trở thành ứng dụng chính.           |
| 1994 — Fenwick                                      | Mã hoá số học cần cập nhật tần suất và hỏi tổng lũy tích liên tục.    | Cấu trúc dựa trên bit thấp nhất; ngăn tới mức thành lựa chọn mặc định cho tổng đoạn có cập nhật.                     |
| Thập niên 1980–2000 — bài toán RMQ và LCA           | Truy vấn cực tiểu trên đoạn xuất hiện khắp nơi, cần lời giải $O(1)$ . | Sparse table và các lời giải tiền xử lý tuyến tính; nối RMQ với bài toán tổ tiên chung gần nhất trên cây ( $C/17$ ). |
| Thập niên 2000 — cộng đồng thi đấu                  | Nhiều bài truy vấn không có tính kết hợp nên cây chịu.                | Phân rã căn bậc hai và thuật toán Mo: đổi tính trực tuyến lấy khả năng xử lý phép toán bất kỳ.                       |
| Thập niên 2000 — cấu trúc bền vững                  | Cần truy vấn trạng thái ở các thời điểm quá khứ.                      | Cây phân đoạn bền vững bằng chia sẻ đường đi; giải được “phần tử nhỏ thứ $k$ trong đoạn”.                            |

## 5.9 Liên hệ ngang

### Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực                  | Ý tưởng tương ứng                                                        | Giống ở đâu, khác ở đâu                                                                                                                                                                                                                      |
|---------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Đại số trừu tượng         | Vị nhóm: phép kết hợp cộng phần tử trung hoà                             | Không phải chỉ tương tự — đây đúng là điều kiện toán học để cây phân đoạn hoạt động. Biết khái niệm này giúp trả lời ngay câu “bài này dùng cây phân đoạn được không”: chỉ cần kiểm tính kết hợp.                                            |
| Thị giác máy tính, đồ hoạ | Kim tự tháp ảnh, mipmap, ảnh tích phân                                   | Cùng ý tưởng tổng hợp phân cấp: mỗi tầng lưu bản tóm tắt của tầng dưới với độ phân giải giảm nửa. Khác: ở đó mục tiêu là lọc và lấy mẫu theo tỉ lệ, ở đây là trả lời truy vấn nhanh.                                                         |
| Cơ sở dữ liệu phân tích   | Khối dữ liệu OLAP, bảng tổng hợp trước, chuỗi thời gian có tổng theo cấp | Cùng đánh đổi trả trước để truy vấn rẻ, và cùng vấn đề: dữ liệu thay đổi thì phải cập nhật tổng hợp. Cây phân đoạn chính là bản trong bộ nhớ của ý tưởng đó.                                                                                 |
| Xử lý tín hiệu            | Biến đổi wavelet, phân tích đa phân giải                                 | Cùng cấu trúc phân cấp nhị phân trên miền chỉ số. Bài học chung: biểu diễn đa mức cho phép mô tả cả thay đổi cục bộ lẫn tổng thể với chi phí lôgarit.                                                                                        |
| Tính toán song song       | Cây rút gọn (reduce) và scan trên GPU                                    | Cây phân đoạn chính là cây rút gọn được lưu lại thay vì tính xong rồi bỏ. Điều kiện dùng được cũng giống nhau: phép toán phải kết hợp — đó là lý do reduce trong mọi thư viện song song đều yêu cầu điều kiện này.                           |
| Lập trình hàm             | Cấu trúc bền vững, chia sẻ cấu trúc                                      | Cây phân đoạn bền vững là ứng dụng trực tiếp của kỹ thuật chia sẻ đường đi[10]; cùng lý do vì sao cây hợp với bất biến còn mảng thì không.                                                                                                   |
| Hệ thống giám sát         | Cửa sổ thời gian, tổng hợp theo cấp phút/giờ/ngày                        | Cùng cấu trúc phân cấp theo thời gian. Điểm giống thú vị: các hệ này cũng chỉ hỗ trợ được các phép tổng hợp có tính kết hợp, và “số giá trị phân biệt” cũng chính là phép khó ở đó — nên họ dùng cấu trúc xấp xỉ như HyperLogLog ( $B/08$ ). |

## TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì Fenwick tính đoạn  $[l, r]$  bằng *hiệu* của hai tiền tố, nên nó cần phép toán có nghịch đảo. Với cực đại thì biết max của  $[1, r]$  và của  $[1, l - 1]$  không suy ra được max của  $[l, r]$  — thông tin đã mất. Cây phân đoạn không dùng phép trừ mà *ghép* kết quả của các đoạn con rời nhau, nên chỉ cần tính kết hợp chứ không cần nghịch đảo. Đây là ranh giới rõ ràng: có nghịch đảo thì Fenwick (ngắn hơn, nhanh hơn); chỉ có kết hợp thì cây phân đoạn.
- A2. Vì sparse table đòi mảng *tĩnh*: cập nhật một phần tử có thể làm hỏng  $O(n)$  ô (ở tầng  $k$ , tối đa  $2^k$  ô chứa điểm đó). Xây lại toàn bảng tốn  $O(n \log n)$ , nhưng sửa đúng các ô bị ảnh hưởng tốn  $O(n)$ . Nó cũng đòi phép toán lũy đẳng, vì hai đoạn dài  $2^k$  dùng để phủ truy vấn có phần chồng lên nhau — với tổng thì phần chồng bị đếm hai lần. Vậy sparse table thắng đúng một tình huống: mảng không đổi, phép lũy đẳng, và số truy vấn rất lớn nên  $O(1)$  đáng giá.
- A3. Vì bất biến được duy trì là: giá trị tổng hợp lưu tại mỗi nút *đã đúng*, còn khoản nợ tại nút chỉ mô tả phần chưa được truyền xuống các con. Mọi thao tác muốn đọc thông tin của con đều phải đi xuống qua nút cha, và ta luôn đẩy nợ trước khi đi xuống — nên không ai từng đọc dữ liệu chưa cập nhật. Đây là lập luận bất biến chuẩn của A/04, và nó cũng chỉ ra chỗ dễ sai nhất khi cài: quên đẩy nợ ở đúng một nhánh.
- A4. Ở hằng số, không ở bậc. Fenwick là một vòng lặp trên mảng với vài phép toán bit, không đệ quy, không cấp phát, truy cập gần như tuần tự nên rất hợp cache. Cây phân đoạn thường viết đệ quy, mỗi nút truy cập hai con ở vị trí xa nhau, và dùng gấp bốn lần bộ nhớ. Với cùng  $\log n$  bước, chênh lệch thực tế thường là ba tới năm lần — đủ để quyết định giữa đạt và quá thời gian.
- A5. Có, và đó là lý do chính để biết phân rã căn bậc hai. Ví dụ “có bao nhiêu giá trị phân biệt trong đoạn  $[l, r]$ ” không kết hợp được: biết số giá trị phân biệt của hai nửa không cho biết số giá trị phân biệt của cả đoạn, vì không biết phần chung. Thuật toán Mo giải được bằng cách di chuyển hai đầu mút dần dần và duy trì một bảng đếm — với cái giá là mọi truy vấn phải biết trước, tức là mất tính trực tuyến.

## NÓI LẠI THẬT ĐƠN GIẢN

Hãy hình dung một dãy phố với một trăm cửa hàng, và bạn thường xuyên phải trả lời câu “tổng doanh thu từ cửa hàng số 20 tới số 57 là bao nhiêu?”. Nếu doanh thu không bao giờ đổi, bạn chỉ cần một bảng cộng dồn từ đầu phố — lấy số ở 57 trừ số ở 19 là xong.

Nhưng nếu mỗi ngày có vài cửa hàng báo lại doanh thu mới thì bảng cộng dồn hỏng: sửa một cửa hàng làm sai mọi con số phía sau nó.

Cách chữa là chia phố thành nhóm lồng nhau: hai nửa phố, mỗi nửa lại chia đôi, cứ thế xuống từng cửa hàng. Mỗi nhóm ghi sẵn tổng của nhóm mình. Khi một cửa hàng đổi số, chỉ những nhóm chứa nó — khoảng bảy nhóm với một trăm cửa hàng — phải sửa. Khi có người hỏi một khoảng, bạn ghép khoảng đó từ vài nhóm có sẵn thay vì cộng từng cửa hàng.

Còn nếu sếp nói “tăng thuế thêm 3% cho cả đoạn từ 20 tới 57” thì bạn không cần đi sửa từng cửa hàng. Bạn dán một mẫu giấy nợ lên các nhóm nằm trọn trong đoạn đó, và chỉ khi nào có ai hỏi chi tiết bên trong một nhóm thì mới truyền mẫu giấy ấy xuống các nhóm con. Đó là lazy propagation, và nó đúng vì bất kỳ ai muốn nhìn vào bên trong đều phải đi qua chỗ dán giấy.

Một điều cuối cùng đáng nhớ: cách này chỉ chạy được với những câu hỏi mà *biết đáp án của hai nửa thì suy ra được đáp án của cả đoạn*. Tổng thì được, lớn nhất thì được, nhưng “mặt hàng bán chạy nhất trong đoạn” thì không — và đó là ranh giới giữa những bài dùng được cây và những bài phải dùng cách khác.

Một câu: Chia đoạn thành các mảnh lồng nhau: sửa một chỗ chỉ đụng  $\log n$  mảnh, hỏi một đoạn chỉ ghép  $\log n$  mảnh.

**Chỗ không rút gọn được:** Việc phép toán có kết hợp hay không là tính chất của bài toán, không phải của cấu trúc — không có mẹo nào làm cho “giá trị phổ biến nhất” trở nên kết hợp được.

| Thuật ngữ / câu hỏi                              | Trả lời                                                                                                                                                                                                              |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● Điều kiện chung của cả chương                | Phép toán phải <i>kết hợp</i> : kết quả cả đoạn ghép được từ hai nửa. Cộng thêm phần tử trung hoà thì có một vị nhóm.                                                                                                |
| ●●● Fenwick so với cây phân đoạn                 | Fenwick cần phép có <i>ngịch đảo</i> (dùng hiệu hai tiền tố), thường gọn hơn; tốc độ cần đo. Cây phân đoạn chỉ cần kết hợp, tổng quát hơn, làm được lazy.                                                            |
| ●●● Vì sao truy vấn cây phân đoạn là $O(\log n)$ | Mỗi tầng đóng góp tối đa hai nút được dùng nguyên vẹn; các nhánh còn lại hoặc nằm gọn (dùng) hoặc không giao (bỏ).                                                                                                   |
| ●●● Lazy propagation — bất biến                  | Giá trị tại nút đã đúng; khoản nợ mô tả phần chưa áp cho con. Mọi đường đi xuống đều đẩy nợ trước khi đọc con.                                                                                                       |
| ●●● Sparse table — điều kiện                     | Mảng tĩnh và phép <i>luỹ đẳng</i> ( $a \oplus a = a$ ), vì hai đoạn phủ truy vấn chồng lên nhau. Tiền xử lý $O(n \log n)$ , truy vấn $O(1)$ .                                                                        |
| ●●● Bảng chọn cấu trúc                           | Tĩnh + có nghịch đảo: tổng tiền tố. Tĩnh + lũy đẳng: sparse table. Động + nghịch đảo: Fenwick. Động + chỉ kết hợp: cây phân đoạn. Không có trạng thái gộp gọn: xét chia khối hoặc Mo, rồi kiểm chi phí các thao tác. |
| ●● Lỗi cài đặt phổ biến nhất                     | Phần tử trung hoà sai: trả 0 cho truy vấn cực đại trên đoạn rỗng khi mảng có số âm. Hãy đặt nó thành hằng số có tên.                                                                                                 |
| ●● Thuật toán Mo                                 | Truy vấn ngoại tuyến, sắp theo khối của đầu trái rồi đầu phải, di chuyển hai con trỏ; $O((n + q)\sqrt{n})$ . Giải được bài không kết hợp như đếm giá trị phân biệt.                                                  |
| ●● Phân rã căn bậc hai                           | $\sqrt{n}$ khối, mỗi khối giữ tổng hợp; truy vấn $O(\sqrt{n})$ . Cần gộp mỗi khối đủ rẻ; không tự động xử lý mọi phép toán.                                                                                          |
| ●● Cây phân đoạn bền vững                        | Mỗi cập nhật sao chép đúng một đường $O(\log n)$ nút, chia sẻ phần còn lại; giải “phần tử nhỏ thứ $k$ trong đoạn”.                                                                                                   |
| ●● Fenwick tìm vị trí theo tổng                  | Đi xuống theo bit để tìm vị trí nhỏ nhất có tổng tiền tố vượt $x$ , $O(\log n)$ — thay được cây thống kê thứ tự ở B/09.                                                                                              |
| ● Hai chiều                                      | Fenwick 2D cho tổng trên hình chữ nhật; cây phân đoạn của cây phân đoạn cho $O(\log^2 n)$ với phép tổng quát hơn, tốn bộ nhớ.                                                                                        |
| ●● Phép biến đổi Bentley–Saxe                    | Bài toán phân rã được cộng một bộ cấu trúc tính kích thước 1, 2, 4, ..., hợp nhất như phép cộng nhị phân: thêm được phần tử trực tuyến với giá một thừa số $O(\log n)$ . Không hỗ trợ xoá.                           |

●●● phải thuộc ●● nên nắm ● biết là có

## I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

### 1. Tôi nên học Fenwick hay cây phân đoạn trước?

| Fenwick trước, vì nó ngắn, cho kết quả nhanh và dạy đúng ý tưởng cốt lõi “mỗi phần tử nằm trong  $\log n$  đoạn tóm tắt”. Nhưng đừng dừng ở đó: cây phân đoạn mới là cấu trúc tổng quát, và một khi đã quen thì bạn viết nó gần như tự động. Lộ trình hợp lý là Fenwick cho tổng, rồi cây phân đoạn không lazy, rồi lazy, rồi mới tới các mở rộng.

### 2. Cây phân đoạn nên cài đệ quy hay lặp?

| Bản đệ quy dài hơn nhưng dễ mở rộng và bắt buộc nếu cần lazy; bản lặp (cây phân đoạn không đệ quy) ngắn hơn và nhanh hơn nhưng khó thêm lazy và khó xử lý các phép không giao hoán. Lời khuyên thực dụng: học kỹ bản đệ quy trước vì nó là bản dùng được cho mọi bài, rồi mới thêm bản lặp vào thư viện riêng cho những bài chỉ cần cập nhật điểm.

### 3. Bài yêu cầu “gán mọi phần tử trong đoạn bằng $x$ ” thay vì cộng — lazy có khác gì không?

| Có một khác biệt quan trọng: phép gán *không giao hoán* với phép cộng, nên khi hai loại cập nhật cùng xuất hiện, bạn phải định nghĩa rõ cách gộp hai khoản nợ và thứ tự áp dụng. Quy tắc an toàn: biểu diễn khoản nợ dưới dạng một phép biến đổi affine ( $a \mapsto ka + b$ ) và định nghĩa phép hợp thành của hai biến đổi. Khi đó mọi thứ trở lại đúng khuôn “vị nhóm”, chỉ là vị nhóm của các phép biến đổi thay vì của các giá trị.

4. Khi nào tôi nên dùng thuật toán Mo thay vì cố tìm cấu trúc dữ liệu?

| Khi cả ba điều kiện cùng đúng: mọi truy vấn biết trước; thao tác thêm/bớt một phần tử ở biên là  $O(1)$ ; và  $O((n+q)\sqrt{n})$  nằm trong ngân sách. Điều kiện thứ hai hay bị bỏ qua — nếu mỗi bước di chuyển con trỏ tốn  $O(\log n)$  thì tổng thành  $O((n+q)\sqrt{n} \log n)$  và thường quá giờ. Trước khi dùng Mo, hãy luôn tự hỏi “phép này có kết hợp không” một lần nữa, vì cây phân đoạn luôn tốt hơn khi dùng được.

5. Nén toạ độ là gì và vì sao nó hay đi kèm chương này?

| Là việc ánh xạ các giá trị thật — có thể tới  $10^9$  — về các chỉ số liên tiếp  $0, 1, 2, \dots$  bằng cách sắp xếp và loại trùng. Nó cần thiết vì Fenwick và cây phân đoạn cấp phát bộ nhớ theo kích thước miền giá trị, chứ không theo số phần tử. Với  $10^5$  giá trị nằm rải rác trong  $[0, 10^9]$ , nén xuống còn  $10^5$  chỉ số làm cấu trúc khả thi. Đây là bước tiền xử lý gần như bắt buộc cho các bài đếm nghịch thế hay truy vấn theo giá trị.

6. Có cách nào cập nhật đoạn và truy vấn đoạn chỉ bằng Fenwick không?

| Có, dùng hai mảng Fenwick. Ý tưởng: viết tổng tiền tố sau các cập nhật đoạn dưới dạng một biểu thức tuyến tính theo chỉ số, rồi tách thành phần hệ số và phần hằng, mỗi phần một Fenwick. Kết quả là code vẫn ngắn hơn cây phân đoạn có lazy và nhanh hơn đáng kể — đáng để có sẵn trong thư viện riêng. Hạn chế: chỉ áp dụng cho tổng, không mở rộng sang cực trị.

7. Truy vấn cực tiểu trên đoạn có liên quan gì tới bài toán tổ tiên chung gần nhất trên cây?

| Hai bài quy dẫn được về nhau và đây là một liên hệ đẹp. Tổ tiên chung gần nhất của hai đỉnh chính là đỉnh có độ sâu nhỏ nhất trong đoạn tương ứng của dãy Euler khi duyệt cây — tức là một truy vấn cực tiểu trên đoạn. Ngược lại, mọi bài cực tiểu trên đoạn cũng dựng được thành bài tổ tiên chung trên cây Cartesian của mảng. Nhờ quy dẫn đó, một lời giải tốt cho bài này lập tức là lời giải tốt cho bài kia — chi tiết ở C/17.

**Bài tập** (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                                    | Nguồn                                   | Điều cần nhận ra                                                                                                     |
|--------------------------------------------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| ● Tổng đoạn có cập nhật điểm                           | CSES “Dynamic Range Sum Queries”[1]     | Fenwick; viết lại từ trí nhớ tới khi không cần tra.                                                                  |
| ● Cực tiểu đoạn với mảng tĩnh                          | CSES “Static Range Minimum Queries”[1]  | Sparse table; giải thích vì sao cách này sai nếu đổi sang tổng.                                                      |
| ●● Cực tiểu đoạn có cập nhật điểm                      | CSES “Dynamic Range Minimum Queries”[1] | Cây phân đoạn; chú ý phần tử trung hoà.                                                                              |
| ●● Đếm số nghịch thế bằng Fenwick sau khi nén toạ độ   | Bài kinh điển                           | Hai kỹ thuật ghép lại; so với cách sắp xếp trộn ở C/14.                                                              |
| ●● Cộng một giá trị vào đoạn, hỏi giá trị tại một điểm | CSES “Range Update Queries”[1]          | Fenwick trên mảng hiệu; đối chiếu với cách mảng hiệu thuần ở B/06.                                                   |
| ●● Cộng vào đoạn và hỏi tổng đoạn                      | Bài kinh điển                           | Cây phân đoạn có lazy; hoặc hai Fenwick — hãy cài cả hai và so thời gian.                                            |
| ●●● Đoạn con có tổng lớn nhất, có cập nhật điểm        | Bài thi đấu kinh điển                   | Mỗi nút lưu bốn giá trị (tổng, tiền tố tốt nhất, hậu tố tốt nhất, đáp án); ví dụ đẹp nhất về việc thiết kế phép gộp. |
| ●●● Số giá trị phân biệt trong đoạn                    | Bài kinh điển                           | Thuật toán Mo hoặc cách ngoại tuyến với Fenwick; giải thích vì sao cây phân đoạn không dùng được trực tiếp.          |
| ●●● Phần tử nhỏ thứ $k$ trong đoạn                     | Bài thi đấu kinh điển                   | Cây phân đoạn bền vững; hiệu của hai phiên bản chính là “đếm trong đoạn”.                                            |
| ●●● Tổng trên hình chữ nhật có cập nhật điểm           | CSES “Forest Queries II”[1]             | Fenwick hai chiều; so bộ nhớ với cây phân đoạn hai chiều.                                                            |

| Bài                                                                | Nguồn            | Điều cần nhận ra (tiếp)                                                             |
|--------------------------------------------------------------------|------------------|-------------------------------------------------------------------------------------|
| ●●● Cài phân rã căn bậc hai cho một truy vấn không kết hợp tự chọn | Bài tập thiết kế | Mục tiêu là tự nhận ra ranh giới “kết hợp được hay không”, không phải cài cho xong. |

Kết chương và đọc tiếp

Chương này khép lại câu chuyện bắt đầu từ B/06: từ tổng tiền tố tĩnh, qua Fenwick và cây phân đoạn cho dữ liệu động, tới phân rã căn bậc hai cho những phép toán mà cây chịu. Quyết định luôn chỉ dựa trên hai câu hỏi — phép toán có kết hợp không, và mảng có thay đổi không — và bảng ở mục 4 là thứ đáng thuộc nhất.

Chương tiếp theo là cấu trúc có tỉ lệ sức mạnh trên độ dài code cao nhất trong cả quyển sách: hợp nhất tập hợp, chỉ hai chục dòng nhưng đứng sau thuật toán cây khung nhỏ nhất, các bài liên thông ngoại tuyến, và một phân tích khẩu hao đẹp tới mức hàm chi phí của nó tăng chậm hơn mọi hàm bạn từng gặp (B/11).

**Đọc thêm: công trình gốc** (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

| Mức         | Công trình và tác giả                                                                 | Nơi công bố, năm                        | Vì sao đọc, đọc phần nào                                                                                                                   |
|-------------|---------------------------------------------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| nền tảng    | Bentley & Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation” | Journal of Algorithms, 1980             | Nguồn của phép biến đổi ở mục 6. Đọc phần định nghĩa bài toán phân rã được và bảng các phép biến đổi; phần tối ưu tổ hợp đọc sau.          |
| nền tảng    | Fenwick, “A New Data Structure for Cumulative Frequency Tables”                       | Software: Practice and Experience, 1994 | Bài gốc của cây chỉ số nhị phân, viết cho bài toán nén số học. Mười trang, và cách trình bày lowbit ở đây rõ hơn phần lớn tài liệu về sau. |
| tra khi cần | Bentley, “Programming Pearls”                                                         | Addison-Wesley, tái bản 2000            | Đọc các chương về tổng tiền tố và về thiết kế cấu trúc dữ liệu nhỏ; văn phong ngắn, hợp đọc rải rác.                                       |
| tra khi cần | Demaine, khoá học MIT 6.851 <i>Advanced Data Structures</i>                           | tài liệu khoá học công khai, MIT        | Bài giảng về cấu trúc bền vững và về “động hoá” cấu trúc tĩnh, đi tiếp đúng hướng của mục này.                                             |

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [4] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [5] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [6] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [7] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [8] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [9] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [10] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [11] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] AtCoder. “AC Library: Lazy Segtree.” (), address: [https://atcoder.github.io/ac-library/master/document\\_en/lazysegtree.html](https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html) (visited on 09/07/2026).
- [13] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [14] R. E. Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets,” *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [15] M. L. Fredman and M. E. Saks, “The Cell Probe Complexity of Dynamic Data Structures,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [16] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [17] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [18] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-line String Searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

## Hợp nhất tập hợp — DSU (Disjoint Set Union)

**PHỤ THUỘC** A/03 cho phân tích khâu hao; A/04 cho bất biến. Là điều kiện cần cho C/15 (Kruskal) và nhiều bài ở C/17.

### TÓM TẮT

DSU trả lời đúng hai câu hỏi: “hai phần tử này có cùng nhóm không” và “gộp hai nhóm lại”. Chỉ hai thao tác, khoảng hai chục dòng code, nhưng đó là cấu trúc có tỉ lệ sức mạnh trên độ dài code cao nhất trong cả quyển sách — nó đứng sau thuật toán Kruskal, các bài liên thông xử lý ngoại tuyến, kỹ thuật gộp nhỏ vào lớn, và nhiều mẹo đếm. Hai tối ưu đơn giản — gộp theo kích thước và nén đường — đưa chi phí xuống gần như hằng số, với một phân tích khâu hao thuộc loại đẹp nhất ngành. Điểm yếu duy nhất, nhưng rất quan trọng: DSU *không xoá được*. Nếu chỉ nhớ một điều: DSU là cấu trúc một chiều — nhóm chỉ có thể to ra — và mọi kỹ thuật nâng cao trong chương chỉ là cách né hạn chế đó.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **quan hệ tương đương** — quan hệ có tính phản xạ, đối xứng, bắc cầu; nó chia tập thành các lớp rời nhau. DSU chính là cấu trúc duy trì các lớp đó khi quan hệ được bổ sung dần.
- **đại diện (representative)** — một phần tử được chọn làm “tên” của cả nhóm; hai phần tử cùng nhóm khi và chỉ khi có cùng đại diện.
- **gộp theo kích thước / hạng** — luôn treo cây nhỏ vào gốc cây lớn, để chiều cao không vượt  $O(\log n)$ .
- **nén đường (path compression)** — sau mỗi lần tìm đại diện, nối thẳng mọi nút trên đường lên gốc, làm các lần tìm sau rẻ hơn.
- **hàm Ackermann ngược  $\alpha(n)$**  — hàm tăng cực chậm; với mọi  $n$  trong vũ trụ thực tế thì  $\alpha(n) \leq 4$ . Chi phí khâu hao của DSU là  $O(\alpha(n))$ .
- **gộp nhỏ vào lớn (small to large)** — kỹ thuật tổng quát hơn: khi gộp hai tập hợp thật sự, luôn duyệt tập nhỏ và đổ vào tập lớn; tổng chi phí  $O(n \log n)$  vì mỗi phần tử chỉ bị chuyển  $O(\log n)$  lần.

### CÂU HỎI MỞ ĐẦU

- Q1. Vì sao chỉ cần hai mẹo đơn giản — gộp theo kích thước và nén đường — đã đưa chi phí từ  $O(n)$  xuống gần như  $O(1)$ ?
- Q2. Vì sao DSU không hỗ trợ thao tác tách nhóm, và trong bài toán bắt buộc phải tách thì làm thế nào?
- Q3. Kỹ thuật “gộp nhỏ vào lớn” cho  $O(n \log n)$  bằng một lập luận rất ngắn. Lập luận đó là gì?
- Q4. DSU chỉ biết “cùng nhóm hay không”. Làm sao nó trả lời được câu hỏi tinh vi hơn như “hai phần tử này khác phe nhau”?
- Q5. Trong bài toán mà các cạnh bị xoá dần theo thời gian, vì sao xử lý ngược thời gian lại cứu được tình hình?

Có một dạng bài toán rất phổ biến: các phần tử dần dần được nối lại với nhau, và ta cần biết ai đang cùng nhóm với ai. Máy tính trong mạng được nối thêm dây; các pixel cùng màu được gộp

thành vùng; các đỉnh của đồ thị được nối thêm cạnh. Nếu mỗi lần hỏi ta lại chạy một lượt duyệt đồ thị thì chi phí là  $O(n)$  cho mỗi câu hỏi — quá đắt khi có hàng trăm nghìn câu.

DSU là câu trả lời, và điều làm nó đặc biệt không phải sự tinh vi mà là sự *tiết chế*. Nó không lưu danh sách thành viên của mỗi nhóm, không lưu cấu trúc của các liên kết, không trả lời được “nhóm này gồm những ai”. Nó chỉ lưu đúng một con trỏ cha cho mỗi phần tử, và chỉ trả lời đúng câu “ai là đại diện của nhóm chứa phần tử này”. Chính vì hứa ít nên nó rẻ — lại một lần nữa nguyên tắc xuyên suốt phần B.

## 6.1 Từ ngây thơ tới gần như hằng số

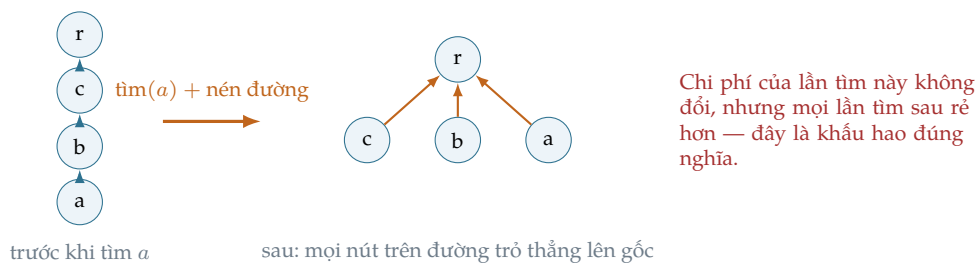
**Biểu diễn:** mỗi phần tử có một con trỏ cha; đi ngược theo cha tới khi gặp nút trỏ vào chính nó, đó là đại diện. Gộp hai nhóm: cho gốc này trỏ vào gốc kia.

Cách ngây thơ này có thể tạo ra một chuỗi dài  $n$ , và mỗi lần tìm tốn  $O(n)$ . Hai tối ưu chứa được điều đó, và điều đáng chú ý là chúng độc lập nhau nhưng bổ sung nhau.

**Gộp theo kích thước.** Luôn treo cây có ít phần tử hơn vào gốc của cây lớn hơn. Bất biến sinh ra: một cây có chiều cao  $h$  phải chứa ít nhất  $2^h$  phần tử — chứng minh bằng quy nạp, vì chiều cao chỉ tăng khi hai cây cùng chiều cao gộp lại, và khi đó kích thước ít nhất gấp đôi. Suy ra chiều cao tối đa  $\log_2 n$ , và mọi thao tác là  $O(\log n)$ .

**Nén đường.** Trong lúc đi tìm đại diện, ta đã đi qua cả một đường; hãy nối thẳng mọi nút trên đường đó lên gốc. Việc này không tốn thêm chi phí tiệm cận nào ở lần tìm hiện tại, nhưng làm mọi lần tìm sau rẻ đi. Đây là một ví dụ mẫu mực cho ý tưởng khấu hao ở chương A/03: trả thêm một chút bây giờ, hưởng lợi nhiều lần về sau.

Dùng cả hai, chi phí khấu hao mỗi thao tác là  $O(\alpha(n))$  với  $\alpha$  là hàm Ackermann ngược — một hàm tăng chậm tới mức với  $n$  nhỏ hơn số nguyên tử trong vũ trụ quan sát được thì  $\alpha(n) \leq 4$ . Kết quả này do Tarjan chứng minh năm 1975[13], và nó nổi tiếng vì kết luận rất bất ngờ: một cấu trúc chỉ dùng một mảng số nguyên lại có chi phí không phải hằng số nhưng gần tới mức không phân biệt được. Về sau người ta còn chứng minh được rằng  $O(\alpha(n))$  là *tối ưu* — không cấu trúc nào làm tốt hơn cho bài toán này.



**Hình 6.1.** Nén đường. Điều đáng nhớ là nó không làm lần tìm hiện tại nhanh hơn — ta vẫn phải đi hết đường — mà làm cấu trúc phẳng đi cho tương lai. Kết hợp với gộp theo kích thước, chi phí khấu hao trở thành  $O(\alpha(n))$ , thực tế là hằng số.

## 6.2 Hạn chế cốt lõi: không xóa được

DSU là cấu trúc *một chiều*: nhóm chỉ có thể to ra, không bao giờ nhỏ đi. Lý do rất trực tiếp: nén đường đã phá hủy thông tin về cấu trúc gốc, nên không có cách nào biết phải tách ở đâu.

Đây không phải khiếm khuyết cài đặt mà là đánh đổi cố ý — chính việc vứt bỏ thông tin làm cấu trúc rẻ như vậy. Nhưng rất nhiều bài toán thật lại có cả xoá: cạnh mạng bị đứt, kết nối bị huỷ. Có ba cách né, và cả ba đều đáng biết vì chúng là mẫu tư duy dùng lại được.

*Cách thứ nhất — đảo ngược thời gian.* Nếu bài toán chỉ có xoá chứ không có thêm, hãy xử lý các truy vấn theo thứ tự ngược: xoá theo chiều xuôi trở thành thêm theo chiều ngược. Đây là một mẹo đơn giản tới mức dễ bị bỏ qua, nhưng nó giải quyết trọn vẹn cả một họ bài. Điều kiện: truy vấn phải biết trước, tức là bài toán ngoại tuyến.

*Cách thứ hai — DSU có hoàn tác.* Bỏ nén đường, chỉ giữ gộp theo kích thước (chi phí thành  $O(\log n)$ ), và ghi lại mọi thay đổi vào một ngăn xếp để có thể hoàn tác. Kết hợp với chia để trị trên trục thời gian, ta giải được bài liên thông động ngoại tuyến tổng quát: mỗi cạnh tồn tại trong một khoảng thời gian, và ta duyệt cây phân đoạn trên thời gian, thêm cạnh khi đi xuống và hoàn tác khi đi lên.

*Cách thứ ba — cấu trúc khác hẳn.* Nếu bài buộc phải trực tuyến và có cả thêm lẫn xoá, DSU không đủ và cần cấu trúc như link-cut tree. Xếp vào loại biết là có.

#### CẠM BÃY

Sự kết hợp giữa nén đường và hoàn tác là không tương thích — đây là lỗi thường gặp khi người ta cố ghép hai kỹ thuật. Nén đường thay đổi nhiều con trỏ trong một lần tìm, và những thay đổi đó không được ghi lại đúng cách nếu bạn chỉ ghi các thao tác gộp. Muốn hoàn tác thì phải bỏ nén đường và chấp nhận  $O(\log n)$ .

### 6.3 Tăng cường DSU: đếm, phe phái, khoảng cách

Giống như cây tìm kiếm ở chương B/09, DSU mạnh lên rất nhiều khi được gắn thêm thông tin — và điều kiện cũng tương tự: thông tin phải *gộp được* khi hai nhóm hợp nhất.

*Đếm.* Lưu kích thước mỗi nhóm ở gốc: gộp thì cộng lại. Cho phép trả lời “nhóm chứa  $x$  có bao nhiêu phần tử” và “hiện có bao nhiêu nhóm”. Đây là dạng dùng nhiều nhất.

*Phe phái, hay DSU có trọng số.* Lưu thêm khoảng cách tương đối từ mỗi nút tới cha nó theo một nhóm cộng — thường là tính chẵn lẻ. Khi đó DSU trả lời được “ $x$  và  $y$  cùng phe hay khác phe”, và phát hiện được mâu thuẫn khi có ràng buộc trái ngược. Ứng dụng: kiểm tra đồ thị hai phía trực tuyến, và các bài ràng buộc kiểu “ $x$  và  $y$  khác loại”. Chú ý khi nén đường phải cộng dồn trọng số dọc đường — đây là chỗ dễ sai nhất.

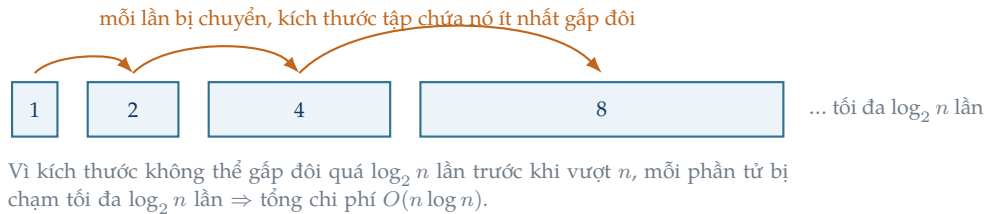
*DSU trên miền giá trị — mẹo “vị trí trống kế tiếp”.* Một cách dùng rất khéo: cho DSU trỏ mỗi vị trí đã bị chiếm sang vị trí trống kế tiếp bên phải. Khi cần “tìm ô trống đầu tiên từ  $i$  trở đi”, chỉ việc tìm đại diện; khi chiếm ô đó thì gộp nó với ô kế tiếp. Kỹ thuật này biến nhiều bài “tô màu đoạn, mỗi ô chỉ tô một lần” từ  $O(n^2)$  thành gần tuyến tính — và nó là ví dụ điển hình cho việc dùng DSU ở nơi mà bài toán không hề nói tới “nhóm” nào cả.

### 6.4 Gộp nhỏ vào lớn: người anh em tổng quát hơn

DSU không lưu danh sách thành viên. Khi bài toán *cần* danh sách — ví dụ phải duyệt mọi phần tử của nhóm khi gộp — thì ta dùng kỹ thuật gộp nhỏ vào lớn: giữ tập hợp thật cho mỗi nhóm, và khi gộp thì luôn duyệt tập nhỏ hơn rồi đổ vào tập lớn hơn.

Chi phí tổng cộng là  $O(n \log n)$ , và lập luận cực ngắn: mỗi lần một phần tử bị chuyển, nó chuyển từ tập nhỏ sang tập lớn, nên kích thước tập chứa nó ít nhất gấp đôi. Kích thước không thể gấp đôi quá  $\log_2 n$  lần, nên mỗi phần tử bị chuyển tối đa  $\log_2 n$  lần. Đây là một trong những lập luận khẩu hao đẹp và dễ nhớ nhất trong quyển sách, và nó dùng lại ở rất nhiều nơi.

Biến thể quan trọng là *DSU trên cây*: khi giải các bài truy vấn về cây con — “trong cây con của  $u$  có bao nhiêu màu phân biệt” — ta duyệt cây, giữ lại cấu trúc đếm của con nặng nhất và gộp các con nhẹ vào. Chi phí  $O(n \log n)$  nhờ đúng lập luận trên.



**Hình 6.2.** Lập luận “gộp nhỏ vào lớn”. Điều đáng học không phải kết quả mà là cách đếm: ta không tính chi phí theo từng phép gộp — có phép gộp rất đắt — mà theo *tổng số lần các phần tử bị chạm*. Cùng kiểu lập luận với khẩu hao ở A/03.

## 6.5 “Tối ưu” nghĩa là gì ở đây: hai cận dưới cho hợp nhất tập hợp

Mục 1 kết thúc bằng một câu đáng ngờ nếu đọc kỹ:  $O(\alpha(n))$  là *tối ưu*. Tối ưu so với cái gì? Không ai chứng minh được cận dưới cho “mọi thuật toán khả dĩ” — chương D/24 giải thích vì sao điều đó gần như bất khả thi. Cái người ta chứng minh được là hai phát biểu hẹp hơn nhưng mạnh một cách bất ngờ, và chúng đáng biết vì chúng cho thấy hàm  $\alpha$  không phải vết tích của một phép phân tích vụng về, mà là một tính chất của chính bài toán.

Kết quả thứ nhất là của Tarjan[14]. Ông xét một lớp thuật toán được mô tả rõ: những thuật toán chỉ thao tác với các phần tử qua con trỏ, đúng như cách DSU đi từ nút lên cha, không được phép làm số học trên nhãn để đi tắt. Trong lớp đó, mọi thuật toán trả lời trực tuyến  $m$  thao tác trên  $n$  phần tử đều cần  $\Omega(m \alpha(m, n))$  trong trường hợp xấu nhất. Vậy trong thế giới con trỏ, nén đường cộng gộp theo kích thước không chỉ tốt: nó chạm cận.

Kết quả thứ hai mạnh hơn nhiều vì nó bỏ luôn giả thiết con trỏ. Fredman và Saks[15] làm việc trong mô hình *cell-probe*, ở đó ta chỉ đếm số ô nhớ mà thuật toán chạm vào và cho phép nó tính bất cứ thứ gì nó muốn trên các ô ấy — kể cả các mẹo bit và song song mức từ máy. Ngay trong mô hình rộng rãi đó, chi phí khẩu hao mỗi thao tác vẫn là  $\Omega(\alpha)$ . Nói cách khác, không có mẹo cài đặt nào đưa bài toán này xuống hằng số thật sự.

Ba điều đáng mang theo. Thứ nhất, đây là một trong số ít trường hợp mà cận trên và cận dưới khớp nhau ở một hàm kỳ dị như  $\alpha$ ; sự khớp đó là bằng chứng rằng hàm ấy thuộc về bài toán chứ không thuộc về cách ta phân tích. Thứ hai, và tổng quát hơn: mọi phát biểu “tối ưu” đều kèm một mô hình, và khi đọc một kết quả như vậy, câu hỏi đầu tiên phải là *tối ưu trong lớp thuật toán nào* (D/24). Thứ ba, về thực hành thì kết luận vẫn giản dị: đừng đi tìm DSU hằng số, và cũng đừng bỏ bớt một trong hai tối ưu — dùng riêng gộp theo kích thước hay riêng nén đường đều chỉ cho  $O(\log n)$ , phải có cả hai mới xuống tới  $\alpha$ .

## 6.6 Gốc rễ: từ câu lệnh EQUIVALENCE của Fortran

Bài toán mà DSU giải ra đời cùng với những trình biên dịch đầu tiên. Ngôn ngữ Fortran có câu lệnh EQUIVALENCE cho phép lập trình viên khai báo rằng hai biến dùng chung một vùng nhớ. Trình biên dịch phải xử lý một loạt khai báo như vậy và tính ra các *lớp tương đương* — nhóm các biến thực sự chia sẻ bộ nhớ. Đó chính xác là bài toán “hợp nhất và hỏi cùng nhóm”.

Năm 1964, Bernard Galler và Michael Fischer công bố thuật toán cho bài toán đó, với cấu trúc rừng cây và ý tưởng đại diện mà ta dùng tới ngày nay. Điều đáng chú ý về mặt phương pháp: bài toán được phát hiện từ một nhu cầu công cụ rất cụ thể, giống hệt câu chuyện bảng băm ở chương B/08.

Phần lý thuyết mất thêm mười năm. Nén đường và gộp theo hạng được dùng khá sớm như những mẹo hiển nhiên, nhưng không ai chứng minh được chúng nhanh tới mức nào. Năm 1975 Tarjan cho lời giải: chi phí khấu hao là  $O(\alpha(n))$  [13] — một kết quả gây bất ngờ vì hàm Ackermann ngược trước đó thuộc về logic và lý thuyết đệ quy, không ai chờ nó xuất hiện trong phân tích một cấu trúc dữ liệu thực dụng. Về sau người ta chứng minh thêm rằng cận này không cải thiện được, tức DSU với hai tối ưu là tối ưu về bản chất, không chỉ tối ưu trong các cách cài đã biết.

Còn một liên hệ ít được nhắc nhưng rất sâu: cùng cấu trúc này là hạt nhân của thuật toán *hợp nhất* trong suy diễn kiểu của trình biên dịch. Khi trình biên dịch suy ra kiểu của một biểu thức, nó liên tục phát biểu “kiểu này phải bằng kiểu kia” và phải duy trì các lớp tương đương của các biến kiểu — đúng bài toán của Galler và Fischer, chỉ khác ngữ cảnh. Nghĩa là DSU vẫn đang chạy trong trình biên dịch mỗi lần bạn dịch một chương trình có suy diễn kiểu.

### I Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                                       | Bài toán thúc ép lúc đó                                                                | Điều thay đổi sau đó                                                                                                                 |
|-------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 1964 — Galler & Fischer                   | Trình biên dịch Fortran phải xử lý câu lệnh EQUIVALENCE: các biến dùng chung vùng nhớ. | Cấu trúc rừng với đại diện; bài toán lớp tương đương được hình thức hoá.                                                             |
| Cuối thập niên 1960 — các cài đặt thực tế | Chuỗi cha dài làm thao tác tìm chậm.                                                   | Gộp theo hạng và nén đường xuất hiện như mẹo hiển nhiên, nhưng chưa ai biết chúng nhanh tới đâu.                                     |
| 1975 — Tarjan                             | Cần biết chính xác chi phí của hai mẹo đó.                                             | Chi phí khấu hao $O(\alpha(n))$ ; hàm Ackermann ngược lần đầu xuất hiện trong phân tích một cấu trúc thực dụng.                      |
| Sau đó — các kết quả cận dưới             | Có làm tốt hơn $\alpha$ được không?                                                    | Không: $O(\alpha(n))$ là tối ưu cho bài toán này trong mô hình con trỏ; một trong số ít cấu trúc dữ liệu đạt tối ưu chứng minh được. |
| Thập niên 1970–80 — suy diễn kiểu         | Trình biên dịch phải duy trì các ràng buộc “kiểu này bằng kiểu kia”.                   | Thuật toán hợp nhất dùng đúng cấu trúc DSU; nó chạy mỗi lần bạn biên dịch một chương trình có suy diễn kiểu.                         |

## I 6.7 Liên hệ ngang

### I Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực                        | Ý tưởng tương ứng                                                               | Giống ở đâu, khác ở đâu                                                                                                                                                                                                                                          |
|---------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trình biên dịch                 | Hợp nhất trong suy diễn kiểu                                                    | Cùng cấu trúc, cùng thao tác: gộp hai lớp và hỏi cùng lớp. Điểm khác đáng chú ý: ở đó việc phát hiện mâu thuẫn (hai kiểu không hợp nhất được) chính là thông báo lỗi kiểu mà bạn nhìn thấy.                                                                      |
| Thị giác máy tính               | Gán nhãn thành phần liên thông trên ảnh nhị phân                                | Thuật toán hai lượt kinh điển dùng DSU để gộp các nhãn tạm thời. Ứng dụng gần: gộp các vùng ảnh hoặc các điểm đặc trưng thuộc cùng vật thể.                                                                                                                      |
| Vật lý thống kê                 | Mô phỏng thấm (percolation): các ô mở dần, hỏi khi nào hai bờ nối được với nhau | Đây là ứng dụng lịch sử nổi tiếng và trùng khít về hình thức: các ô mở dần chính là các phép gộp, câu hỏi “đã thấm chưa” chính là truy vấn cùng nhóm.                                                                                                            |
| Mạng và hạ tầng                 | Kiểm tra liên thông mạng, thiết kế mạng chi phí nhỏ nhất                        | Kruskal (C/15) chạy trực tiếp trên DSU. Điểm khác trong thực tế: mạng thật có cả sự cố đứt kết nối, tức là có xoá — và đó chính là chỗ DSU thuận tuy không đủ.                                                                                                   |
| Cơ sở dữ liệu, làm sạch dữ liệu | Gộp bản ghi trùng lặp thành một thực thể                                        | “Hai bản ghi này là cùng một người” là một quan hệ tương đương được bổ sung dần — đúng mô hình DSU. Vấn đề thực tế: quyết định sai thì không rút lại được, đúng như hạn chế một chiều ở mục 2.                                                                   |
| Toán học                        | Quan hệ tương đương và tập thương                                               | DSU là hiện thực hoá tính toán của khái niệm tập thương: duy trì các lớp khi quan hệ sinh được bổ sung dần. Nhìn theo góc này thì “không tách được” là hệ quả tự nhiên: bỏ bớt một quan hệ sinh có thể làm cả cấu trúc lớp thay đổi.                             |
| SLAM                            | Gộp các quan sát về cùng một điểm mốc                                           | Khi hai đường quan sát được xác định là cùng một điểm mốc thì hai tập quan sát phải gộp lại — đúng thao tác của DSU. Cùng cạm bẫy: nếu sau đó phát hiện gộp nhầm thì không tách ra được, nên trong thực tế người ta trì hoãn việc gộp cho tới khi đủ bằng chứng. |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai mẹo tấn công hai nguyên nhân khác nhau của chuỗi dài. Gộp theo kích thước ngăn không cho cây cao lên ngay từ đầu: một cây cao  $h$  buộc phải chứa ít nhất  $2^h$  nút, nên chiều cao bị chặn bởi  $\log_2 n$ . Nén đường thì làm phẳng những đường đã đi qua, nên chi phí đã trả một lần không phải trả lại. Kết hợp lại, mọi cấu hình xấu đều tự triệt tiêu sau vài thao tác, và phân tích chặt cho ra  $O(\alpha(n))$  khâu hao.
- A2. Vì nén đường đã phá huỷ thông tin về cấu trúc liên kết gốc: sau khi mọi nút trở thẳng lên gốc, không còn dấu vết của việc ai đã nối với ai. Đây là đánh đổi cố ý, và ba cách né là: xử lý ngược thời gian nếu bài chỉ có xoá; DSU có hoàn tác (bỏ nén đường, ghi nhật ký thao tác) kết hợp chia để trị trên trục thời gian nếu bài ngoại tuyến; hoặc dùng cấu trúc mạnh hơn như link-cut tree nếu buộc phải trực tuyến.
- A3. Mỗi khi một phần tử bị chuyển, nó đi từ tập nhỏ sang tập lớn, nên kích thước tập chứa nó ít nhất tăng gấp đôi. Kích thước không thể gấp đôi quá  $\log_2 n$  lần trước khi vượt  $n$ , nên mỗi phần tử bị chuyển tối đa  $\log_2 n$  lần, tổng  $O(n \log n)$ . Điều đáng chú ý về mặt phương pháp: ta không đếm chi phí theo từng thao tác gộp — có phép gộp rất đắt — mà đếm theo *tổng số lần các phần tử bị chạm*, đúng tinh thần phân tích khâu hao ở A/03.
- A4. Bằng cách lưu thêm một “khoảng cách” tương đối từ mỗi nút tới cha của nó, thường là bit chẵn lẻ. Khi đó “cùng phe” hay “khác phe” được suy ra từ tổng chẵn lẻ dọc đường lên gốc, và một ràng buộc mới mâu thuẫn với các ràng buộc cũ sẽ bị phát hiện ngay khi hai đầu đã cùng nhóm mà chẵn lẻ không khớp. Chỗ dễ sai nhất là phải cộng dồn trọng số trong lúc nén đường; quên bước đó thì cấu trúc vẫn chạy nhưng trả lời sai.
- A5. Vì DSU chỉ làm được một chiều — gộp — nên bài toán “xoá dần” là bài toán ngược của thứ nó làm được. Chạy các truy vấn theo thứ tự ngược thời gian biến mọi phép xoá thành phép thêm,

và lúc đó DSU dùng được nguyên vẹn. Cái giá là ta phải biết toàn bộ truy vấn từ trước, tức là chấp nhận lời giải ngoại tuyến — đây là cùng loại đánh đổi với thuật toán Mo ở B/10.

### NÓI LẠI THẬT ĐƠN GIẢN

Hãy hình dung một lớp học mà học sinh dần dần kết bạn. Bạn cần trả lời hai câu hỏi: “hai bạn này có chung một nhóm bạn không?” và “từ giờ hai nhóm này nhập làm một”.

Cách làm rất đơn giản: mỗi nhóm cử một bạn làm nhóm trưởng, và mỗi người chỉ cần nhớ mình theo ai. Muốn biết mình thuộc nhóm nào thì cứ hỏi ngược lên tới khi gặp người tự làm trưởng. Hai người cùng nhóm khi và chỉ khi họ ra cùng một trưởng.

Có hai mẹo làm việc này nhanh hẳn lên. Mẹo thứ nhất: khi nhập hai nhóm, luôn để nhóm đồng giữ trưởng và nhóm ít theo về — như vậy chuỗi hỏi ngược không bao giờ dài. Mẹo thứ hai: mỗi lần bạn phải hỏi ngược qua một chuỗi dài, hãy dặn tất cả những người trên đường “từ giờ cứ theo trưởng trưởng luôn” — lần sau ai hỏi cũng chỉ mất một bước. Hai mẹo tầm thường ấy làm chi phí gần như bằng hằng số, và điều đó phải mất mười năm mới được chứng minh chặt chẽ.

Nhưng có một điều DSU không làm được, và cần nhớ kỹ: *không tách nhóm ra được*. Sau khi mọi người đã theo trưởng trưởng, không ai còn nhớ ai đã kết bạn với ai lúc đầu, nên không có cách nào biết phải cắt ở đâu. Nếu bài toán của bạn có chuyện “huỷ kết bạn”, thì mẹo thường dùng là chạy ngược thời gian: xem cuốn phim từ cuối về đầu, lúc đó mọi việc huỷ trở thành việc kết.

**Một câu:** DSU chỉ hứa hai điều — cùng nhóm hay không, và gộp hai nhóm — và chính vì hứa ít nên nó gần như miễn phí.

**Chỗ không rút gọn được:** Chi phí  $O(\alpha(n))$  thực sự không phải hằng số về mặt toán học; nó chỉ là hằng số về mặt thực tiễn vì hàm đó tăng quá chậm.

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                    | Trả lời                                                                                                                                                   |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Hai thao tác của DSU               | “Tìm đại diện” và “gộp hai nhóm”. Không lưu danh sách thành viên, không trả lời được “nhóm gồm những ai”.                                                 |
| ••• Hai tối ưu và tác dụng             | Gộp theo kích thước (cây cao $h$ phải có $\geq 2^h$ nút $\Rightarrow$ chiều cao $\leq \log_2 n$ ); nén đường (làm phẳng đường vừa đi cho tương lai).      |
| ••• Chi phí khấu hao                   | $O(\alpha(n))$ với $\alpha$ là Ackermann ngược, thực tế $\leq 4$ ; và cận này đã được chứng minh là tối ưu.                                               |
| ••• Hạn chế cốt lõi                    | Không xoá, không tách — nén đường đã phá huỷ cấu trúc liên kết gốc. Đây là đánh đổi cố ý, không phải thiếu sót.                                           |
| ••• Ba cách né hạn chế đó              | Xử lý ngược thời gian (bài chỉ có xoá); DSU có hoàn tác + chia để trị trên trục thời gian (ngoại tuyến); link-cut tree (trực tuyến).                      |
| ••• Gộp nhỏ vào lớn                    | Luôn đổ tập nhỏ vào tập lớn; mỗi phần tử bị chuyển $\leq \log_2 n$ lần vì mỗi lần chuyển thì kích thước tập chứa nó ít nhất gấp đôi. Tổng $O(n \log n)$ . |
| •• DSU có trọng số / chặn lẻ           | Lưu quan hệ tương đối tới cha; trả lời “cùng phe hay khác phe”, phát hiện mâu thuẫn. Phải cộng dồn trọng số khi nén đường.                                |
| •• Mẹo “vị trí trống kế tiếp”          | Cho mỗi ô đã dùng trở sang ô trống kế tiếp; biến bài “tô đoạn, mỗi ô tô một lần” từ $O(n^2)$ thành gần tuyến tính.                                        |
| •• Nén đường và hoàn tác               | Không tương thích: muốn hoàn tác thì phải bỏ nén đường và chấp nhận $O(\log n)$ .                                                                         |
| •• DSU trên cây                        | Giữ cấu trúc đếm của con nặng nhất, gộp các con nhẹ vào; $O(n \log n)$ theo đúng lập luận gộp nhỏ vào lớn.                                                |
| •• Kiểm tra đồ thị hai phía trực tuyến | DSU chặn lẻ: mâu thuẫn xuất hiện khi hai đầu một cạnh đã cùng nhóm mà cùng chặn lẻ.                                                                       |
| • Gốc rễ                               | Ra đời 1964 cho câu lệnh EQUIVALENCE của Fortran; ngày nay vẫn chạy trong suy diễn kiểu của trình biên dịch.                                              |

| Thuật ngữ / câu hỏi                  | Trả lời (tiếp)                                                                                                                                                             |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●● “ $\alpha$ là tối ưu” nghĩa là gì | Tối ưu trong mô hình con trỏ (Tarjan, 1979) và vẫn $\Omega(\alpha)$ khấu hao ngay trong mô hình cell-probe (Fredman–Saks, 1989). Mọi phát biểu tối ưu đều kèm một mô hình. |

●●● phải thuộc ●● nên nắm ● biết là có

## I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

### 1. Chỉ dùng một trong hai tối ưu thì chi phí là bao nhiêu?

| Chỉ gộp theo kích thước cho  $O(\log n)$  trường hợp xấu nhất — đây là đảm bảo thật, không phải khấu hao, nên nó là lựa chọn khi cần hoàn tác. Chỉ nén đường cho  $O(\log n)$  khấu hao. Dùng cả hai cho  $O(\alpha(n))$  khấu hao. Trong thực hành, nếu không cần hoàn tác thì luôn dùng cả hai vì chúng chỉ tốn vài ký tự code.

### 2. Tôi có nên cài DSU bằng đệ quy không?

| Bản đệ quy cho hàm tìm đại diện rất ngắn và tự nhiên, nhưng có rủi ro tràn ngăn xếp nếu bạn chưa nén đường và cây sâu — tình huống này xảy ra trong các bài xây dựng đối nghịch. Bản lặp hai lượt (đi lên tìm gốc, đi lại lần nữa để nén) an toàn hơn và cũng nhanh hơn một chút. Với  $n$  lớn, nên dùng bản lặp; đây là ứng dụng cụ thể của lời khuyên ở E/25.

### 3. Làm sao dùng DSU để đếm số thành phần liên thông sau mỗi lần thêm cạnh?

| Khởi tạo số nhóm bằng  $n$ , và mỗi lần gộp thành công — tức hai đầu cạnh chưa cùng nhóm — thì giảm đi một. Cạnh nối hai đỉnh đã cùng nhóm không đổi gì, và đó cũng chính là dấu hiệu phát hiện chu trình, tức phần lõi của Kruskal ở C/15.

### 4. Khi nào tôi phải nén toạ độ trước khi dùng DSU?

| Khi các phần tử không phải số nguyên trong khoảng nhỏ. DSU cấp phát mảng theo số phần tử, nên với các khoá là số lớn, chuỗi hoặc cặp giá trị, bạn cần ánh xạ chúng về  $0..n-1$  trước bằng cách sắp xếp và loại trùng, hoặc bằng bảng băm. Đây là bước tiền xử lý giống hệt như với Fenwick ở B/10.

### 5. Bài toán “liên thông động” đầy đủ — vừa thêm vừa xoá, trực tuyến — giải thế nào?

| Đây là bài toán khó thật sự, không phải bài tập thông thường. Với cây (không có chu trình), link-cut tree cho  $O(\log n)$  khấu hao. Với đồ thị tổng quát, các cấu trúc liên thông động có chi phí *polylogarit* nhưng cài đặt rất nặng. Trong thực hành thi đấu, nếu bài cho phép ngoại tuyến thì hãy dùng chia để trị trên trục thời gian với DSU có hoàn tác — ngắn hơn nhiều lần và đủ nhanh.

### 6. Gộp nhỏ vào lớn khác gì DSU thông thường?

| Khác ở chỗ nó giữ *dữ liệu thật* của mỗi nhóm — một tập hợp, một bảng đếm, một danh sách — chứ không chỉ giữ đại diện. Vì vậy nó tốn  $O(n \log n)$  thay vì gần như tuyến tính, nhưng đổi lại trả lời được những câu mà DSU chịu, ví dụ “trong nhóm này có bao nhiêu giá trị phân biệt”. Quy tắc chọn: nếu chỉ cần biết cùng nhóm hay không thì DSU; nếu cần thống kê bên trong nhóm thì gộp nhỏ vào lớn.

### 7. Có bài nào trông không liên quan gì tới “nhóm” mà vẫn dùng DSU không?

| Có, và đó là dấu hiệu bạn đã thật sự nắm cấu trúc này. Mẹo “vị trí trống kế tiếp” ở mục 3 là ví dụ rõ nhất: bài toán nói về việc tô màu các ô, không nói gì về nhóm, nhưng quan hệ “ô này đã đầy, hãy hỏi ô bên phải” lại có đúng cấu trúc hợp nhất một chiều. Một ví dụ khác là các bài ràng buộc bằng nhau kiểu “ $x_i = x_j$ ”, nơi DSU dùng để phát hiện mâu thuẫn với các ràng buộc khác nhau.

## I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                                                       | Nguồn                           | Điều cần nhận ra                                                                  |
|---------------------------------------------------------------------------|---------------------------------|-----------------------------------------------------------------------------------|
| • Đếm số thành phần liên thông khi thêm cạnh dần                          | CSES “Road Construction”[1]     | DSU cơ bản kèm đếm kích thước nhóm lớn nhất.                                      |
| • Kiểm tra đồ thị có chu trình khi thêm cạnh                              | Bài kinh điển                   | Cạnh nối hai đỉnh đã cùng nhóm chính là cạnh tạo chu trình.                       |
| •• Cây khung nhỏ nhất bằng Kruskal                                        | CSES “Road Reparation”[1]       | Sắp cạnh rồi DSU; chứng minh tính đúng ở $C/15$ .                                 |
| •• Kiểm tra đồ thị hai phía khi thêm cạnh dần                             | Bài kinh điển                   | DSU chắn lẻ; chú ý cộng dồn trọng số lúc nén đường.                               |
| •• Tô màu các đoạn, mỗi ô chỉ tô một lần                                  | Bài thi đấu kinh điển           | Mẹo “vị trí trống kế tiếp”; so với cây phân đoạn dễ thấy DSU ngắn hơn nhiều.      |
| •• Số đảo trên lưới sau mỗi lần thêm đất                                  | LeetCode “Number of Islands II” | DSU trên lưới; ánh xạ tọa độ hai chiều về chỉ số một chiều.                       |
| ••• Liên thông khi các cạnh bị xoá dần                                    | Bài kinh điển                   | Đảo ngược thời gian; đây là bài mẫu để hiểu mẹo đó.                               |
| ••• Liên thông động ngoại tuyến với cạnh có khoảng thời gian tồn tại      | Bài thi đấu kinh điển           | DSU có hoàn tác + cây phân đoạn trên trục thời gian; kết hợp B/10 với chương này. |
| ••• Với mỗi cây con, đếm số màu phân biệt                                 | Bài kinh điển                   | DSU trên cây (gộp nhỏ vào lớn); tự chứng minh cận $O(n \log n)$ trước khi cài.    |
| ••• Ràng buộc “ $x_i$ và $x_j$ bằng nhau / khác nhau”, kiểm tra mâu thuẫn | Bài kinh điển                   | DSU chắn lẻ tổng quát; liên hệ với hợp nhất trong suy diễn kiểu ở mục 5.          |
| ••• Mô phỏng thăm trên lưới $n \times n$ , tìm ngưỡng thăm                | Bài tập thực nghiệm             | Ứng dụng vật lý cổ điển; thêm hai nút ảo cho hai bờ là mẹo cài đặt cần thiết.     |

## Kết chương và đọc tiếp

DSU là ví dụ rõ nhất trong quyển sách này cho luận điểm “hứa ít thì rẻ”: hai thao tác, một mảng số nguyên, và chi phí gần như hằng số với một chứng minh mà phải mất mười năm mới có. Hai điều đáng mang theo: lập luận gộp nhỏ vào lớn — mỗi phần tử chỉ bị chuyển  $\log n$  lần — vì nó dùng lại ở nhiều nơi ngoài DSU; và ý thức về hạn chế một chiều, cùng ba cách né nó.

Chương cuối của phần B chuyển sang một miền dữ liệu riêng, nơi các cấu trúc chuyên biệt tạo ra khác biệt lớn nhất: chuỗi ký tự. Bạn sẽ thấy cùng một ý tưởng “đừng tính lại thứ đã biết” của chương B/06 xuất hiện lại dưới dạng hàm tiền tố của KMP, và sẽ thấy vì sao một cấu trúc như suffix array biến cả một họ bài chuỗi thành bài mảng thông thường (B/12).

**Đọc thêm: công trình gốc** (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

| Mức         | Công trình và tác giả                                                                  | Nơi công bố, năm                              | Vì sao đọc, đọc phần nào                                                                                                              |
|-------------|----------------------------------------------------------------------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| nền tảng    | Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm”                      | Journal of the ACM, 1975                      | Bài chứng minh cận $O(\alpha)$ . Đọc phần phát biểu kết quả và định nghĩa $\alpha$ ; phần chứng minh dài và có thể để sau.            |
| nền tảng    | Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets” | Journal of Computer and System Sciences, 1979 | Cận dưới trong mô hình con trỏ — tài liệu cho thấy “tối ưu” ở đây nghĩa là gì.                                                        |
| tra khi cần | Fredman & Saks, “The Cell Probe Complexity of Dynamic Data Structures”                 | STOC, 1989                                    | Cận dưới $\Omega(\alpha)$ trong mô hình cell-probe. Đọc phần mở đầu; đây cũng là cửa vào cho các cận dưới cấu trúc dữ liệu nói chung. |

| Mức         | Công trình và tác giả                                                                                 | Nơi công bố, năm | Vì sao đọc (tiếp)                                                                 |
|-------------|-------------------------------------------------------------------------------------------------------|------------------|-----------------------------------------------------------------------------------|
| tra khi cần | Cormen, Leiserson, Rivest, Stein, <i>Introduction to Algorithms</i> (4th ed.), chương về tập rời nhau | MIT Press, 2022  | Tra khi cần một chứng minh đầy đủ, viết theo chuẩn giáo trình, cho cận $\alpha$ . |

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [4] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [5] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [6] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [7] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [8] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [9] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [10] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [11] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] AtCoder. “AC Library: Lazy Segtree.” (), address: [https://atcoder.github.io/ac-library/master/document\\_en/lazysegtree.html](https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html) (visited on 09/07/2026).
- [13] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [14] R. E. Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets,” *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [15] M. L. Fredman and M. E. Saks, “The Cell Probe Complexity of Dynamic Data Structures,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [16] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [17] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [18] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-line String Searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

## Chuỗi và so khớp (Strings and Pattern Matching)

**PHỤ THUỘC** B/06 cho ý tưởng không tính lại; B/08 cho băm và nghịch lý ngày sinh; B/09 cho cây. Đây là chương cuối của phần B.

### TÓM TẮT

Chuỗi là mảng, nhưng có một tính chất riêng làm nó đáng có chương riêng: các đoạn con của nó *chồng lấn* nhau, và sự chồng lấn ấy vừa là nguồn của mọi khó khăn vừa là nguồn của mọi lời giải nhanh. Chương đi từ bài toán so khớp một mẫu — KMP và hàm Z, cả hai đều dựa trên ý “thất bại ở đây cho ta thông tin, đừng vứt đi” — tới nhiều mẫu cùng lúc với Aho–Corasick, tới các cấu trúc nhìn được toàn bộ chuỗi con là trie và mảng hậu tố. Băm chuỗi được đặt cạnh chúng như một lựa chọn ngắn và mạnh với cái giá xác suất. Nếu chỉ nhớ một điều: mọi thuật toán chuỗi nhanh đều trả lời cùng một câu hỏi — *khi so khớp thất bại tại vị trí  $i$ , tôi đã biết được gì mà không phải quay lại từ đầu?*

### KIẾN THỨC CẦN CÓ TRƯỚC

- **tiền tố, hậu tố** — đoạn đầu và đoạn cuối của một chuỗi. Chuỗi abc có tiền tố a, ab, abc.
- **biên (border)** — một chuỗi vừa là tiền tố vừa là hậu tố thực sự của chuỗi đã cho; ví dụ aba có biên a. Khái niệm trung tâm của KMP.
- **hàm tiền tố  $\pi$**  —  $\pi[i]$  là độ dài biên dài nhất của tiền tố kết thúc tại  $i$ . Đây là toàn bộ nội dung “tiền xử lý” của KMP.
- **hàm Z** —  $z[i]$  là độ dài tiền tố chung dài nhất giữa chuỗi và hậu tố bắt đầu tại  $i$ . Cùng sức mạnh với  $\pi$ , thường dễ nghĩ hơn.
- **trie** — cây trong đó mỗi cạnh mang một ký tự, nên mỗi đường từ gốc biểu diễn một tiền tố; dùng để lưu một tập chuỗi và tra theo tiền tố.
- **mảng hậu tố (suffix array)** — mảng các vị trí hậu tố, sắp theo thứ tự từ điển của các hậu tố; cùng với mảng LCP, nó biến nhiều bài chuỗi thành bài mảng thông thường.

### CÂU HỎI MỞ ĐẦU

- Q1. Thuật toán so khớp ngây thơ chạy  $O(nm)$ . KMP chỉ dùng thêm một mảng số nguyên mà xuống  $O(n + m)$ . Thông tin nào đã bị lãng phí trong cách ngây thơ?
- Q2. Nếu băm chuỗi làm được gần hết những gì KMP làm và code ngắn hơn nhiều, vì sao vẫn phải học KMP?
- Q3. Vì sao mảng hậu tố lại được coi là “giải quyết một họ bài” chứ không chỉ là một cấu trúc?
- Q4. Aho–Corasick tìm đồng thời hàng nghìn mẫu trong một lượt quét. Nó khác gì so với chạy KMP nhiều lần, và cái gì làm cho khác biệt đó có thể xảy ra?
- Q5. Bài toán tìm chuỗi con đối xứng dài nhất có lời giải  $O(n)$ . Ý tưởng nào cho phép bỏ qua phần lớn công việc?

Chuỗi ký tự là mảng, và mọi kỹ thuật ở chương B/06 đều dùng được. Nhưng có một điều làm chuỗi khác hẳn: một chuỗi độ dài  $n$  có  $n(n+1)/2$  đoạn con, và các đoạn ấy chồng lên nhau chằng chịt. Điều này vừa là tin xấu — không thể liệt kê hết — vừa là tin tốt, vì sự chồng lấn nghĩa là có

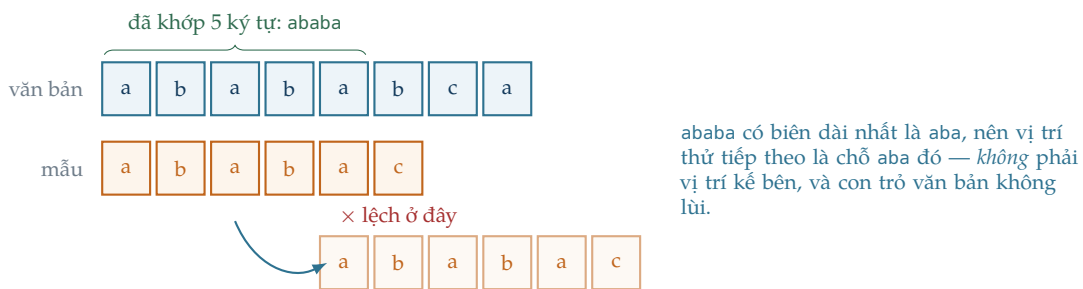
rất nhiều thông tin lặp lại, và mọi thuật toán chuỗi nhanh đều sống nhờ việc khai thác chỗ lặp lại đó.

Bài toán mở đầu là bài toán so khớp: tìm mọi vị trí xuất hiện của mẫu  $P$  độ dài  $m$  trong văn bản  $T$  độ dài  $n$ . Cách ngây thơ thử mọi vị trí bắt đầu và so từng ký tự, cho  $O(nm)$ . Cái lãng phí nằm ở chỗ: khi so khớp thất bại sau khi đã khớp được  $k$  ký tự, ta *đã biết*  $k$  ký tự đó của văn bản là gì — nhưng cách ngây thơ vứt bỏ thông tin ấy và bắt đầu lại từ vị trí kế tiếp. Toàn bộ chương này là các cách khác nhau để không vứt.

## 7.1 KMP: dùng lại thông tin của lần thất bại

Giả sử ta đang so mẫu  $P$  với văn bản và đã khớp được  $k$  ký tự thì gặp ký tự không khớp. Ta biết rằng  $k$  ký tự vừa qua của văn bản đúng bằng  $P[1..k]$ . Câu hỏi đúng là: *vị trí bắt đầu tiếp theo nào còn có cơ hội?* Một vị trí như vậy chỉ khả dĩ nếu phần mẫu tính từ đó khớp với đuôi của  $P[1..k]$  — nghĩa là ta cần một chuỗi vừa là tiền tố vừa là hậu tố của  $P[1..k]$ . Đó chính là khái niệm *biên*.

Vậy toàn bộ việc tiền xử lý của KMP là tính hàm  $\pi$ : với mỗi tiền tố của mẫu, độ dài biên dài nhất của nó. Khi thất bại sau  $k$  ký tự khớp, ta không lùi con trỏ văn bản chút nào; ta chỉ nhảy con trỏ mẫu về  $\pi[k]$  và so tiếp.



**Hình 7.1.** Ý tưởng của KMP. Khi lệch sau 5 ký tự khớp, ta biết văn bản vừa qua là ababa. Vị trí bắt đầu tiếp theo có cơ hội phải là chỗ mà mẫu khớp với một hậu tố của ababa đồng thời là tiền tố của mẫu — tức một biên. Biên dài nhất là aba, nên ta trượt mẫu đúng hai bước, và không ký tự nào của văn bản bị đọc lại.

Chi phí  $O(n + m)$ , và lập luận lại chính là lập luận khấu hao quen thuộc: con trỏ văn bản chỉ tiến, còn con trỏ mẫu tăng tối đa  $n$  lần tổng cộng nên cũng chỉ giảm tối đa  $n$  lần. Cùng dạng lập luận với hai con trỏ (B/06) và ngăn xếp đơn điệu (B/07) — đây là lần thứ ba nó xuất hiện, và đó không phải trùng hợp.

Hàm  $Z$  là công cụ tương đương về sức mạnh nhưng thường dễ nghĩ hơn:  $z[i]$  là độ dài đoạn khớp dài nhất giữa chuỗi và hậu tố bắt đầu tại  $i$ . Tính được trong  $O(n)$  bằng cách duy trì đoạn khớp phải nhất đã biết và tái sử dụng nó. Mẹo dùng chung cho cả hai: để tìm  $P$  trong  $T$ , hãy tính hàm trên chuỗi ghép  $P + \# + T$  với  $\#$  là ký tự không xuất hiện ở đâu cả; mọi vị trí có giá trị bằng  $m$  chính là một lần xuất hiện.

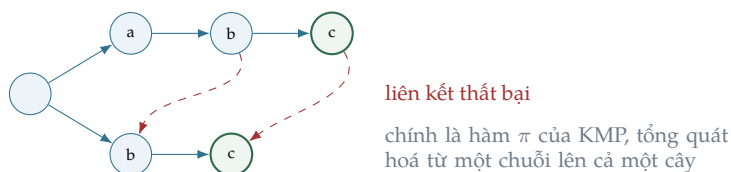
Hàm  $\pi$  còn cho thêm hai kết quả đáng nhớ, và chúng thường là mục đích thật sự của bài chứ không phải việc so khớp: *chu kỳ nhỏ nhất* của chuỗi bằng  $n - \pi[n]$  khi  $n$  chia hết cho hiệu đó, và *mọi biên* của chuỗi tìm được bằng cách lặp  $\pi[n], \pi[\pi[n]], \dots$

## 7.2 Aho–Corasick: KMP cho nhiều mẫu cùng lúc

Nếu có  $k$  mẫu, chạy KMP  $k$  lần cho  $O(k \cdot n)$ . Aho–Corasick làm trong một lượt quét văn bản.

Ý tưởng ghép hai thứ đã biết: xếp mọi mẫu vào một *trie*, rồi thêm vào mỗi nút một “liên kết thất bại” trở tới nút ứng với hậu tố dài nhất của chuỗi hiện tại mà cũng là một tiền tố của mẫu nào đó. Nói cách khác: liên kết thất bại là hàm  $\pi$  của KMP, tổng quát hoá từ một chuỗi lên cả một cây. Khi quét văn bản, ta đi trên *trie*; gặp ký tự không có cạnh thì theo liên kết thất bại, hết như KMP nhảy về  $\pi[k]$ .

Kết quả là  $O(n + \text{tổng độ dài mẫu} + \text{số lần khớp})$ . Đây là thuật toán chạy phía sau các công cụ tìm nhiều từ khoá trong một lượt — lệnh tìm nhiều mẫu của Unix, các hệ phát hiện xâm nhập quét gói tin theo hàng nghìn chữ ký, các bộ lọc nội dung.



**Hình 7.2.** Automaton Aho–Corasick cho tập mẫu  $\{abc, bc\}$ . Các mẫu chia sẻ tiền tố nên đi chung nhánh; liên kết thất bại (nét đứt) cho phép chuyển trạng thái khi lệch mà không đọc lại văn bản. Nhờ vậy chi phí quét không phụ thuộc số mẫu.

### 7.3 Băm chuỗi: lựa chọn ngắn với cái giá xác suất

Băm đa thức (B/08) cho phép so sánh hai đoạn con bất kỳ trong  $O(1)$  sau tiền xử lý  $O(n)$ . Với công cụ đó, rất nhiều bài chuỗi sụp xuống thành dễ: tìm mẫu, tìm chuỗi con lặp lại dài nhất (kết hợp nhị phân trên độ dài, C/19), so sánh hai đoạn, đếm chuỗi con phân biệt.

Ba điều phải nhớ mỗi lần dùng, đã nói ở chương B/08 nhưng đáng nhắc lại vì đây là nơi chúng gây hại nhiều nhất: chọn cơ số *ngẫu nhiên lúc chạy*; dùng hai modulo khi số chuỗi cần so lớn hơn cỡ  $3 \cdot 10^4$ ; và nhớ rằng đây là thuật toán xác suất.

Vậy vì sao vẫn học KMP? Ba lý do. Thứ nhất, KMP cho kết quả *chắc chắn đúng*. Thứ hai, hàm  $\pi$  cho thông tin về cấu trúc chuỗi — biên, chu kỳ — mà băm không cho. Thứ ba, và quan trọng nhất về mặt học tập: ý tưởng của KMP — dùng lại thông tin của lần thất bại — là ý tưởng tổng quát, và bạn sẽ gặp lại nó ở Aho–Corasick, ở các thuật toán tự động hoá, và ở nhiều nơi ngoài chuỗi.

### 7.4 Nhìn được mọi chuỗi con: trie, mảng hậu tố, automaton

Các công cụ trên trả lời câu hỏi về *một mẫu cho trước*. Có một lớp câu hỏi khác đòi hỏi nhiều hơn: “chuỗi con lặp lại dài nhất là gì”, “có bao nhiêu chuỗi con phân biệt”, “hai chuỗi có đoạn chung dài nhất là bao nhiêu”. Những câu này cần một cấu trúc *nhìn được toàn bộ tập chuỗi con* cùng lúc.

*Trie* là bước đầu: lưu một tập chuỗi thành cây theo ký tự. Nó cho tra cứu theo tiền tố, tự động hoàn thành, và là nền của Aho–Corasick. Nhược điểm: bộ nhớ lớn nếu bảng chữ cái rộng.

*Mảng hậu tố* là công cụ chủ lực. Ý tưởng: lấy toàn bộ  $n$  hậu tố của chuỗi và sắp chúng theo thứ tự từ điển; mảng hậu tố lưu vị trí bắt đầu của chúng theo thứ tự đó. Kèm theo là mảng LCP, ghi độ dài tiền tố chung của hai hậu tố liền nhau trong thứ tự đã sắp. Xây được trong  $O(n \log n)$  hoặc  $O(n)$ .

Vì sao nó “giải quyết một họ bài”? Vì sau khi có mảng hậu tố và LCP, các câu hỏi về chuỗi con trở thành các câu hỏi về *đoạn trên một mảng số* — thứ mà cả phần B trước đó đã trang bị đầy đủ công cụ. Chuỗi con lặp lại dài nhất chính là giá trị lớn nhất trong mảng LCP. Số chuỗi con phân biệt là tổng độ dài các hậu tố trừ tổng LCP. Chuỗi con chung dài nhất của hai chuỗi giải bằng cách nối

hai chuỗi và tìm cực đại LCP giữa hai hậu tố khác nguồn gốc. Nghĩa là mảng hậu tố đóng vai trò một *phép biến đổi*: nó chuyển bài toán từ miền chuỗi sang miền mảng, nơi ta đã mạnh.

*Suffix automaton* và *suffix tree* là hai cấu trúc mạnh hơn, biểu diễn được mọi chuỗi con trong  $O(n)$  trạng thái và cho nhiều bài lời giải tuyến tính đẹp. Chúng cũng khó cài và khó nhớ hơn hẳn. Lời khuyên thực dụng: nắm chắc mảng hậu tố trước; *suffix automaton* xếp vào loại tham khảo, học khi gặp bài thật cần.

## 7.5 Manacher và một mẫu tư duy đáng nhớ

Bài tìm chuỗi con đối xứng dài nhất có lời giải  $O(n)$  của Manacher, và ý tưởng của nó là một mẫu dùng lại được: *tái sử dụng đối xứng đã biết*. Khi ta đã biết một chuỗi đối xứng lớn quanh tâm  $c$ , thì với một vị trí  $i$  nằm trong nó, ta biết trước phần lớn thông tin về bán kính đối xứng tại  $i$  — vì nó phải giống với bán kính tại vị trí đối xứng của  $i$  qua  $c$ , ít nhất cho tới khi chạm biên.

Mẫu này giống hệt cách tính hàm Z (tái sử dụng đoạn khớp phải nhất) và giống cách Aho–Corasick tái dùng liên kết thất bại. Nếu bạn muốn nhớ một điều duy nhất về nhóm thuật toán chuỗi tuyến tính, hãy nhớ: *luôn duy trì cấu trúc phải nhất đã biết, và dùng nó để bắt đầu chỗ mới không phải từ số không*.

| Công cụ           | Xây             | Trả lời                  | Chọn khi                               |
|-------------------|-----------------|--------------------------|----------------------------------------|
| Băm chuỗi         | $O(n)$          | so sánh đoạn $O(1)$      | Cần nhanh và ngắn; chấp nhận xác suất  |
| KMP / hàm Z       | $O(n + m)$      | so khớp một mẫu          | Cần chắc chắn đúng; cần biên và chu kỳ |
| Aho–Corasick      | $O(\sum  P_i )$ | nhiều mẫu, một lượt      | Hàng nghìn mẫu cố định                 |
| Trie              | $O(\sum  S_i )$ | tra theo tiền tố         | Tự động hoàn thành, từ điển            |
| Mảng hậu tố + LCP | $O(n \log n)$   | mọi câu hỏi về chuỗi con | Bài về chuỗi con lặp/chung/phân biệt   |
| Suffix automaton  | $O(n)$          | mọi chuỗi con, đếm       | Khi mảng hậu tố không đủ; tham khảo    |

## 7.6 Gốc rễ: từ tìm kiếm thư mục tới giải mã bộ gen

Bài toán so khớp chuỗi có một lịch sử gắn liền với những nhu cầu rất thực tế của từng thời kỳ, và mỗi thời kỳ lại sinh ra một cấu trúc.

Thuật toán KMP được công bố năm 1977[16], nhưng câu chuyện đằng sau đáng nhớ: Morris và Pratt tìm ra thuật toán trước, còn Knuth đến với nó từ hướng lý thuyết — ông đang nghiên cứu một kết quả về automaton hai chiều và nhận ra nó kéo theo một thuật toán so khớp tuyến tính. Ba người ghép công trình lại. Điều đáng học ở đây là hai con đường rất khác nhau — kỹ thuật và lý thuyết — dẫn tới cùng một thuật toán, và bài báo chung mạnh hơn cả hai.

Aho và Corasick công bố thuật toán của họ năm 1975[17] với một bài toán rất cụ thể trong tiêu đề: tra cứu thư mục thư viện. Nhu cầu là quét một khối văn bản để tìm hàng nghìn từ khoá cùng lúc — và cấu trúc họ tạo ra sau đó trở thành công cụ chuẩn cho việc quét mẫu trong các hệ thống an ninh mạng, nơi mỗi gói tin phải được đối chiếu với hàng nghìn chữ ký tấn công.

Nhánh cấu trúc hậu tố có một câu chuyện về đánh đổi bộ nhớ rất đáng kể. Cây hậu tố được Weiner đưa ra năm 1973 và cho lời giải tuyến tính cho rất nhiều bài, nhưng nó tốn bộ nhớ lớn — hàng số cao và phụ thuộc bảng chữ cái. Năm 1990, Manber và Myers[18] đề xuất mảng hậu tố

với động cơ rõ ràng là *tiết kiệm bộ nhớ*: thay cây bằng một mảng số nguyên. Đây là ví dụ mẫu mực cho việc một cấu trúc “yếu hơn” về lý thuyết lại thắng trong thực tế nhờ hằng số.

Và cú hích lớn nhất cho cả nhánh này đến từ ngoài ngành: dự án giải mã bộ gen người. Việc căn chỉnh hàng tỉ đoạn đọc DNA lên một bộ gen tham chiếu là bài toán so khớp chuỗi ở quy mô chưa từng có, với ràng buộc bộ nhớ khắc nghiệt. Điều đó thúc đẩy các chỉ mục nén dựa trên phép biến đổi Burrows–Wheeler, cho phép tìm kiếm trên bộ gen ba tỉ ký tự trong vài gigabyte bộ nhớ — và các công cụ căn chỉnh phổ biến trong tin sinh học ngày nay đều dựa trên đó.

### I Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                            | Bài toán thúc ép lúc đó                                                   | Điều thay đổi sau đó                                                                                                               |
|--------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 1970–77 — Morris, Pratt, Knuth | So khớp một mẫu trong văn bản mà không lùi con trỏ văn bản.               | KMP; hai con đường (kỹ thuật và lý thuyết automaton) gặp nhau ở cùng một thuật toán.                                               |
| 1975 — Aho & Corasick          | Tra cứu thư mục thư viện: tìm hàng nghìn từ khoá trong một lượt quét.     | Trie + liên kết thất bại; nay là lõi của quét chữ ký trong hệ thống an ninh mạng.                                                  |
| 1973 — Weiner                  | Cần trả lời mọi câu hỏi về chuỗi con trong thời gian tuyến tính.          | Cây hậu tố: mạnh nhưng tốn bộ nhớ, hằng số cao, phụ thuộc bảng chữ cái.                                                            |
| 1990 — Manber & Myers          | Cây hậu tố quá tốn bộ nhớ cho dữ liệu lớn thời đó.                        | Mảng hậu tố: đổi cấu trúc cây lấy một mảng số nguyên; ví dụ điển hình cho việc hằng số thắng lý thuyết.                            |
| 1994 — Burrows & Wheeler       | Nén văn bản tốt hơn bằng cách sắp xếp lại ký tự cho dễ nén.               | Phép biến đổi BWT; sau này thành nền cho chỉ mục nén tìm kiếm được.                                                                |
| Thập niên 2000 — tin sinh học  | Căn chỉnh hàng tỉ đoạn đọc DNA lên bộ gen ba tỉ ký tự với bộ nhớ hạn chế. | Chỉ mục FM dựa trên BWT; các công cụ căn chỉnh chuẩn của ngành đều dùng, đưa thuật toán chuỗi vào ứng dụng quy mô lớn nhất của nó. |

## I 7.7 Liên hệ ngang

### I Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực        | Ý tưởng tương ứng                                         | Giống ở đâu, khác ở đâu                                                                                                                                                                                   |
|-----------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tin sinh học    | Căn chỉnh đoạn đọc DNA, tìm mẫu trong bộ gen              | Cùng bài toán, khác quy mô và khác ràng buộc: bảng chữ cái chỉ bốn ký tự, nhưng chuỗi dài ba tỉ và phải chịu được sai khác. Đây là lý do ngành này đẩy mạnh chỉ mục nén, thứ mà lập trình thi đấu ít gặp. |
| Nén dữ liệu     | LZ77, BWT                                                 | Nén tìm các đoạn lặp lại — đúng thông tin mà mảng hậu tố và LCP mô tả. Liên hệ hai chiều: cấu trúc chuỗi giúp nén, và phép biến đổi phục vụ nén (BWT) quay lại thành chỉ mục tìm kiếm.                    |
| An ninh mạng    | Hệ phát hiện xâm nhập quét gói tin theo hàng nghìn chữ ký | Aho–Corasick nguyên bản. Ràng buộc thêm so với bài tập: phải chạy ở tốc độ đường truyền, nên bộ nhớ của automaton và tính thân thiện cache là yếu tố quyết định.                                          |
| Trình biên dịch | Phân tích từ vựng bằng automaton hữu hạn                  | Aho–Corasick là một automaton nhận biết tập mẫu; bộ phân tích từ vựng cũng là automaton. Cùng ý tưởng: biên dịch tập mẫu thành một máy trạng thái rồi quét văn bản một lượt.                              |

| Lĩnh vực                | Ý tưởng tương ứng                                       | Giống ở đâu, khác ở đâu (tiếp)                                                                                                                                                                               |
|-------------------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Công cụ tìm kiếm        | Chỉ mục ngược, gợi ý tự động, sửa lỗi chính tả          | Trie cho gợi ý theo tiền tố; khoảng cách chỉnh sửa (C/16) cho sửa lỗi. Khác quy mô: chỉ mục thật phân tán và nén, nhưng ý tưởng gốc vẫn là các cấu trúc ở đây.                                               |
| Xử lý ngôn ngữ tự nhiên | Tách từ con (subword tokenization) bằng từ điển và trie | Bộ tách từ của mô hình ngôn ngữ dùng cấu trúc kiểu trie để khớp tham lam đoạn dài nhất. Cùng cấu trúc, khác mục tiêu: ở đây là biểu diễn đầu vào chứ không phải tìm kiếm.                                    |
| Thị giác máy tính, SLAM | So khớp mô tả nhị phân, phát hiện lặp lại chuỗi ảnh     | Không dùng KMP, nhưng chung tinh thần: chuyển bài so khớp về một biểu diễn cho phép so sánh $O(1)$ (băm, mã nhị phân) rồi kiểm chứng lại bằng bước đắt hơn. Kiến trúc “lọc rẻ rồi kiểm đắt” giống hệt mục 3. |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Thông tin về *phần đã khớp*. Khi lệch sau  $k$  ký tự khớp, ta biết chính xác  $k$  ký tự vừa qua của văn bản bằng  $P[1..k]$ ; cách này vớt bỏ điều đó và đọc lại từ vị trí kế tiếp. KMP dùng nó để xác định vị trí thử tiếp theo duy nhất còn cơ hội — vị trí ứng với biên dài nhất của  $P[1..k]$  — nên con trỏ văn bản không bao giờ lùi. Đây là lần thứ ba trong phần B xuất hiện lập luận “mỗi chỉ số chỉ đi một chiều”.
- A2. Ba lý do. Một, KMP đúng đắn chắc chắn, còn băm có xác suất sai — nhỏ nhưng khác không, và nó tăng theo bình phương số phép so sánh. Hai, hàm  $\pi$  cho thông tin cấu trúc mà băm không cho: mọi biên, chu kỳ nhỏ nhất — và nhiều bài thật ra hỏi đúng những thứ này chứ không hỏi việc so khớp. Ba, ý tưởng “dùng lại thông tin của lần thất bại” là ý tưởng tổng quát, quay lại ở Aho–Corasick và ở các thuật toán tự động hoá khác.
- A3. Vì nó là một *phép biến đổi* miền bài toán: sau khi có mảng hậu tố và mảng LCP, các câu hỏi về chuỗi con trở thành câu hỏi về đoạn trên một mảng số nguyên — miền mà cả phần B đã trang bị đầy đủ công cụ. Chuỗi con lặp lại dài nhất thành cực đại của LCP; số chuỗi con phân biệt thành một hiệu của hai tổng; chuỗi chung dài nhất thành truy vấn trên đoạn. Một cấu trúc mở khoá cả một họ bài chính là vì lý do đó.
- A4. Khác ở chỗ nó *biên dịch toàn bộ tập mẫu thành một máy trạng thái* trước, rồi quét văn bản đúng một lần với chi phí không phụ thuộc số mẫu. Điều làm việc đó khả thi là các mẫu chia sẻ tiền tố với nhau — trie khai thác đúng phần chia sẻ ấy — và liên kết thất bại cho phép chuyển trạng thái khi lệch mà không cần quay lui. Chạy KMP  $k$  lần thì mỗi lần đọc lại toàn bộ văn bản, tức là bỏ qua chính phần chia sẻ đó.
- A5. Tái sử dụng đối xứng đã biết. Nếu đã biết một chuỗi đối xứng lớn quanh tâm  $c$ , thì với vị trí  $i$  nằm bên trong nó, bán kính đối xứng tại  $i$  ít nhất bằng bán kính tại điểm đối xứng của  $i$  qua  $c$ , cho tới khi chạm biên của chuỗi lớn. Nhờ đó phần lớn vị trí được khởi tạo sẵn thay vì bắt đầu từ số không, và tổng chi phí mở rộng là tuyến tính. Cùng mẫu với cách tính hàm Z: luôn duy trì cấu trúc phải nhất đã biết.

### NÓI LẠI THẬT ĐƠN GIẢN

Giả sử bạn đang dò xem từ “ababac” có xuất hiện trong một trang sách hay không. Cách vụng là đặt từ đó lên đầu dòng, so từng chữ; hễ sai một chữ thì dịch sang phải một ô rồi so lại từ đầu. Cách này phí ở chỗ: khi bạn đã so đúng năm chữ rồi mới sai, bạn *đã biết* năm chữ đó của trang sách là gì — vậy mà bạn vứt hết và dò lại.

KMP chỉ làm một việc: trước khi dò, nó đọc kỹ chính cái từ cần tìm và ghi lại một bảng nhỏ trả lời câu “nếu tôi khớp được  $k$  chữ rồi sai, thì phải trượt sang bao nhiêu ô là hợp lý”. Câu trả lời phụ thuộc vào chỗ từ đó tự lặp lại chính nó ở đầu và cuối — ví dụ “ababa” có phần “aba” vừa là đầu vừa là đuôi, nên trượt hai ô là đủ, không cần trượt một ô. Nhờ bảng đó, mắt bạn không bao giờ phải đọc lại một chữ nào của trang sách.

Nếu cần tìm hàng nghìn từ cùng lúc thì đừng dò từng từ một. Hãy xếp tất cả các từ vào một cái cây theo chữ cái đầu — những từ có chung phần đầu thì đi chung một nhánh — rồi đọc trang sách một lượt và đi trên cây đó. Đó là Aho–Corasick.

Cuối cùng, có một cách nhìn khác cho các câu hỏi kiểu “đoạn nào lặp lại nhiều nhất”: hãy liệt kê mọi phần đuôi của chuỗi rồi xếp chúng theo thứ tự từ điển, như xếp từ trong từ điển. Khi đó những đoạn giống nhau tự nằm cạnh nhau, và câu hỏi “đoạn lặp dài nhất” chỉ còn là “hai hàng xóm nào giống nhau nhiều chữ nhất”. Đó là mảng hậu tố, và giá trị của nó là biến một bài về chuỗi thành một bài về dãy số thông thường.

**Một câu:** Mọi thuật toán chuỗi nhanh đều trả lời cùng một câu: khi thất bại, tôi đã biết được gì để không phải làm lại từ đầu?

**Chỗ không rút gọn được:** Cấu trúc hậu tố (mảng hậu tố, automaton) không có cách giải thích nào thật sự ngắn gọn; phải cài một lần và chạy tay trên ví dụ nhỏ mới thấy được.

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                             | Trả lời                                                                                                                                                   |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Ý chung của mọi thuật toán chuỗi tuyến tính | Khi thất bại tại vị trí $i$ , khai thác thông tin đã biết thay vì bắt đầu lại; duy trì “cấu trúc phải nhất đã biết” và dùng nó khởi tạo cho vị trí mới.   |
| ••• Biên và hàm $\pi$                           | Biên là chuỗi vừa là tiền tố vừa là hậu tố thực sự; $\pi[i]$ là biên dài nhất của tiền tố kết thúc tại $i$ . Toàn bộ tiền xử lý của KMP.                  |
| ••• Vì sao KMP là $O(n + m)$                    | Con trỏ văn bản chỉ tiến; con trỏ mẫu tăng tối đa $n$ lần nên giảm tối đa $n$ lần. Cùng lập luận với hai con trỏ và ngăn xếp đơn điệu.                    |
| ••• Chu kỳ nhỏ nhất từ $\pi$                    | Bằng $n - \pi[n]$ khi $n$ chia hết cho hiệu đó; mọi biên tìm bằng cách lặp $\pi[n], \pi[\pi[n]], \dots$                                                   |
| ••• Mẹo dùng chung của KMP và hàm Z             | Tính hàm trên chuỗi ghép $P + \# + T$ ; vị trí có giá trị bằng $m$ là một lần xuất hiện.                                                                  |
| ••• Mảng hậu tố giải quyết gì                   | Biến câu hỏi về chuỗi con thành câu hỏi về đoạn trên mảng số: chuỗi lặp dài nhất = $\max \text{LCP}$ ; số chuỗi con phân biệt = tổng độ dài trừ tổng LCP. |
| •• Aho–Corasick                                 | Trie của mọi mẫu + liên kết thất bại (chính là $\pi$ tổng quát hoá lên cây); quét văn bản một lượt, chi phí không phụ thuộc số mẫu.                       |
| •• Ba lưu ý khi băm chuỗi                       | Cơ số ngẫu nhiên lúc chạy; hai modulo khi số chuỗi vượt cỡ $3 \cdot 10^4$ ; luôn nhớ đây là thuật toán xác suất.                                          |
| •• Vì sao vẫn học KMP dù có băm                 | Chắc chắn đúng; cho thông tin cấu trúc (biên, chu kỳ); và ý tưởng “dùng lại lần thất bại” tổng quát hơn nhiều so với một thuật toán.                      |
| •• Manacher                                     | Tái sử dụng đối xứng: bán kính tại $i$ khởi tạo từ bán kính tại điểm đối xứng qua tâm của chuỗi đối xứng phải nhất đã biết.                               |
| •• Trie                                         | Cây theo ký tự, mỗi đường là một tiền tố; dùng cho từ điển, gợi ý tự động, và làm nền cho Aho–Corasick.                                                   |
| • Suffix automaton / suffix tree                | Biểu diễn mọi chuỗi con trong $O(n)$ trạng thái; mạnh hơn mảng hậu tố nhưng khó cài — học sau khi đã chắc mảng hậu tố.                                    |

••• phải thuộc    •• nên nắm    • biết là có

### Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

#### 1. Tôi nên học KMP hay hàm Z trước?

| Hàm Z, vì định nghĩa của nó trực tiếp hơn — “đoạn khớp với tiền tố bắt đầu tại  $i$  dài bao nhiêu” — và cách tính dễ hình dung bằng hình. KMP có định nghĩa gián tiếp hơn qua khái niệm biên, nhưng

lại cần thiết khi bài hỏi về chu kỳ, về biên, hoặc khi cần xây automaton. Trong thực hành thi đấu, nhiều người dùng hàm Z cho việc so khớp và chỉ dùng  $\pi$  khi bài hỏi cấu trúc.

## 2. Khi nào tôi thực sự cần mảng hậu tố thay vì băm chuỗi?

| Khi bài hỏi về *tập hợp chuỗi con* chứ không về một mẫu cụ thể: đếm số chuỗi con phân biệt, sắp xếp các chuỗi con, tìm chuỗi con thứ  $k$  theo thứ tự từ điển. Băm giải tốt các bài “so sánh hai đoạn” và nhiều bài tìm kiếm có kết hợp nhị phân, nhưng nó không cho bạn một thứ tự trên tập chuỗi con. Quy tắc thực dụng: đề có chữ “phân biệt”, “thứ  $k$  theo từ điển”, “sắp xếp các chuỗi con” thì nghĩ tới mảng hậu tố.

## 3. Xây mảng hậu tố $O(n \log^2 n)$ có đủ dùng không?

| Hầu như luôn đủ với  $n$  tới vài trăm nghìn, và nó ngắn hơn hẳn bản  $O(n \log n)$  dùng sắp xếp đếm. Trong bài mà  $n$  tới  $10^6$ , hãy dùng bản  $O(n \log n)$ . Bản tuyến tính đẹp về lý thuyết nhưng dài và ít khi cần — đúng tinh thần “chọn cấu trúc yếu nhất còn đủ dùng”.

## 4. Aho–Corasick tốn bộ nhớ thế nào khi bảng chữ cái lớn?

| Nếu mỗi nút lưu một mảng chuyển trạng thái đầy đủ thì bộ nhớ là số nút nhân kích thước bảng chữ cái — với bảng Unicode thì không khả thi. Ba cách xử lý: dùng bảng băm cho các cạnh thay vì mảng; chỉ lưu cạnh thật và tra liên kết thất bại khi thiếu (chậm hơn một chút nhưng gọn hơn nhiều); hoặc ánh xạ bảng chữ cái về tập ký tự thực sự xuất hiện. Trong hệ thống thật, lựa chọn này quyết định việc automaton có nằm vừa trong cache hay không, và đó mới là yếu tố chi phối tốc độ.

## 5. Bài so khớp có cho phép sai khác vài ký tự thì dùng gì?

| Đó là bài toán khác hẳn và các công cụ trong chương này không giải trực tiếp. Với sai khác nhỏ, hướng chuẩn là quy hoạch động khoảng cách chỉnh sửa ( $C/16$ ), có thể tăng tốc bằng cách chỉ tính một dải quanh đường chéo khi số sai khác bị chặn. Với dữ liệu rất lớn như bộ gen, người ta dùng chiến lược “hạt giống rồi mở rộng”: tìm các đoạn khớp chính xác ngắn bằng chỉ mục, rồi mở rộng bằng căn chỉnh có chi phí — lại đúng kiến trúc lọc rỏ trước, kiểm đất sau.

## 6. Vì sao ký tự phân cách # trong mẹo $P + \# + T$ lại quan trọng?

| Vì nếu không có nó, một đoạn khớp có thể vắt qua ranh giới giữa mẫu và văn bản, tạo ra kết quả sai. Ký tự phân cách phải *không xuất hiện* trong cả mẫu lẫn văn bản, và trong bài dùng bảng chữ cái đầy đủ thì bạn phải chọn một giá trị ngoài bảng — đây là lỗi hay gặp khi bài cho phép mọi byte làm ký tự.

## 7. Có mối liên hệ nào giữa chương này và automaton trong lý thuyết tính toán không?

| Có, và nó khá trực tiếp. Hàm  $\pi$  của KMP chính là hàm chuyển trạng thái của một automaton hữu hạn nhận biết mẫu; Aho–Corasick là automaton nhận biết một tập mẫu; suffix automaton là automaton nhận biết mọi hậu tố của chuỗi. Nhìn theo góc này, cả chương là một chuỗi các cách xây automaton nhỏ gọn cho những ngôn ngữ rất cụ thể — và đó cũng là con đường mà Knuth đã đi tới KMP.

## ▮ Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                                     | Nguồn                     | Điều cần nhận ra                                                              |
|---------------------------------------------------------|---------------------------|-------------------------------------------------------------------------------|
| ● Tìm mọi vị trí xuất hiện của một mẫu                  | CSES “String Matching”[1] | Làm bằng cả ba cách: băm, hàm Z, KMP; so độ dài code và thời gian chạy.       |
| ● Tính hàm tiền tố và liệt kê mọi biên của chuỗi        | Bài kinh điển             | Lập $\pi[n], \pi[\pi[n]], \dots$ ; kiểm chứng bằng tay trên chuỗi ngắn.       |
| ● Kiểm tra một chuỗi có phải là phép quay của chuỗi kia | Bài kinh điển             | Nối chuỗi với chính nó rồi so khớp — một mẹo đáng nhớ.                        |
| ●● Chu kỳ nhỏ nhất của chuỗi                            | CSES nhóm bài chuỗi[1]    | $n - \pi[n]$ khi chia hết; hãy tự chứng minh điều kiện chia hết là cần thiết. |
| ●● Đếm số lần xuất hiện của mỗi mẫu trong một tập mẫu   | Bài kinh điển             | Aho–Corasick; cộng dồn theo cây liên kết thất bại.                            |

| Bài                                                                  | Nguồn                                    | Điều cần nhận ra (tiếp)                                                                    |
|----------------------------------------------------------------------|------------------------------------------|--------------------------------------------------------------------------------------------|
| •• Chuỗi con đối xứng dài nhất                                       | LeetCode “Longest Palindromic Substring” | Manacher; hoặc băm cộng nhị phân trên bán kính — so hai cách.                              |
| •• Tiền tố chung dài nhất của một tập chuỗi                          | Bài kinh điển                            | Trie hoặc so trực tiếp; bài đơn giản nhưng luyện thói quen xét trường hợp tập rỗng.        |
| ••• Chuỗi con lặp lại dài nhất                                       | Bài kinh điển                            | Mảng hậu tố + LCP, hoặc băm + nhị phân; hai cách cùng $O(n \log n)$ với hằng số rất khác.  |
| ••• Đếm số chuỗi con phân biệt                                       | CSES “Distinct Substrings”-loại[1]       | Tổng độ dài hậu tố trừ tổng LCP; tự dẫn ra công thức trước khi tra.                        |
| ••• Chuỗi con chung dài nhất của hai chuỗi                           | Bài kinh điển                            | Nối hai chuỗi với ký tự phân cách; cực đại LCP giữa hai hậu tố khác nguồn.                 |
| ••• Chuỗi con thứ $k$ theo thứ tự từ điển                            | Bài thi đấu kinh điển                    | Bài mà băm không giải được — lý do rõ ràng nhất để học mảng hậu tố hoặc suffix automaton.  |
| ••• Cài Aho–Corasick và dùng nó lọc một tập từ khoá trên văn bản lớn | Bài tập ứng dụng                         | Đo thời gian so với chạy KMP nhiều lần; số liệu sẽ cho thấy khác biệt rõ hơn mọi lập luận. |

## Kết chương và đọc tiếp

Phần B khép lại ở đây. Bảy chương vừa qua đều trả lời cùng một câu hỏi — *dữ liệu nên nằm ở dạng nào để thao tác tôi lập lại nhiều nhất trở nên rẻ* — và đều tuân theo cùng một nguyên tắc: chọn cấu trúc duy trì *đúng* lượng trật tự mà bài cần, không hơn. Heap biết ít hơn cây nên rẻ hơn; DSU biết ít hơn đồ thị nên gần như miễn phí; băm bỏ thứ tự nên nhanh hơn cây. Mỗi khả năng bạn đòi thêm đều có giá.

Phần C đổi câu hỏi lần nữa, và đây là câu hỏi trung tâm của cả quyển sách: *dùng khuôn tư duy nào để sinh ra lời giải*. Nó bắt đầu từ chỗ mọi lời giải bắt đầu — vết cạn — và chỉ ra rằng phần lớn thuật toán hay chỉ là vết cạn cộng thêm một quan sát cho phép bỏ bớt nhánh (C/13).

**Đọc thêm: công trình gốc** (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

| Mức         | Công trình và tác giả                                                       | Nơi công bố, năm                | Vì sao đọc, đọc phần nào                                                                                                                         |
|-------------|-----------------------------------------------------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| nền tảng    | Knuth, Morris, Pratt, “Fast Pattern Matching in Strings”                    | SIAM Journal on Computing, 1977 | Bài gốc của KMP. Đọc phần xây hàm thất bại — chỗ mà mọi tài liệu về sau đều rút gọn quá tay.                                                     |
| nền tảng    | Aho & Corasick, “Efficient String Matching: An Aid to Bibliographic Search” | Communications of the ACM, 1975 | Động cơ ban đầu là tra cứu thư mục; đọc để thấy vì sao cấu trúc phải là trie cộng liên kết thất bại.                                             |
| tra khi cần | Manber & Myers, “Suffix Arrays: A New Method for On-line String Searches”   | SIAM Journal on Computing, 1993 | Bài đưa mảng hậu tố thành lựa chọn thay cây hậu tố; đọc phần so sánh bộ nhớ.                                                                     |
| tra khi cần | Kärkkäinen, Sanders, Burkhardt, “Linear Work Suffix Array Construction”     | Journal of the ACM, 2006        | Thuật toán skew: dựng mảng hậu tố trong $O(n)$ bằng chia để trị theo phần dư của chỉ số. Đọc khi cần bản tuyến tính thay cho bản $O(n \log n)$ . |
| tra khi cần | Ukkonen, “On-line Construction of Suffix Trees”                             | Algorithmica, 1995              | Bản dựng cây hậu tố trực tuyến; tra khi cần chứ không cần thuộc.                                                                                 |

# Nguồn

---

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [4] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [5] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [6] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [7] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [8] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [9] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [10] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [11] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] AtCoder. “AC Library: Lazy Segtree.” (), address: [https://atcoder.github.io/ac-library/master/document\\_en/lazysegtree.html](https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html) (visited on 09/07/2026).
- [13] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [14] R. E. Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets,” *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [15] M. L. Fredman and M. E. Saks, “The Cell Probe Complexity of Dynamic Data Structures,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [16] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [17] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [18] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-line String Searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [3] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [4] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [5] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [6] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [7] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [8] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [9] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [10] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [11] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [12] AtCoder. “AC Library: Lazy Segtree.” (), address: [https://atcoder.github.io/ac-library/master/document\\_en/lazysegtree.html](https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html) (visited on 09/07/2026).
- [13] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [14] R. E. Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets,” *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [15] M. L. Fredman and M. E. Saks, “The Cell Probe Complexity of Dynamic Data Structures,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [16] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [17] A. V. Aho and M. J. Corasick, “Efficient String Matching: An Aid to Bibliographic Search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [18] U. Manber and G. Myers, “Suffix Arrays: A New Method for On-line String Searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.